

考虑变物性与径向收缩的药材 热湿传递及烘干时长预测

摘 要

针对圆柱形药材烘干过程中温湿分布不均、材料物性变化和失水收缩问题，建立由固定域向收缩域递进的径向热湿传递模型，计算规定位置的温度与干基含水率，并以全域最大含水率确定烘干时长。

针对问题一，从圆柱环层的热量与水分收支出发，建立常热物性导热方程和含水率相关的非线性扩散方程，中心采用对称边界，表面采用对流交换边界。对烘房实测数据作保形三次插值，使用表面加密有限体积法和隐式 BDF 求解，并以 Bessel–Duhamel 展开独立核验温度。1800 s 时，中心与表面温度分别为 33.5758、36.7879 °C，含水率分别为 2.5500、1.5102 kg/kg，表明预热期升温已传入内部，而失水主要集中在外层。

针对问题二，从初始状态统一采用附录 3 的经验物性，通过含水率对热容量、导热系数的影响及温度对扩散系数的影响构成双向耦合。3 h 时中心温度为 49.8494 °C、含水率为 1.7662 kg/kg。温度趋于均衡并不意味着水分已达到干燥要求，需继续计算非线性水分迁移过程。

针对问题三，将全部内部节点的最大含水率与 0.15 kg/kg 之差定义为事件函数，在前 4 h 观测输入及其后 50 °C、0.05 kg/kg 恒定环境假设下进行连续事件定位。区分临界根与严格低于阈值的报告时刻，结合时空加密结果取烘干时长为 **57.4731 h**；此时最大含水率为 0.1499993913 kg/kg，位于中心。

针对问题四，利用给定半径轨迹和比例径向收缩假设，由干物质与水质量守恒推导材料坐标方程。坐标变换保留内部扩散的 R^{-2} 因子与表面交换的 R^{-1} 因子，避免对干基含水率错误添加体积压缩项。采用附录 4 物性得到烘干时长 **51.0877 h**，终时半径为 1.2000 cm，最大含水率为 0.1499989333 kg/kg，计算未超出半径观测范围。同用附录 4 物性时，固定半径对照需 129.8448 h，收缩使预测时长减少 78.7571 h（约 60.65%）。

通过解析对照、退化情形、收支与时空加密检验数值实现。第三问另作轴对称二维校核，在端面与侧面交换系数相等的情景下，开放端面仅使配对达标时间提前约 8 s（0.004%），中心仍控制达标，支持保留径向主模型。所报时长为既定环境下的有效模型预测，潜热及界面平衡关系仍构成适用限制。

关键词：热湿传递；干基含水率；有限体积法；材料坐标；全域阈值

一 问题重述

1.1 问题背景

干燥是影响中药材成品品质的重要工序，热风烘干是常见的干燥方式，其过程包括预热平衡与恒温干燥两个阶段。工艺参数选取不当可能造成干燥效率低、能耗高和品质不稳定，因而需要通过数学模型刻画内部传热与水分迁移规律。

在题给变物性关系下，升温会增大水分扩散系数，含水率下降则改变密度、比热容、导热系数和扩散系数。表层与内部的失水进程不同，平均含水率不能直接代表所有位置均已达标。同时，失水引起的尺寸变化会改变实际迁移距离，需要在收缩条件下重新建立传递模型。

1.2 问题提出

某中药材近似为长 25 cm、初始半径 2 cm 的圆柱，初始温度为 28 °C、干基含水率为 2.55 kg/kg。附件 1 记录初期烘房温湿环境，附件 2 记录烘干中的半径变化，题目按问题分别给出物性参数。据此研究以下四个问题。

1. **问题一：**采用附录 2 参数，建立预热平衡阶段的温度与含水率模型，预测前 1800 s 的径向分布。在指定七个时刻、五个半径位置列出结果，并按 1 s、0.1 cm 间隔输出完整数据。
2. **问题二：**烘干过程一般持续 2 至 3 天，预热平衡与恒温干燥阶段的参数有所不同。全程统一采用附录 3 经验式建立温湿模型，给出前 3 h 每 0.5 h 的指定位置结果，并按 1 s、0.1 cm 间隔输出。
3. **问题三：**以药材各处含水率均低于 0.15 kg/kg 为干燥标准，确定累计烘干时长。列出每 6 h、每 0.5 cm 及结束时刻的含水率，并按 60 s、0.1 cm 间隔保存完整结果。
4. **问题四：**利用附件 2 的半径轨迹和附录 4 物性，重新确定尺寸变化条件下的烘干时长。按问题三的时间与距离间隔输出含水率，并在论文表格中单列当前药材表面。

四问依次考察预热传递、全过程变物性、全域干燥终点和收缩条件下的传递规律。完整结果分别保存至 `result1.xlsx` 至 `result4.xlsx`。所有结果表保留四位小数，数值推进和达标判定使用未舍入值。

二 问题分析

四问的共同核心是内部扩散与表面交换的共同作用。集中参数模型只能描述平均状态，难以直接满足题目对不同径向位置的要求。Hussain 和 Dincer 研究了圆柱湿物体干燥中的二维热湿传递 [1]；Kaya 等讨论了圆柱湿物体强制对流干燥的数值建模 [2]。本文保留径向空间分布，以题给物性建立有效模型，不移植其他材料的经验参数。

四问按预热阶段固定域、变物性耦合、全域达标、收缩材料域递进。其中第四问同时改变几何与物性，不能把它与第三问的结果差全部归因于收缩。烘房条件和半径轨迹属于输入，统一处理见第四节。各问均为初边值预测问题，不引入题目没有要求的工艺优化目标；均从题给初始状态计算，不拼接其他问题的末态。

2.1 问题一的分析

问题一要求计算固定半径下前 1800 s 药材各位置的温度与含水率。将药材沿半径分成环层，分别列出热量与水分收支：热量按 Fourier 导热，水分按有效 Fick 扩散，表面采用对流换热与等效传质边界，中心取零梯度。附录 2 的热物性均为常数，扩散系数仅依赖含水率，因此在不显式计入潜热与水分携热的近似下，两场可以分开求解。水分方程仍具有非线性，离散时需按当前状态更新扩散系数。数值上采用环形有限体积离散空间、隐式 BDF 推进时间，温度场另用 Bessel–Duhamel 展开独立校核。

2.2 问题二的分析

问题二与问题一的差别在物性关系和计算时间尺度。附录 3 的密度、比热容、导热系数均随含水率变化，扩散系数同时依赖含水率和局部温度。含水率影响温度传播，温度又影响水分迁移，故必须联合求解两个场。升温与失水对扩散系数的作用方向相反，不能将其冻结为单一常数。求解仍采用环形有限体积与隐式 BDF，在每次右端求值中更新物性，并用短时制造解检验耦合项和变系数离散。

2.3 问题三的分析

问题三沿用问题二的方程、物性和固定半径，将计算延伸至全域达标。判据应取径向最大含水率 $M(t) = \max_r C(r, t)$ ，避免以平均值或表面值提前结束。连续穿越阈值时的临界根满足等号，而题目要求严格低于 0.15 kg/kg，因此需另选稍晚的报告时刻，并在未舍入解上复核。表面附近的水分梯度及较低扩散系数要求足够的空间分辨率，求解采用表面加密网格、阈值附近局部精算，并直接检查事件时刻和全场结果的时空收敛。长时间端面影响由模型检验中的轴对称校核量化，主模型仍取径向一维。

2.4 问题四的分析

问题四将几何改为附件 2 给定的半径轨迹，物性改用附录 4。外半径不能唯一决定内部运动，需要先以比例径向收缩假设闭合材料速度，再由干物质与水分守恒引入材料坐标 $\xi = r/R(t)$ 。变换后内部扩散带 R^{-2} 、表面梯度带 R^{-1} ，二者均需保留。题给 ρc_p 作为有效热容量，干物质密度由质量守恒单独核算，保留经验密度与几何不完全兼容的适用限制。达标判据沿用问题三，但输出需映射回当前物理半径，域外位置标为“—”，当前表面单列。

三 模型假设与符号说明

3.1 模型假设及其用途

- (1) 药材及交换条件沿周向均匀，主模型忽略轴向梯度和端面交换，采用一维径向形式。第三问另作轴对称二维校核，假设物性各向同性、两端环境相同，端面交换系数与侧面相等，见第 7.7 节。
- (2) 初始温度与干基含水率均匀；问题一至三半径与长度固定。问题四长度仍取 0.25 m，骨架按给定 $R(t)$ 作比例径向收缩，干物质不流失。比例收缩用于由外半径轨迹闭

合内部速度场。

- (3) 题给 ρc_p 作为有效体积热容量, 采用 Fourier 导热和有效 Fick 扩散描述内部传递。热方程不显式包含汽化潜热、水分携焓及机械功。此处理用于在题给数据下封闭模型, 不代表这些机制已被实验排除。
- (4) 沿用 $h = 25 \text{ W}/(\text{m}^2 \cdot \text{K})$ 和 $h_m = 8 \times 10^{-7} \text{ m/s}$ 作为表面交换系数。同类圆柱干燥数值研究亦采用对流换热与对流传质的第三类边界 [2]。本题仅有常数系数, 故将 $h_m(C_s - C_a)$ 作为题给信息下的经验 Robin 闭合, 不把 C_a 认定为实测药材平衡含水率。气相与固相的 kg/kg 具有不同于基载体。
- (5) 固定域采用均匀、恒定的参考干物质密度; 收缩域按干物质守恒更新该密度。它与用于热方程的经验 $\rho(C)$ 分别记账, 不强制二者逐点满足湿密度换算关系。
- (6) 附件 1 仅覆盖前 4 h。鉴于末 1 h 烘房温度和水分量已趋于稳定, 问题三、四在 4 h 后采用 $T_a = 50 \text{ }^\circ\text{C}$ 、 $C_a = 0.05 \text{ kg/kg}$ 的恒定环境延续。该条件用于封闭长期预测, 并非后期实测数据, 其影响通过敏感性分析检验。

径向近似在问题一中可作尺度说明。初始长度与直径之比为 6.25, 两端总面积与侧面积之比为 0.08。附录 2 给出热扩散率 $\alpha = 0.36/(820 \times 2600) = 1.6886 \times 10^{-7} \text{ m}^2/\text{s}$, 1800 s 内的传播尺度 $\sqrt{\alpha t} = 0.0174 \text{ m}$, 小于中截面到端面的距离 0.125 m。这支持短时中截面的径向近似, 但面积比和传播尺度都不是后续数日预测的误差上界。

3.2 主要符号

符号	含义	单位
t, r	入炉后的时间、当前径向距离	s, m
$R, R(t), L$	固定半径、随时间变化的半径、长度	m
T, T_a	药材局部温度、烘房温度	$^\circ\text{C}$
T_K	药材局部绝对温度, $T_K = T + 273.15$	K
C, C_a	药材干基含水率、烘房题给水分量	kg/kg
ρ, c_p, k	有效密度、比热容、导热系数	$\text{kg}/\text{m}^3, \text{J}/(\text{kg} \cdot \text{K}), \text{W}/(\text{m} \cdot \text{K})$
$B = \rho c_p, D$	有效体积热容量、水分扩散系数	$\text{J}/(\text{m}^3 \cdot \text{K}), \text{m}^2/\text{s}$
h, h_m	对流换热系数、有效传质系数	$\text{W}/(\text{m}^2 \cdot \text{K}), \text{m/s}$
s, v	当前体积干物质密度、骨架径向速度	$\text{kg}/\text{m}^3, \text{m/s}$
ξ, U, Θ	材料坐标、该坐标下的含水率和温度	1, kg/kg, $^\circ\text{C}$
W_i, w_i	物理域、材料域控制体的径向权重	$\text{m}^2, 1$
M, t_*, t_{rep}	全域最大含水率、临界时刻、报告时刻	kg/kg, s, s

下标 s 表示当前表面, i 表示内部计算节点。正文中的时间公式以 s 为单位, 时长结果及表 4 至表 9 的时间列换算为 h。

四 数据预处理

为使四问模型使用一致的输入口径，本节统一整理附件 1 的烘房温湿环境和附件 2 的药材半径数据，将离散观测转换为数值求解所需的时间函数，并明确长期计算中的环境延续范围。

4.1 数据整理与单位统一

附件 1 给出时间、烘房温度和水分浓度，共 241 组观测，覆盖 0 至 14400 s，间隔 60 s。附件 2 给出时间与药材半径，共 145 组观测，覆盖 0 至 259200 s，即 72 h，间隔 1800 s，即 0.5 h。两份附件分别用于构造表面环境输入 $T_a(t), C_a(t)$ 和第四问的几何输入 $R(t)$ 。

计算中时间统一使用 s，半径由 cm 换算为 m；正文表格按题目要求再换算为 h 和 cm。温度观测保留摄氏度，扩散系数经验式中的绝对温度由药材局部温度加 273.15 得到。附件 1 中的烘房温度只作为边界输入，不代替药材内部温度。水分量保留题给 kg/kg 口径，其气固交换解释见模型假设。

问题一截取附件 1 前 1800 s 的 31 组观测，问题二截取前 10800 s 的 181 组观测，问题三、四使用前 14400 s 的全部 241 组观测。各问均保留相应区间的原始观测和小幅波动，不作全局平滑；环境插值区间与本问的计算口径保持一致。

4.2 烘房环境插值与时间延续

数值求解器需要在观测时刻之间计算表面环境，因此对各问所需观测区间采用保形分段三次 Hermite 插值 (PCHIP) [3]。记相邻观测点为 (t_j, f_j) ，区间长度为 $h_j = t_{j+1} - t_j$ ， $z = (t - t_j)/h_j$ ，则

$$\begin{aligned} f(t) = & (2z^3 - 3z^2 + 1)f_j + (-2z^3 + 3z^2)f_{j+1} \\ & + (z^3 - 2z^2 + z)h_j m_j + (z^3 - z^2)h_j m_{j+1}, \end{aligned} \quad (1)$$

其中 m_j 由保形规则确定， f 分别取烘房温度 T_a 和水分量 C_a 。式(1)精确通过观测点，所得时间函数在求解器需要的时刻提供边界输入。前 30 min 的环境观测插值见图 1，烘房升温不意味着药材内部同步升温。

附件 1 没有覆盖完整干燥时段。其最后 1 h 的温度和水分量均值分别为约 49.999 °C 和 0.04999 kg/kg，已接近平台。根据假设 (6)，问题三、四在 $t \leq 14400$ s 时保留全部观测输入，随后采用基于末段平台值的环境延续假设

$$T_a(t) = 50 \text{ } ^\circ\text{C}, \quad C_a(t) = 0.05 \text{ kg/kg}, \quad t > 14400 \text{ s}. \quad (2)$$

式(2)用于补全长期计算的边界条件，不是后期实测数据，也不是由 PCHIP 直接外推得到。切换时刻保留原观测终值，随后允许边界输入发生小幅变化，药材内部温湿状态保持连续。该延续假设的影响通过问题三的情景敏感性分析量化，并作为长期时长预测的适用限制保留。

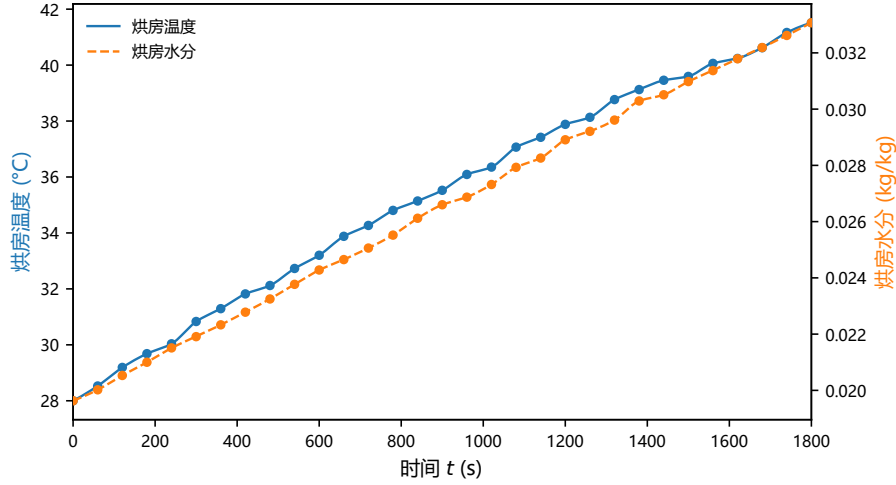


图 1 前 30 min 烘房温度和水分量随时间变化。曲线由附件 1 观测插值得到，输入升温不等于药材内部同步升温。

4.3 半径轨迹插值与数据覆盖范围

附件 2 的半径由 2 cm 降至 1.198 cm。将其换算为 m 后，采用与式(1)相同的插值方法构造

$$R(t) = \text{PCHIP}\{(t_j, R_j)\}, \quad 0 \leq t \leq 259200 \text{ s}. \quad (3)$$

图 2 给出半径观测散点与插值曲线，插值精确通过观测点。 $R(t)$ 仅在附件 2 的覆盖区间内使用，72 h 是数据覆盖终点，不是预设的烘干答案；第四问计算在该范围内达标，无需对半径作数据外推。

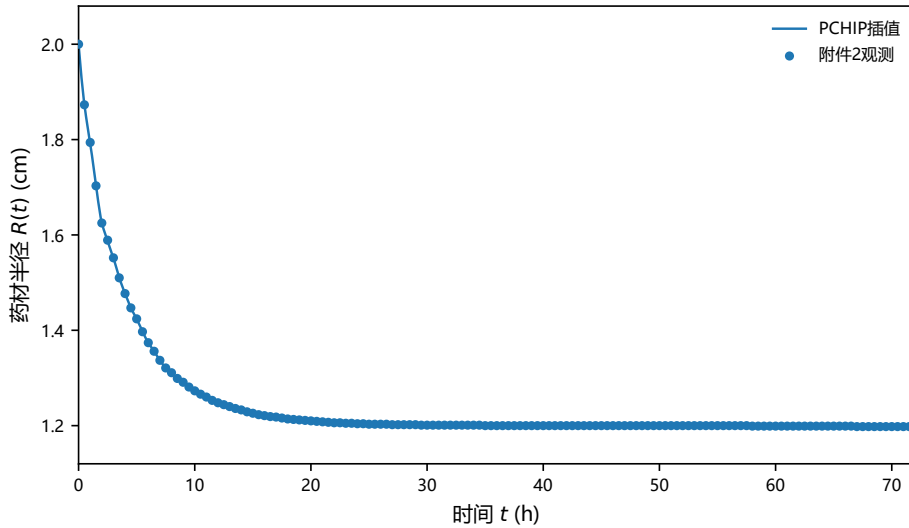


图 2 附件 2 半径观测及 PCHIP 插值。半径由 2 cm 降至 1.198 cm；72 h 为观测覆盖终点，不是预设烘干时长。

半径数据只确定外表面轨迹，不能唯一识别内部材料点的位移。第四问将以此输入建立比例径向收缩假设，并推导材料运动和热湿传递方程；这些模型关系不由数据插值

本身给出。

五 问题一：预热阶段的径向热湿传递

5.1 控制方程的建立

(1) 基于热量守恒的径向导热模型

烘干过程中，烘房通过药材表面向内部传递热量，使药材各位置的温度随时间变化。根据假设（1），药材满足轴对称，本文忽略轴向和周向的温度差异，将温度表示为径向位置与时间的函数 $T(r, t)$ 。在药材内部取半径为 r 至 $r + dr$ 、长度为 L 的薄圆柱环层，其体积为

$$dV = \pi[(r + dr)^2 - r^2]L \approx 2\pi r L dr.$$

以径向向外为热流的正方向。根据傅里叶导热定律，单位面积上的热流密度与温度梯度成正比、方向相反，即

$$q(r, t) = -k \frac{\partial T}{\partial r},$$

其中， k 为药材的导热系数。半径为 r 的圆柱面面积为 $2\pi r L$ ，因此通过该面的热流率为

$$Q(r, t) = 2\pi r L q(r, t) = -2\pi r L k \frac{\partial T}{\partial r}.$$

根据假设（3），在不计内部热源、汽化潜热及水分迁移携带热量的条件下，环层内的蓄热速率等于内外两侧的净流入热流率。由热量平衡可得

$$\rho c_p dV \frac{\partial T}{\partial t} = Q(r, t) - Q(r + dr, t).$$

将右端按 r 作一阶展开，并代入环层体积，得到

$$\rho c_p (2\pi r L dr) \frac{\partial T}{\partial t} = -\frac{\partial}{\partial r} (2\pi r L q) dr.$$

进一步代入傅里叶定律，约去公共因子，得到药材内部的径向热传导方程

$$\rho c_p \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r k \frac{\partial T}{\partial r} \right), \quad 0 < r < R. \quad (4)$$

问题一采用常数热物性参数 $\rho = 820 \text{ kg/m}^3$ 、 $c_p = 2600 \text{ J/(kg}\cdot\text{K)}$ 、 $k = 0.36 \text{ W/(m}\cdot\text{K)}$ 。令热扩散率 $\alpha = k/(\rho c_p)$ ，则式(4)可化为

$$\frac{\partial T}{\partial t} = \alpha \left(\frac{\partial^2 T}{\partial r^2} + \frac{1}{r} \frac{\partial T}{\partial r} \right).$$

其中，含 $1/r$ 的项反映了圆柱面传热面积随半径变化的几何特征。烘房对药材的加热作用通过外表面换热边界引入，内部温度分布则由上述方程确定。

(2) 基于水分守恒的非线性扩散模型

药材表面与烘房发生水分交换后，内部逐渐形成含水率梯度，水分随之由内部向表面迁移。题目中的 $C(r, t)$ 为干基含水率，即单位干物质质量所含的水质量。为建立水质

量守恒关系，按照假设（5），将单位体积中的干物质质量记为常数 s_0 ，则单位体积中的水质量为 $s_0 C$ 。

采用有效 Fick 扩散关系描述内部水分迁移。以径向向外为正方向，相对干物质骨架的水质量通量为

$$j(r, t) = -s_0 D(C) \frac{\partial C}{\partial r},$$

其中， $D(C)$ 为有效水分扩散系数，单位为 m^2/s 。当含水率由内向外降低时， $\partial C / \partial r < 0$ ，对应 $j > 0$ ，即水分向表面迁移。

沿用热传导模型的环层收支思路，水质量的变化率等于净流入的水质量流率。由于环层体积与参考干物质密度保持不变，有

$$s_0 (2\pi r L \, dr) \frac{\partial C}{\partial t} = -\frac{\partial}{\partial r} (2\pi r L j) \, dr.$$

代入扩散通量表达式，消去 s_0 和公共几何因子，得到

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left[r D(C) \frac{\partial C}{\partial r} \right], \quad D(C) = 7 \times 10^{-9} \exp\left(-\frac{0.89}{C}\right). \quad (5)$$

与常系数扩散不同，式(5)中的 D 随局部含水率变化，因而水分迁移过程具有非线性。将其展开可得

$$\frac{\partial C}{\partial t} = D(C) \left(\frac{\partial^2 C}{\partial r^2} + \frac{1}{r} \frac{\partial C}{\partial r} \right) + D'(C) \left(\frac{\partial C}{\partial r} \right)^2, \quad D'(C) = \frac{0.89 D(C)}{C^2} > 0.$$

因此，扩散系数不能作为常数直接移出微分算子。后续求解保留式(5)的通量形式，以便在离散过程中统一计算相邻环层之间的水分交换，并检验整体水分收支。

综上，问题一的热物性均为常数，水分扩散系数仅依赖含水率。在不显式考虑汽化潜热及水分携热的假设下，温度方程与水分方程相互独立，可分别结合各自的初始条件和表面交换条件进行求解。

5.2 初始条件与边界条件

入炉时药材各处温度和含水率均匀，题给初始状态为

$$T(r, 0) = 28, \quad C(r, 0) = 2.55, \quad 0 \leq r \leq R. \quad (6)$$

式(6)分别以 $^\circ\text{C}$ 和 kg/kg 为单位。根据径向对称性，中心处满足

$$\frac{\partial T}{\partial r}(0, t) = \frac{\partial C}{\partial r}(0, t) = 0.$$

这一条件表示中心温度和含水率的径向梯度为零；中心状态仍随内部传递过程变化。

在外表面，导热通量与对流换热通量相平衡，水分扩散通量与假设（4）中的等效水分交换通量相平衡。因此采用 Robin 边界

$$\begin{aligned} -k T_r(R, t) &= h [T_s - T_a(t)], \\ -D(C_s) C_r(R, t) &= h_m [C_s - C_a(t)]. \end{aligned} \quad (7)$$

其中, $T_s = T(R, t)$ 、 $C_s = C(R, t)$, $h = 25 \text{ W}/(\text{m}^2 \cdot \text{K})$ 、 $h_m = 8 \times 10^{-7} \text{ m/s}$ 。式(7)中, $T_a > T_s$ 时向外热流为负, 表示热量进入药材; $C_s > C_a$ 时水分向外迁移。表面温度和含水率由内部传递与外部交换共同决定, 因而通常与烘房状态不同。

环境函数 $T_a(t)$ 和 $C_a(t)$ 采用第四节整理的前 1800 s 观测插值, 在求解器所需时刻计算边界输入。这里的 $h_m(C_s - C_a)$ 仍按等效交换关系使用, C_a 的含义与模型假设保持一致。

均匀初始水分使表面初始梯度为零, 而交换边界在接触烘房后立即产生失水通量, 二者在 $t = 0$ 角点不相容。本文保留题给初值, 在 $t > 0$ 施加表面交换, 求得初始表面层的瞬态发展, 并把前 1 s 的表面结果纳入网格检验。

5.3 模型的数值求解

(1) 空间离散

预热初期, 药材表面含水率变化较快, 较陡的径向梯度主要出现在外层。为分辨这一变化, 采用节点型有限体积法, 在靠近表面处逐步加密网格, 并使题目规定的 0、0.1、...、2.0 cm 输出位置均为实际节点。将每个 0.1 cm 基本区间细分, 在 0 至 1.5、1.5 至 1.8、1.8 至 1.9、1.9 至 2.0 cm 四段内, 分别细分为 8m、16m、32m、64m 份, 记所得网格为 G_m , 共有 $264m + 1$ 个节点。本问采用 G_8 , 即 2113 个节点, 并通过加密比较检验空间离散误差。

设全部节点依次为 r_i ($i = 0, \dots, N$), 内部面为 $r_{i+1/2} = (r_i + r_{i+1})/2$ 。控制体左右端点为 a_i, b_i^* , 中心端取 0、表面端取 R , 定义

$$W_i = \int_{a_i}^{b_i^*} r \, dr = \frac{(b_i^*)^2 - a_i^2}{2}, \quad \sum_{i=0}^N W_i = \frac{R^2}{2}. \quad (8)$$

真实控制体体积为 $2\pi L W_i$ 。式(8)保留了不同半径处环层体积的差异, 中心与表面处的控制体均在物理边界截取。

(2) 通量处理与半离散方程

记 $D_i = D(C_i)$, 在相邻节点之间对扩散系数取调和平均, 得到已乘半径的向外通量

$$\begin{aligned} F_{i+1/2}^T &= -r_{i+1/2} k \frac{T_{i+1} - T_i}{r_{i+1} - r_i}, \\ F_{i+1/2}^C &= -r_{i+1/2} \frac{2D_i D_{i+1}}{D_i + D_{i+1}} \frac{C_{i+1} - C_i}{r_{i+1} - r_i}. \end{aligned} \quad (9)$$

中心控制体在 $r = 0$ 处的边界通量取零, 表面通量直接取 $Rh(T_N - T_a)$ 、 $Rh_m(C_N - C_a)$ 。对式(4)和式(5)作控制体积分, 并用节点状态近似体平均, 得到

$$\rho c_p W_i \dot{T}_i = F_{i-1/2}^T - F_{i+1/2}^T, \quad W_i \dot{C}_i = F_{i-1/2}^C - F_{i+1/2}^C. \quad (10)$$

式(9)在相邻控制体中使用同一内部面通量, 使其在式(10)中以相反符号出现, 对所有控制体求和后仅保留表面交换项。

中心处采用控制体积分，避免直接计算 $1/r$ 。若首个节点间距为 $\delta = r_1 - r_0$ ，则 $W_0 = \delta^2/8$ ，相应温度变化率为 $\dot{T}_0 = 4\alpha(T_1 - T_0)/\delta^2$ ，水分方程具有同样的几何系数。表面节点则保留自身蓄积项，同时接收内部面通量和外部交换通量，从而直接求得真实表面状态。

(3) 时间推进与结果输出

空间离散后，两场分别形成常微分方程组，统一记为 $\dot{\mathbf{y}} = \mathbf{f}(t, \mathbf{y})$ ，其中 $\mathbf{y}$ 分别取全部节点的温度或含水率。表面加密使显式方法受到较小稳定步长的限制，因此采用隐式后向差分公式 (BDF) 进行时间积分 [4]。

以等步长二阶 BDF 为例，下一时刻的状态应满足

$$\mathcal{R}(\mathbf{y}^{n+1}) = 3\mathbf{y}^{n+1} - 4\mathbf{y}^n + \mathbf{y}^{n-1} - 2\Delta t \mathbf{f}(t_{n+1}, \mathbf{y}^{n+1}) = 0. \quad (11)$$

式(11)中的历史状态 $\mathbf{y}^n$ 、 $\mathbf{y}^{n-1}$ 已知，但通量也依赖待求的 $\mathbf{y}^{n+1}$ ，因而需要求解隐式代数方程组。对水分场， $D(C)$ 随未知含水率变化，方程组具有非线性。计算时先由历史状态预测新状态，再进行牛顿型迭代：按试探状态更新扩散系数和通量，计算方程残差，求解线性修正方程并更新状态，直至满足收敛要求。

步内迭代收敛后，还需估计局部时间误差；误差满足容差时才接受该时间步，否则缩小步长重算。实际调用 SciPy 的变步长、变阶 BDF 实现，式(11)仅用于说明其隐式离散形式，计算并不固定采用二阶公式。

本问相对容差设为 10^{-10} ，温度与含水率绝对容差分别为 10^{-10} 、 10^{-12} ，最大步长为 0.5 s。温度场与水分场分别积分；水分方程的每次右端函数求值均按当前含水率更新 $D(C)$ ，线性修正方程的求解利用相邻节点耦合形成的稀疏结构。

在 0 至 1800 s 内完成积分后，利用求解器的连续输出提取题定时刻的状态，并读取相应径向节点。完整结果按 1 s、0.1 cm 间隔保存，题定表格从中选取指定时刻与位置。每秒输出是数据保存要求，求解器的内部步长仍由误差控制自适应选取。

5.4 计算结果与分析

表 1、表 2 给出题定时刻与位置的温度和含水率，图 3 展示相应径向剖面，用于比较预热过程中不同位置的升温与失水情况。

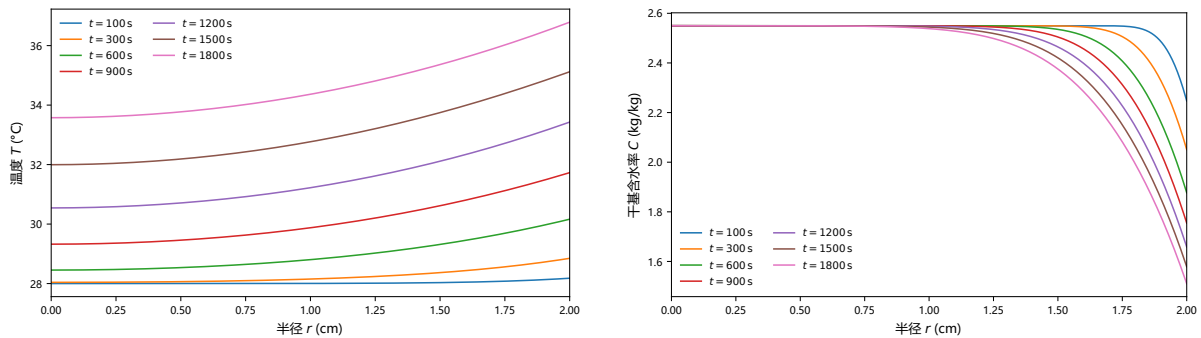


图 3 预热阶段温度与含水率径向剖面。外层先升温、先失水，中心含水率在 30 min 内变化很小；不同曲线对应题定输出时刻。

表 1 30 分钟内药材的温度 (°C)

时间/s	到药材中心的距离/cm				
	0	0.5	1.0	1.5	2.0
100	28.0000	28.0003	28.0038	28.0319	28.1795
300	28.0406	28.0634	28.1513	28.3673	28.8486
600	28.4533	28.5361	28.8039	29.3155	30.1641
900	29.3243	29.4583	29.8757	30.6164	31.7295
1200	30.5429	30.7099	31.2225	32.1130	33.4295
1500	31.9960	32.1871	32.7667	33.7476	35.1211
1800	33.5758	33.7724	34.3642	35.3626	36.7879

表 2 30 分钟内药材的水分浓度 (kg/kg)

时间/s	到药材中心的距离/cm				
	0	0.5	1.0	1.5	2.0
100	2.5500	2.5500	2.5500	2.5500	2.2470
300	2.5500	2.5500	2.5500	2.5492	2.0508
600	2.5500	2.5500	2.5500	2.5353	1.8770
900	2.5500	2.5500	2.5497	2.5045	1.7547
1200	2.5500	2.5500	2.5482	2.4646	1.6586
1500	2.5500	2.5499	2.5445	2.4206	1.5787
1800	2.5500	2.5497	2.5383	2.3755	1.5102

温度分布体现了由外向内的升温过程。至 1800 s，表面温度为 36.7879 °C，中心温度为 33.5758 °C，两者相差 3.2121 °C；表面仍低于同期烘房温度 41.5130 °C。热量先经表面对流进入药材，再通过内部导热向中心传播，有限的表面换热和内部导热共同造成了这一温度滞后。

水分变化主要集中在外层。末时刻表面含水率降至 1.5102 kg/kg，而中心未舍入值为 2.5499924 kg/kg，保留四位小数仍为 2.5500 kg/kg。表面持续向烘房排湿，首先形成含水率梯度；在前 30 min 内，内部水分向外迁移尚不充分，因此中心变化较小，径向剖面表现为中心高、表面低。

含水率下降还会改变后续迁移速率。由 $D'(C) > 0$ 可知，失水使局部扩散系数减小。本问 D 由初始 4.9377×10^{-9} 降至局部最小 $3.8830 \times 10^{-9} \text{ m}^2/\text{s}$ ，表明在相同含水率梯度下，较干区域的水分传递能力有所降低。与冻结初始 D 的计算相比，含水率最大差达到 0.0419 kg/kg，因而结果分析和正式求解均需保留 $D(C)$ 的变化。

5.5 模型检验

为检验数值结果的可靠性, 分别考察网格与时间参数改变后的结果差异、温度数值解与独立参考解的偏差, 以及累计热量和水分收支是否一致。

(1) 空间与时间加密检验

依次采用 G_2 、 G_4 、 G_8 三级网格, 节点数分别为 529、1057 和 2113, 保持物性、环境输入及时间积分参数一致。在 1 至 1800 s 的整秒时刻和 21 个规定径向位置比较相邻两级解, 以全部比较点上的最大绝对差衡量结果变化, 见表 3。同时, 在生产网格 G_8 上将相对、绝对容差均缩小为原来的 1/5, 最大步长由 0.5 s 减为 0.25 s, 单独考察时间积分参数的影响。

表 3 问题一空间与时间加密结果

比较项	$\max \Delta T $ (°C)	$\max \Delta C $ (kg/kg)	最大差位置
G_2/G_4	2.289×10^{-6}	1.828×10^{-5}	T : 684 s, 1.5 cm; C : 1 s, 2.0 cm
G_4/G_8	5.895×10^{-7}	4.564×10^{-6}	T : 678 s, 1.5 cm; C : 1 s, 2.0 cm
G_8 时间收紧	2.874×10^{-8}	1.179×10^{-9}	T : 1501 s, 2.0 cm; C : 1 s, 2.0 cm

从 G_2/G_4 到 G_4/G_8 , 温度和含水率最大差均约缩小至原来的四分之一, 表现出随网格加密而减小的趋势。含水率最大差均位于 1 s 时的表面, 说明早期表面瞬态对空间分辨率较为敏感。时间参数收紧引起的变化小于相应空间加密差, 支持当前时间设置下结果的稳定性。表中差值为不同计算配置之间的比较结果, 不作为连续方程真解误差的严格上界。

题定 7 组时刻、5 个位置的温度与含水率, 在 G_4/G_8 及时间收紧比较下均保留四位小数一致。完整逐秒输出仍有少量末位舍入变化, 因此“四位稳定”的结论限定于题定表格。

(2) 独立参考解对照

问题一的温度方程为常系数线性方程, 可以构造独立于径向有限体积离散的 Bessel–Duhamel 参考解。记 J_0 、 J_1 分别为零阶、一阶第一类 Bessel 函数, 对导热方程作分离变量, 表面换热条件给出模态特征方程

$$\lambda_n J_1(\lambda_n) = \text{Bi } J_0(\lambda_n), \quad \text{Bi} = \frac{hR}{k}. \quad (12)$$

由式(12)确定各阶模态, 结合 $T_a(0) = 28$, 得到温度参考解

$$T_{\text{ref}}(r, t) = T_a(t) - \sum_{n=1}^{\infty} A_n J_0(\lambda_n r/R) \int_0^t e^{-\beta_n(t-\tau)} T'_a(\tau) d\tau, \quad (13)$$

其中, $\beta_n = \alpha \lambda_n^2 / R^2$, 模态系数为

$$A_n = \frac{2J_1(\lambda_n)}{\lambda_n [J_0^2(\lambda_n) + J_1^2(\lambda_n)]}.$$

式(13)使用相同的 PCHIP 环境输入，通过模态常微分方程计算卷积积分，详细推导见完整版本附录 C。先将级数项数由 480 增至 960，参考解的最大变化为 $2.7632 \times 10^{-9} \text{ }^{\circ}\text{C}$ ；再将 960 项参考解与有限体积解比较，全部规定输出点上的最大温度差为 $2.1547 \times 10^{-7} \text{ }^{\circ}\text{C}$ 。级数加倍带来的变化明显小于两种方法之间的差异，支持该参考解用于检查所选径向导热方程及换热边界的数值实现。

对水分方程，另设冻结 $D = D(C_0)$ 的辅助算例，并与相同初边值条件下的 Bessel 参考解比较，最大差为 $1.5090 \times 10^{-6} \text{ kg/kg}$ 。该结果支持圆柱控制体几何、常系数扩散通量及 Robin 边界的离散一致性。正式计算仍采用变 $D(C)$ ，其数值可靠性由网格与时间加密、非线性水分收支检验共同评价。

(3) 热量与水分收支检验

对式(10)在所有控制体上求和，内部面通量成对抵消，有效蓄热增量应等于累计表面热输入，水分存量的减少应等于累计表面排湿。分别定义

$$\begin{aligned}\mathcal{E}_T(t) &= 2\pi L \rho c_p \sum_i W_i [T_i(t) - T_i(0)], \\ Q_T(t) &= 2\pi R L \int_0^t h [T_a(\tau) - T_s(\tau)] d\tau, \\ M_C(t) &= \sum_i W_i C_i(t), \quad L_C(t) = R \int_0^t h_m [C_s(\tau) - C_a(\tau)] d\tau.\end{aligned}\tag{14}$$

式(14)中的量应满足 $\mathcal{E}_T = Q_T$ 和 $\Delta M_C = -L_C$ ，其中 $\Delta M_C(t) = M_C(t) - M_C(0)$ 。 M_C 为归一化水分存量，乘以 $2\pi L s_0$ 后才对应水质量。为比较收支偏差与实际交换量的大小，定义相对残差

$$\varepsilon_T = \frac{|\mathcal{E}_T - Q_T|}{\max(|\mathcal{E}_T|, |Q_T|, \eta_T)}, \quad \varepsilon_C = \frac{|\Delta M_C + L_C|}{\max(|\Delta M_C|, |L_C|, \eta_C)}.\tag{15}$$

其中， $\eta_T = 10^{-12} \text{ J}$ 、 $\eta_C = 10^{-18} \text{ m}^2 \cdot (\text{kg/kg})$ 为防止分母接近零的数值下限。

利用求解器的连续输出，在 PCHIP 观测分区间上对表面通量作自适应积分，再与末态存量独立比较。在 $t = 1800 \text{ s}$ 处，前 1800 s 累计收支的相对残差为

$$\varepsilon_T = 2.0484 \times 10^{-9}, \quad \varepsilon_C = 2.4254 \times 10^{-11}.$$

按式(15)计算的两个残差均低于所设 10^{-6} 的数值检查阈值，表明常热物性温度场和正式非线性水分场的末时累计收支与边界交换一致。这些数值对应 1800 s 末时检查，不是全过程残差的最大值。

综合三类检验，所报结果在现有加密配置下保持稳定，并与独立温度参考解及所建方程的累计收支相符。该结论针对一维等效模型的数值求解；端面、潜热及等效水分边界等物理简化的预测误差，仍需借助额外观测或扩展模型评估。

六 问题二：变物性条件下的热湿耦合

6.1 统一物性与耦合方程

问题二建立整个烘干过程的变物性模型，全程统一使用附录 3 经验式，本问给出前 3 h 的结果。因此保留问题一的固定几何、原始初值与边界结构，从 $t = 0$ 重新积分。局部物性为

$$\begin{aligned}\rho(C) &= 650 + 128C, \\ c_p(C) &= 1450 + 2736 \frac{C}{1+C}, \\ k(C) &= 0.21 + 0.38 \frac{C}{1+C}, \\ D(C, T) &= 2.4 \times 10^{-3} \exp\left(-\frac{0.45}{C}\right) \exp\left[-\frac{3850}{T + 273.15}\right].\end{aligned}\quad (16)$$

其中 ρ 、 c_p 、 k 和 D 的单位分别为 kg/m^3 、 $\text{J}/(\text{kg}\cdot\text{K})$ 、 $\text{W}/(\text{m}\cdot\text{K})$ 和 m^2/s ， C 为干基含水率，单位为 kg/kg 。温度场变量 T 按 $^\circ\text{C}$ 表示，扩散系数指数中的 $T + 273.15$ 为药材局部绝对温度，单位为 K 。

问题二仍采用与问题一相同的固定圆柱环层作为控制体，环层几何和水分守恒结构不变。附录 3 将热物性与扩散系数改为局部温湿状态的函数， $\rho(C)c_p(C)$ 仅作为有效体积热容量 $B(C)$ ，固定域水分守恒仍按参考干物质骨架计量。根据假设 (3)，仍不显式引入汽化潜热和水分携焓，故在相同守恒结构下更新本构关系并联合求解。将式(16)代入该环层模型，得到

$$\begin{aligned}B(C)T_t &= \frac{1}{r} \partial_r [rk(C)T_r], \\ C_t &= \frac{1}{r} \partial_r [rD(C, T)C_r], \\ B(C) &= \rho(C)c_p(C).\end{aligned}\quad (17)$$

初始条件沿用式(6)，中心仍采用零梯度条件；表面边界沿用式(7)并将 k, D 替换为当前表面物性。题目附录 3 未另给交换系数，按假设 (4) 继续使用前述 h, h_m 。

式(17)中的耦合有两个方向： C 改变 B 和 k ，从而改变温度传播； T 改变 D ，从而改变水分迁移。进一步展开水分扩散项可得

$$C_t = D \left(C_{rr} + \frac{C_r}{r} \right) + D_C (C_r)^2 + D_T T_r C_r. \quad (18)$$

式(18)中的 $D_T T_r C_r$ 由空间变化的扩散系数产生，不是另加的热扩散经验项。采用原扩散通量形式离散，可同时保留上述非线性项。

6.2 有效热容量与干物质守恒核算

问题二沿用固定参考干物质骨架，水分方程中的参考干密度保持恒定。经验 $\rho(C)$ 仅用于构造 $B(C)$ ，不能在水分守恒中又取 $s = \rho(C)/(1 + C)$ 并随时间更新，否则会与固定体积中干物质质量不变的假设冲突。

同样， $B(C)T_t$ 不能直接替换为 $\partial_t [B(C)T]$ 。取初始参考温度 $T_{\text{ini}} = 28^\circ\text{C}$ ，有

$$\partial_t [B(C)(T - T_{\text{ini}})] = B(C)T_t + B'(C)(T - T_{\text{ini}})C_t. \quad (19)$$

式(19)中的第二项反映所定义状态量随组成变化而改变。本文采用以温度形式表达的有效热方程，在收支检验中保留修正项，但这并不代表已经描述完整多相系统的能量交换。

6.3 联合求解与结果

沿用问题一的环形控制体，热导率与扩散系数均取相邻节点的调和平均。将 $\mathbf{y} = (T_0, \dots, T_N, C_0, \dots, C_N)^T$ 作为联合状态，其中 T_i 和 C_i 分别表示第 i 个节点的温度和干基含水率；这里的下标 0 表示节点编号，不表示参考温度。在每次右端函数求值及隐式迭代中，按局部温湿状态更新物性。每个节点只与相邻节点的两个场相联系，利用这一稀疏结构进行 BDF 积分。生产网格采用 G_4 ，相对容差 10^{-9} ，温度、含水率绝对容差分别为 10^{-8} 、 10^{-10} ，最大步长 1 s。

表 4 和表 5 给出每 0.5 h 的论文摘录结果，图 4 展示相应径向剖面，用于比较烘干过程中不同位置的升温与失水情况；完整的逐秒、逐 0.1 cm 结果另存于 `result2.xlsx` 中。

表 4 3 小时内药材的温度 (°C)

时间/h	到药材中心的距离/cm				
	0	0.5	1.0	1.5	2.0
0.5	32.1896	32.3824	32.9660	33.9608	35.4132
1	40.3822	40.5542	41.0609	41.8783	43.0000
1.5	45.8471	45.9351	46.1932	46.6055	47.1397
2	48.4503	48.4881	48.5985	48.7736	49.0033
2.5	49.4670	49.4793	49.5136	49.5645	49.6617
3	49.8494	49.8553	49.8746	49.9096	49.9645

表 5 3 小时内药材的水分浓度 (kg/kg)

时间/h	到药材中心的距离/cm				
	0	0.5	1.0	1.5	2.0
0.5	2.5499	2.5489	2.5256	2.3257	1.6486
1	2.5257	2.4948	2.3578	2.0230	1.4711
1.5	2.3861	2.3256	2.1344	1.8020	1.3476
2	2.1708	2.1084	1.9236	1.6259	1.2311
2.5	1.9566	1.9006	1.7360	1.4720	1.1166
3	1.7662	1.7165	1.5702	1.3333	1.0081

3 h 时中心温度为 49.8494 °C，已接近烘房温度，但中心含水率仍为 1.7662 kg/kg，远高于 0.15 kg/kg 的干燥要求。由此可见，温度接近稳定不能作为停止烘干的条件。

问题一和问题二在相同时间的结果差异主要源于物性设定和耦合方式不同：问题一

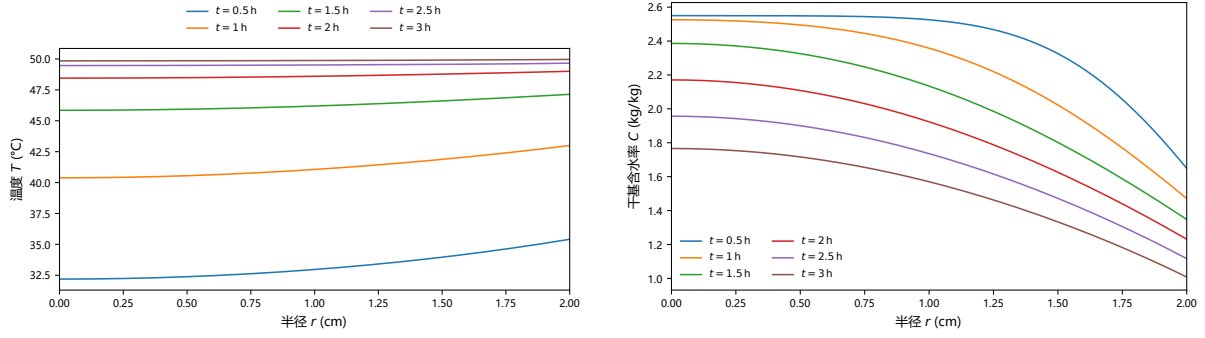


图 4 前 3 h 变物性模型的温度与含水率剖面。温度逐渐接近烘房水平，水分仍保持中心高、表面低的径向差异。

将温度场与干基含水率场分别求解，问题二则对两个场进行双向耦合求解。问题二不能在 1800 s 处读取问题一末态后才切换参数，否则前 0.5 h，即 0 ~ 1800 s 的过程不再满足题目要求的统一经验式。按题目输出要求，从 1 s 到 10800 s 以 1 s、0.1 cm 采样，温度场和干基含水率场各含 10800×21 个数值，合计 453600 个数值。

6.4 升温与失水对扩散系数的竞争

由式(16)直接求导可得

$$\frac{\partial \ln D}{\partial C} = \frac{0.45}{C^2} > 0, \quad \frac{\partial \ln D}{\partial T} = \frac{3850}{(T + 273.15)^2} > 0. \quad (20)$$

升温倾向于增大扩散系数，失水则使其减小，两种作用沿实际轨迹竞争。固定 $C = 2.55$ 时，由 28 升至 50 °C 使 D 增至约 2.388 倍；固定 $T = 50$ °C 时， C 从 2.55 降至 0.15 使 D 降至 $C = 2.55, T = 50$ °C 时 D 值的约 5.94%。这些是对经验式的条件性代入，不是独立温度控制实验。

数值结果中前 3 h 局部 D 约在 5.5476×10^{-9} 至 $1.2522 \times 10^{-8} \text{ m}^2/\text{s}$ 之间变化。因各位置升温 and 失水程度不同，不能用单一常数代表整个时空场。与固定为初始值 D_0 的对照算例相比，3 h 内干基含水率最大差为 0.3981 kg/kg，说明将 D 简化为常数会明显改变干基含水率场，正式计算应保留变化的 D 。至于其对烘干时长的影响，还需要在问题三中通过临界时间进一步验证。

七 问题三：全域达标条件与烘干时间

7.1 模型承接与长期烘干环境

问题二给出了变物性条件下药材温度和水分浓度的时空变化规律。问题三进一步要求确定各处水分浓度均低于 0.15 kg/kg 的烘干时长，因此沿用式(16)–(17)及其初边值条件，将计算延伸至全域达标。药材半径仍为 $R = 0.02 \text{ m}$ ，采用固定圆柱的一维径向模型，不在本问引入收缩。虽然题目只要求输出水分浓度，计算中仍联合求解温度场，因为局部扩散系数 $D(C, T)$ 随温度变化。

长期环境输入采用第四节的统一处理：前 4 h 使用附件 1 的全部观测插值，之后按

假设（6）及式(2)作恒定延续。所得时长以该环境延续情景为条件，药材内部温湿状态在切换处连续接续。

累计烘干时间从最初入炉计起。本问从 $T(r, 0) = 28\text{ }^{\circ}\text{C}$ 、 $C(r, 0) = 2.55\text{ kg/kg}$ 独立积分全过程，以保持物性、环境输入与初始条件的一致性。

7.2 全域达标判据的建立

题目要求药材各处均低于阈值，故定义径向最大水分浓度及事件函数

$$M(t) = \max_{0 \leq r \leq R} C(r, t), \quad g(t) = M(t) - 0.15, \quad t_* = \inf\{t \geq 0 : M(t) < 0.15\}. \quad (21)$$

其中 t_* 表示首次达标的临界时间。使用 $M(t)$ 可避免表面或平均水分浓度已经达标、内部仍存在湿区时提前结束计算。离散求解中取全部内部网格节点的最大值 $M_h(t) = \max_i C_i(t)$ ，同时记录最大值位置并检查径向分布。表格中的 5 个半径和 Excel 中的 21 个半径仅用于输出采样；只有计算结果支持中心最湿时，才将中心值作为控制位置的解释。

对于连续向下穿越阈值的过程，临界时刻满足 $M(t_*) = 0.15$ 。题目要求严格低于阈值，因此另取稍晚的报告时刻 t_{rep} ，并用未舍入结果复核 $M_h(t_{\text{rep}}) < 0.15$ 。这里的全域指一维径向模型中的 $0 \leq r \leq R$ ，其适用范围受忽略轴向变化的假设限制。

7.3 数值求解与达标事件定位

空间离散沿用问题二的径向环形有限体积法，保留变系数散度结构，并根据局部温湿状态更新物性。采用表面加密的 G_8 网格，共 2113 个节点，以隐式 BDF 方法联合推进温度和水份浓度。相对容差取 10^{-9} ，温度和水份浓度的绝对容差分别为 10^{-8} 和 10^{-10} ；前 4 h 最大步长为 2 s，之后为 60 s，并在环境切换处分段接续。

积分过程中监测 $g_h(t) = M_h(t) - 0.15$ ，搜索其由正到非正的穿越区间。在阈值附近以最大 0.5 s 步长局部精算，再利用连续数值插值和带括号求根定位 t_* 。因此，每 60 s 的文件输出间隔不限制事件定位的时间精度。

严格达标时刻的选取还需考虑离散差异。根据空间加密和时间容差收紧的结果设置经验裕量 ε_C ，先寻找 $M_h(t) < 0.15 - \varepsilon_C$ 的时刻，再向上选择整秒并向上保留四位小时。最后在生产解、细网格解和时间收紧解上核对同一报告时刻的最大水分浓度。裕量的定义及数值核验见第 7.6 节。

7.4 烘干时长与水分分布结果

生产网格得到的临界时刻为 $t_* = 57.4725195022\text{ h}$ 。按上述严格达标处理，药材所需烘干时长为

$$t_{\text{rep},3} = 57.4731\text{ h} = 206903.16\text{ s}. \quad (22)$$

考虑到 4 h 后的烘房条件来自假设（6），进一步进行小规模情景敏感性分析。前 4 h 仍使用附件 1 数据。后期温度和水分量按指定情景改变；有效传质系数则在全程按比例缩放。水分量 $\pm 10\%$ 是对后期延续假设的工程扰动，大于附件 1 末 1 h 的观测波动。其余数值设置和全域阈值判据保持不变。

表 6 问题三长期输入敏感性分析

情景	$T_a / ^\circ\text{C}$	$C_a / \text{kg/kg}$	h_m/h_m^0	达标时间 / h	相对变化
基准	50	0.050	1.0	57.4725	0.00%
$T_a - 2^\circ\text{C}$	48	0.050	1.0	61.2927	+6.65%
$T_a + 2^\circ\text{C}$	52	0.050	1.0	53.9791	-6.08%
$C_a - 10\%$	50	0.045	1.0	57.3815	-0.16%
$C_a + 10\%$	50	0.055	1.0	57.5826	+0.19%
$h_m - 20\%$	50	0.050	0.8	59.1690	+2.95%
$h_m + 20\%$	50	0.050	1.2	56.4333	-1.81%

结果表明，后期温度延续情景可使达标时间变化约 6%，全程传质系数情景约 2–3%，后期水分量 $\pm 10\%$ 仅约 0.2%，均明显大于数值离散误差。因此报告时长的末位反映给定情景下的事件定位精度，不代表实际烘干时间具有同等的输入确定性。在基准报告时刻，生产解的最大水分浓度为 0.1499993913 kg/kg，严格低于 0.15 kg/kg，最大值位于中心。

表 7 列出每 6 h 及结束时刻的水分浓度，图 5 展示相应径向剖面。第 7.7 节进一步比较一、二维模型的最大含水率历程，检验端面对达标时间的影响。

表 7 固定半径模型药材烘干过程的水分浓度 (kg/kg)

时间/h	到药材中心的距离/cm				
	0	0.5	1.0	1.5	2.0
6	1.0170	0.9881	0.9015	0.7550	0.5328
12	0.4561	0.4432	0.4031	0.3297	0.1642
18	0.2988	0.2915	0.2687	0.2253	0.0845
24	0.2378	0.2327	0.2167	0.1855	0.0665
30	0.2058	0.2018	0.1892	0.1641	0.0598
36	0.1859	0.1825	0.1718	0.1504	0.0566
42	0.1721	0.1691	0.1597	0.1407	0.0549
48	0.1618	0.1592	0.1507	0.1334	0.0537
54	0.1539	0.1514	0.1436	0.1277	0.0530
57.4731	0.1500	0.1477	0.1402	0.1249	0.0527

注：终时中心未舍入值为 0.1499993913 kg/kg，严格低于 0.15；0.1500 为四位显示。

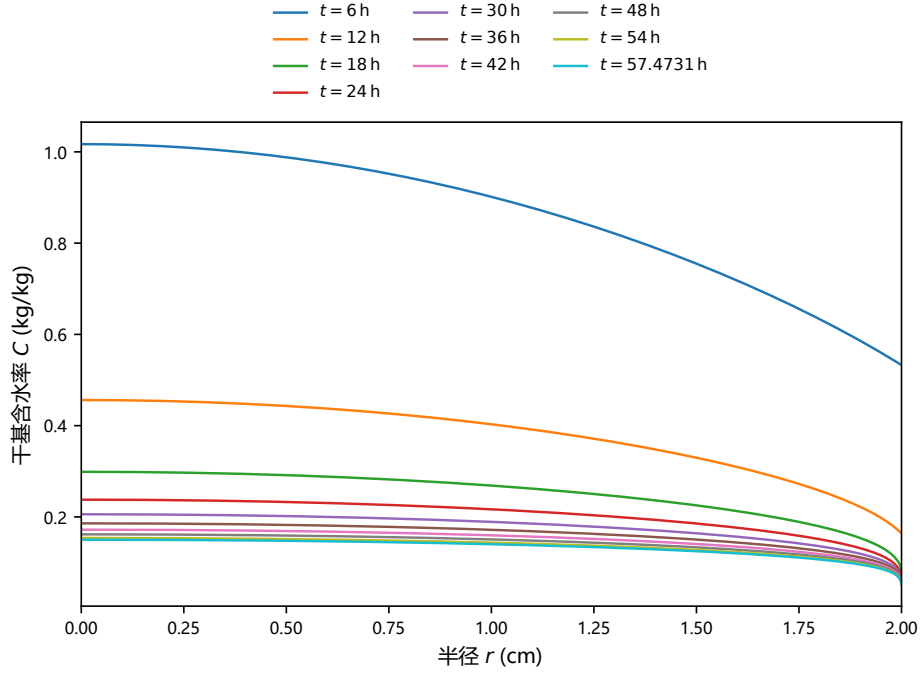


图5 每6 h及结束时刻的径向水分浓度分布

7.5 干燥过程与控制位置分析

计算保存的径向剖面由中心向表面不增，中心附近控制最终达标时间。18 h 时，表面水分浓度已降至 0.0845 kg/kg，而中心仍为 0.2988 kg/kg；48 h 时，中心仍为 0.1618 kg/kg，尚未达标。这些结果说明，表面先达到干燥要求不能代表内部已经完成干燥，也支持本问采用空间最大值判据。

最大水分浓度曲线在后期逐渐变平。由附录 3 的经验式可知，在温度接近稳定后，水分浓度降低会使局部扩散系数减小，水分继续向外迁移的速度随之减慢。因此，总烘干时长需通过变物性模型的长期积分确定，不能将前 3 h 的失水速率线性外推至 0.15 kg/kg。

7.6 数值可靠性与结果输出

以 G_{16} 网格和 G_8 时间收紧解复核生产结果。空间加密后的临界时间变化为 0.585132 s，时间收紧后的变化为 1.7506×10^{-5} s，表 7 的数值在两类加密下保留四位小数均一致；完整配置对照见表 13。

为复核严格不等式，在同一生产网格临界时刻 $t_{*,h}$ 比较三组解，并定义

$$\varepsilon_C = 2 \max \{ |M_h(t_{*,h}) - M_{h/2}(t_{*,h})|, |M_h(t_{*,h}) - M_{h,\text{tight}}(t_{*,h})| \} + 10^{-8}, \quad (23)$$

其中 $M_{h/2}$ 表示细网格解的最大水分浓度， $M_{h,\text{tight}}$ 表示时间收紧解的相应值。本问得到 $\varepsilon_C = 3.5088 \times 10^{-7}$ kg/kg。在 $t_{\text{rep},3}$ 时，生产解最大值加上该裕量仍低于 0.15 kg/kg；细网格解和时间收紧解的最大值分别为 0.1499992209 和 0.1499993913 kg/kg，也均严格达标。该裕量来自已完成的数值加密比较，不是连续方程解的严格误差上界。热湿收支检验见表 11。

表 7 末行中心显示 0.1500，是未舍入值 0.1499993913 保留四位小数的结果，达标判断以未舍入值为准。完整水分浓度结果保存至 **result3.xlsx**，从 60 s 起每 60 s 输出一次，径向从 0 至 2 cm 每 0.1 cm 采样，并追加 206903.16 s 的准确结束时刻。论文表格和结果文件均由同一未舍入数值解生成。

7.7 径向简化的二维端面校核

短时传播尺度不能直接保证约 57 h 预测中的端面影响仍小。参考 Hussain 和 Dincer 对有限圆柱二维热湿传递的研究 [1]，在第三问原初值、附录 3 物性及环境输入下恢复轴向通量。取 $z = 0$ 为中截面，在 $0 \leq r \leq R$ 、 $0 \leq z \leq L/2$ 上建立

$$\begin{aligned} B(C)T_t &= \frac{1}{r} \frac{\partial}{\partial r} (rk(C)T_r) + \frac{\partial}{\partial z} (k(C)T_z), \\ C_t &= \frac{1}{r} \frac{\partial}{\partial r} (rD(C,T)C_r) + \frac{\partial}{\partial z} (D(C,T)C_z). \end{aligned} \quad (24)$$

中心轴和中截面施加零法向梯度，侧面沿用原边界；端面 $z = L/2$ 取 $-kT_z = h_z(T - T_a)$ 、 $-DC_z = h_{m,z}(C - C_a)$ ，其中 $h_z = h$ 、 $h_{m,z} = h_m$ 为指定对照情景。该模型保留本文的变系数守恒形式，以表面加密有限体积法和隐式 BDF 求解，不移植文献材料参数。

以全部二维节点的 $\max C_{ij}(t) = 0.15$ 定位临界根，并对每个径向网格独立计算一维配对解。表 8 中 N_r, N_z 为区间数， $\Delta t_* = t_{*,2D} - t_{*,1D}$ ；采用同径向网格比较，以分离改变网格与开放端面的作用。

表 8 同径向网格下一、二维临界时间对照

$N_r \times N_z$	时间配置	一维 / h	二维 / h	$\Delta t_* / s$
80 × 40	普通	57.502060	57.499795	-8.153
160 × 40	普通	57.479758	57.477496	-8.143
80 × 80	普通	57.502060	57.499937	-7.644
160 × 80	普通	57.479758	57.477637	-7.634
160 × 80	收紧	57.479760	57.477640	-7.635

关闭端面交换后，二维与配对一维临界时间相差 0.0015 s，支持退化一致性。开放端面时累计水分收支相对残差均低于 1.2×10^{-8} 。在细网格附近，径向加密、轴向加密及时间收紧使配对时间差分别变化 0.0102、0.5094 和 0.0012 s，支持端面对中心达标时长影响很小的判断。二维绝对临界时间的径向加密差仍为 80.28 s，不能据输出位数声称秒级绝对精度，也不以此替换原 G_8 一维正式答案。

图 6 表明，端面促进端部干燥，但采样及临界时刻的全域最大值仍由中心控制。细网格开放端面后仅提前约 8 s，约占配对一维时长的 0.004%；两端累计排湿却约占总排湿 5.3%，说明总排湿贡献不能直接换算为中心达标的节时比例。该结果支持本问采用一维径向主模型，并为下一问沿用径向近似提供参考。结论限于各向同性、两端同环境及等交换系数情景，不保证局部二维场四位小数稳定，也不构成实验验证；收缩情景下的沿用依据与限制将在下一问说明。

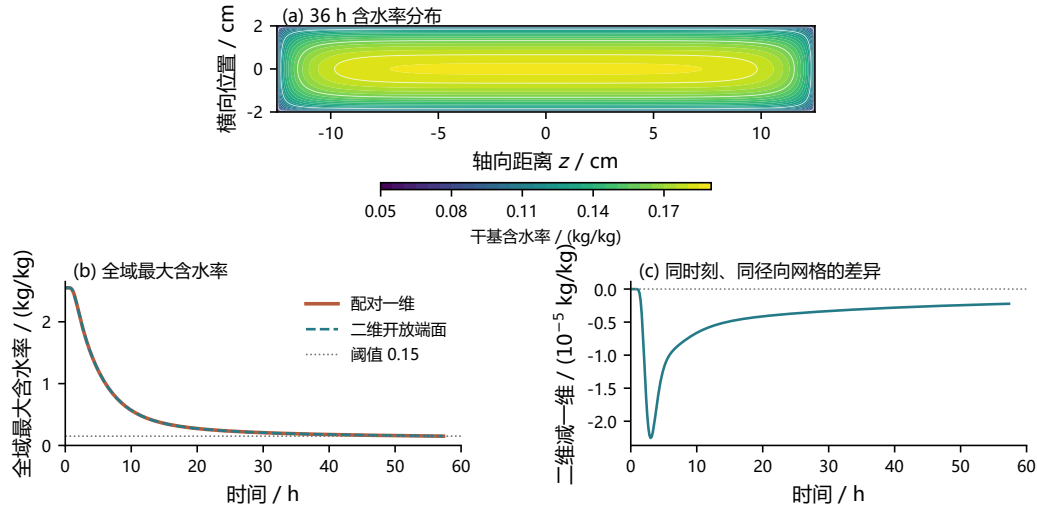


图6 第三问二维端面校核。上图为36 h纵剖面含水率，保持实际长径比；下图比较同径向网格的全域最大值及其差异。

八 问题四：径向收缩条件下的热湿传递

8.1 问题分析与比例收缩假设

第7.7节的二维校核表明，问题三在指定端面条件下，端面效应对中心控制的达标时长影响很小。问题四忽略轴向缩短，长度仍取 $L = 0.25 \text{ m}$ ；随径向收缩，半径与长度之比 $R(t)/L$ 减小，该比值也等于两端总面积与侧面积之比。因此，从几何上看，端面交换的相对作用减弱，预期其对达标时长的影响进一步减小。结合第三问的校核结果，在周向均匀、物性各向同性及相同端面环境假设下，本问继续采用一维径向模型，在移动区域 $0 \leq r \leq R(t)$ 上考察收缩作用。上述判断用于说明模型简化的依据；附录4物性及收缩条件下的端面时间误差尚未单独量化，第三问的0.004%不作为本问误差界。

本问同时更换物性：全部经验式改用附录4，并从 $T(r, 0) = 28^\circ\text{C}$ 、 $C(r, 0) = 2.55 \text{ kg/kg}$ 独立积分，不读取问题三的末态。环境输入和半径轨迹采用第四节的统一处理。第四问新增的本质是

$$R_0 \longrightarrow R(t), \quad \text{附录3物性} \longrightarrow \text{附录4物性}.$$

Brasiello 等讨论了食品干燥中收缩与非等温热湿传递的耦合 [5]。该工作支持将材料运动写入传递方程，其对象、吸附关系与蒸发机制与本题不同，不能证明本题必须采用比例收缩。

题目只给出外半径，没有给出内部各点位移。按假设(2)，令初始位置 r_0 的材料点按相同比例收缩：

$$r(t) = \frac{R(t)}{R_0} r_0. \quad (25)$$

相应骨架径向速度为

$$v(r, t) = \frac{dr}{dt} = \frac{\dot{R}(t)}{R(t)} r. \quad (26)$$

外半径观测不能唯一决定内部材料运动，式(25)和式(26)是用于闭合速度场的模型假设，不是题给事实。长度仍取 $L = 0.25 \text{ m}$ ，忽略轴向缩短。

8.2 收缩条件下的水分守恒与有效传热方程

直接在移动区间 $0 \leq r \leq R(t)$ 上离散不便于沿用前问网格。本节先在物理坐标中写出传递方程，下一节再映到固定区间。

(1) **干物质守恒。** 设 $s(r, t)$ 为单位当前体积中的干物质质量。干物质不流失时，

$$s_t + \frac{1}{r} \partial_r (r s v) = 0. \quad (27)$$

将式(26)代入，并设初始干物质密度均匀，则 s 保持空间均匀，满足

$$\begin{aligned} \dot{s} + 2 \frac{\dot{R}}{R} s &= 0, \\ s(t) &= s_0 \left[\frac{R_0}{R(t)} \right]^2. \end{aligned} \quad (28)$$

其中因子 2 来自仅考虑径向收缩、长度不变：当前体积 $V(t) = \pi R^2(t)L$ 。半径减小后，同样的干物质被压缩到更小体积，因而当前体积干物质密度按 R^{-2} 增大。

(2) **水分守恒。** 干基含水率 C 是水质量与干物质质量之比，不是单位体积的水分浓度。单位当前体积中，干物质质量为 s ，水质量为 sC 。相对固体骨架的有效 Fick 通量为

$$j = -s D(C, T) C_r.$$

水分守恒为

$$\partial_t (sC) + \frac{1}{r} \partial_r [r (sCv + j)] = 0. \quad (29)$$

由式(27)消去几何压缩项，得到

$$C_t + vC_r = \frac{1}{s r} \partial_r (r s D C_r). \quad (30)$$

在式(28)下 $s = s(t)$ 不依赖 r ，故

$$C_t + vC_r = \frac{1}{r} \partial_r [r D(C, T) C_r].$$

若一个材料单元只发生收缩，水和干物质都没有进出，则二者质量之比不变。因此不能对干基含水率再添加 $-C \nabla \cdot v$ 形式的体积压缩源项；那样会把体积浓度的几何浓缩误加到干基量上。

(3) **有效传热方程。** 热方程沿用问题二、三的有效热容量描述。在假设 (3) 下，不显式计入汽化潜热、水分携焓和收缩机械功。对随骨架运动的同一材料微元，温度变化率应取物质导数

$$\frac{DT}{Dt} = T_t + vT_r.$$

因此，将固定域中的有效传热方程推广至收缩域时，以物质导数代替固定空间点的 T_t ，得到

$$\begin{aligned} B(C)(T_t + vT_r) &= \frac{1}{r} \partial_r [rk(C) T_r], \\ B(C) &= \rho(C)c_p(C). \end{aligned} \quad (31)$$

式(31)是有效传热近似，不是完整多相系统的能量守恒方程。经验密度 $\rho(C)$ 仅用于构造 $B(C)$ ；质量记账中的 s 由式(28)决定，不强制 $s = \rho(C)/(1+C)$ 。若同时把 ρ 当作真实湿体积密度，则与给定 $R(t)$ 、固定长度和干物质守恒不能精确相容，该项限制见模型评价。

于是物理坐标下的热湿方程为

$$\begin{cases} B(C)(T_t + vT_r) = \frac{1}{r} \partial_r [rk(C) T_r], \\ C_t + vC_r = \frac{1}{r} \partial_r [rD(C, T) C_r]. \end{cases} \quad (32)$$

相对于前三问，收缩首先通过材料随动项 vT_r 和 vC_r 进入方程。

8.3 材料坐标变换与定解条件

定义

$$\begin{aligned} \xi &= \frac{r}{R(t)}, \quad 0 \leq \xi \leq 1, \\ U(\xi, t) &= C(R(t)\xi, t), \\ \Theta(\xi, t) &= T(R(t)\xi, t). \end{aligned} \quad (33)$$

在比例收缩(25)下，同一材料点满足 $\xi = r_0/R_0$ ，不随时间变化，因而 ξ 标记随骨架运动的相对位置，计算区域始终为固定区间 $[0, 1]$ 。一般非均匀收缩时， ξ 首先只是瞬时归一化坐标，不能自动等同于材料标记。

由链式法则， $C_r = U_\xi/R$ ， $U_t = C_t + \xi \dot{R} C_r$ 。由于 $v = \dot{R}\xi$ ，有 $U_t = C_t + vC_r$ 。温度场同样满足 $\Theta_t = T_t + vT_r$ 。材料运动与坐标网格同步，式(32)中的显式随动项被坐标变换吸收。固定半径模型中内部扩散含因子 R_0^{-2} ；收缩后该因子成为 $R(t)^{-2}$ 。随着 $R(t)$ 减小， R^{-2} 增大，表示在相同无量纲梯度下，中心到表面的实际迁移距离缩短。这是收缩可能改变干燥速度的几何机制，但不能把 R^{-2} 的倍率直接换成烘干时长倍率。

问题四按题意从初始时刻统一使用附录 4 经验式：

$$\begin{aligned} \rho(U) &= 760 + 90U, \\ c_p(U) &= 1850 + 2150 \times \frac{U}{1+U}, \\ k(U) &= 0.12 + 0.20 \times \frac{U}{1+U}, \\ D(U, \Theta) &= 4.2 \times 10^{-4} \exp\left(-\frac{0.30}{U}\right) \times \exp\left[-\frac{3850}{\Theta + 273.15}\right]. \end{aligned} \quad (34)$$

扩散系数仍由药材局部温度进入指数，而不是烘房温度。表面梯度由 $C_r = U_\xi/R$ 改写，因而内部扩散带 R^{-2} 、表面交换带 R^{-1} ，二者必须同时进入定解问题。 $R(t)$ 由附件 2 的

PCHIP 给出, $T_a(t)$ 、 $C_a(t)$ 沿用式(2)。于是在固定区间 $0 \leq \xi \leq 1$ 上, 本问实际积分的定解问题为

$$\begin{cases} U_t = \frac{1}{R^2(t)\xi} \partial_\xi [\xi D(U, \Theta) U_\xi], \\ B(U)\Theta_t = \frac{1}{R^2(t)\xi} \partial_\xi [\xi k(U) \Theta_\xi], \\ U(\xi, 0) = 2.55, \quad \Theta(\xi, 0) = 28, \\ U_\xi(0, t) = \Theta_\xi(0, t) = 0, \\ -\frac{D_s}{R(t)} U_\xi(1, t) = h_m [U_s - C_a(t)], \\ -\frac{k_s}{R(t)} \Theta_\xi(1, t) = h [\Theta_s - T_a(t)]. \end{cases} \quad (35)$$

其中 $B(U) = \rho(U)c_p(U)$, 物性取式(34)。含水率初值以 kg/kg 计, 温度初值以 °C 计。物理坐标式(32)只用于说明随动项如何进入方程, 离散求解针对式(35)。

8.4 数值求解、达标判据与输出转换

在 $\xi \in [0, 1]$ 上沿用前三问的环形有限体积结构。将问题一的 G_8 节点除以 R_0 得到固定材料节点 ξ_i , 内部面取相邻节点中点, 径向权重

$$w_i = \int_{\xi_{i-1/2}}^{\xi_{i+1/2}} \xi \, d\xi,$$

端点按 0 和 1 截取。内部水分通量与半离散方程为

$$\hat{F}_{i+1/2}^C = -\xi_{i+1/2} D_{i+1/2} \cdot \frac{U_{i+1} - U_i}{\xi_{i+1} - \xi_i}, \quad (36)$$

$$R^2(t) w_i \dot{U}_i = \hat{F}_{i-1/2}^C - \hat{F}_{i+1/2}^C.$$

温度方程将 D 换为 k , 并乘以 B_i 。表面通量分别取 $Rh_m(U_N - C_a)$ 与 $Rh(\Theta_N - T_a)$ 。时间推进仍采用隐式 BDF, 局部物性在每次迭代中按当前温湿状态更新。生产计算使用 2113 个材料节点; 相对容差 10^{-8} , 温度与含水率绝对容差分别为 10^{-8} 、 10^{-10} 。前 4 h 最大步长 10 s, 其后 60 s, 并在环境切换处分段接续。网格加密与时间收紧检验见第九节。

烘干完成的定义沿用式(21), 只把固定半径上的最大值换成材料坐标上的最大值:

$$M(t) = \max_{0 \leq \xi \leq 1} U(\xi, t), \quad t_* = \inf\{t \geq 0 : M(t) < 0.15\}.$$

离散求解监测全部材料节点。临界时刻满足 $M(t_*) = 0.15$; 题目要求严格低于阈值, 故另取稍晚的报告时刻 t_{rep} , 并用未舍入结果复核。本问没有重新定义“烘干完成”, 只是把第三问的事件判据放到收缩域上。

题目要求按当前物理距离输出。对 $r_j \leq R(t)$, 先取 $\xi_j = r_j/R(t)$, 再在材料网络上对 U 作保形插值; 若 $r_j > R(t)$, 该位置已在药材外, 单元格记为“—”, 不填 0 或环境值。药材表面始终对应 $\xi = 1$, 含水率与当时半径分别单列, 不能用固定的 2 cm 列代替移动表面。

8.5 烘干时长与含水率分布

生产解临界时刻为 $t_* = 51.0871229810$ h。按式(21)的严格低于阈值规则，得到

$$t_{\text{rep},4} = 51.0877 \text{ h} = 183915.72 \text{ s.} \tag{37}$$

此时生产解的最大含水率为 0.1499989333 kg/kg，严格低于 0.15 kg/kg，最大值位于中心，当时半径为 1.2000 cm，仍在附件 2 的 72 h 覆盖范围内，无需半径外推。规定时刻的径向剖面见图 7，表 9 给出每 6 h 及结束时刻的含水率。

表 9 收缩模型药材烘干过程的水分浓度 (kg/kg)

时间/h	到药材中心的距离/cm						药材表面	$R(t)$ /cm
	0	0.5	1.0	1.5	2.0			
6	1.7188	1.5370	1.0221	—	—	0.4205	1.3740	
12	0.7374	0.6544	0.4082	—	—	0.1669	1.2480	
18	0.4086	0.3686	0.2397	—	—	0.0892	1.2140	
24	0.2851	0.2610	0.1790	—	—	0.0673	1.2040	
30	0.2263	0.2094	0.1493	—	—	0.0595	1.2010	
36	0.1928	0.1796	0.1316	—	—	0.0560	1.2000	
42	0.1712	0.1604	0.1200	—	—	0.0541	1.2000	
48	0.1561	0.1468	0.1116	—	—	0.0530	1.2000	
51.0877	0.1500	0.1413	0.1081	—	—	0.0526	1.2000	

注：—表示该固定距离已超出当时半径；药材表面对应当前 $R(t)$ 。终时中心未舍入值为 0.1499989333 kg/kg。

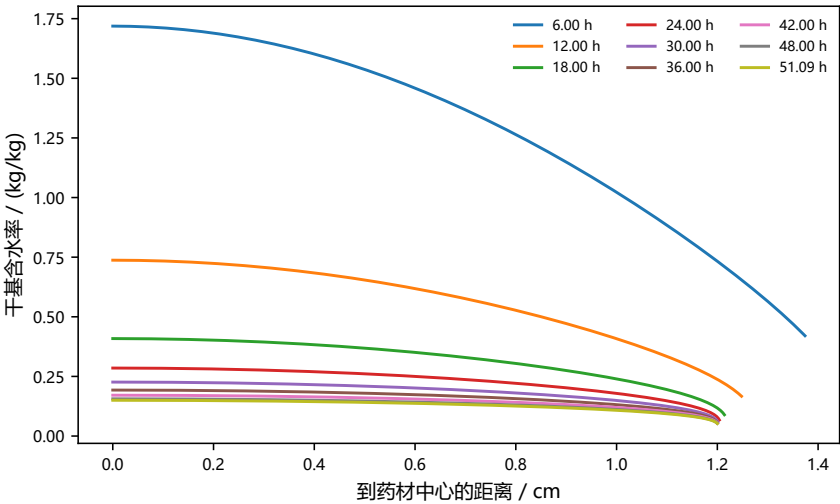


图 7 每 6 h 及结束时刻的含水率剖面。横坐标为当前物理半径，曲线右端随收缩内移。

表 9 中 “—” 表示该固定距离已超出当时半径；药材表面列始终取 $U(1, t)$ ， $R(t)$ 列给出表面位置。终时中心四位显示为 0.1500，对应未舍入值 0.1499989333 kg/kg，达标

判断以未舍入值为准。完整结果保存至 **result4.xlsx**，从 60 s 起每 60 s 输出一次，并追加 183915.72 s 的准确结束时刻。论文表格与结果文件由同一未舍入数值解生成。

18 h 时表面含水率已降至 0.0892 kg/kg，中心仍为 0.4086 kg/kg；48 h 时中心为 0.1561 kg/kg，仍高于阈值。因此表面先接近干燥要求，不能代表内部湿芯已经消失，本问以空间最大值作为达标判据。

8.6 同物性对照与收缩贡献

第三、四问同时改变物性和几何，时长差不能单独归因于收缩。为分离几何作用，在附录 4 物性下增加 $R \equiv R_0 = 0.02 \text{ m}$ 对照，与实测 $R(t)$ 组保持初值、热湿方程、交换系数及全域达标判据一致。两组前 4 h 均用同一环境插值，此后保持 50 °C、0.05 kg/kg；固定组由式(35)取常半径并联合求解温湿场。

表 10 列出两组结果。固定组沿用收缩组的 G_8 及 BDF 配置，均以 G_{16} 和时间收紧复核，并在最晚临界根后留出 2 s、向上取四位小时；报告时刻另经未舍入最大值与经验裕量检查。

表 10 附录 4 物性下固定半径与实测收缩的对照

半径方案	临界时间/h	严格报告时间/h	报告时最大含水率 (kg/kg)
固定 $R = R_0$	129.8442292	129.8448	0.1499996198
实测 $R(t)$	51.0871230	51.0877	0.1499989333

以固定组报告时长为分母，收缩使本模型的预测烘干时间减少

$$\Delta t_{\text{shrink}} = t_{\text{fixed},4} - t_{\text{shrink},4} = 129.8448 - 51.0877 = 78.7571 \text{ h},$$

$$\eta_t = \frac{\Delta t_{\text{shrink}}}{t_{\text{fixed},4}} \times 100\% \approx 60.65\%. \quad (38)$$

两组临界根之差为 78.7571062 h，与报告时长差在四位小数下一致，说明该差异不由严格达标的时间裕量造成。图 8 展示中心含水率全过程及后期放大；保存轨迹中两组中心均与径向最大值重合，收缩组更早穿越阈值。

固定组约 129.84 h 超过 72 h 半径记录，但使用指定常半径，无需外推半径；收缩组仍在观测范围内达标。该对照依赖既定后期环境，量化有效模型中扩散距离、表面交换及温湿反馈共同形成的收缩总效应，不是实测节时比例。

就方程结构而言，材料坐标中内部扩散含 R^{-2} 。报告时刻 $R = 1.2000 \text{ cm}$ ，相对于初值 2 cm，

$$\left(\frac{R_0}{R}\right)^2 = \left(\frac{2.0}{1.2}\right)^2 \approx 2.78.$$

该 2.78 倍仅表示扩散算子的几何系数变化，不能直接当作速度或时长倍率；实际还受局部 D 、温度场和表面 R^{-1} 交换影响。

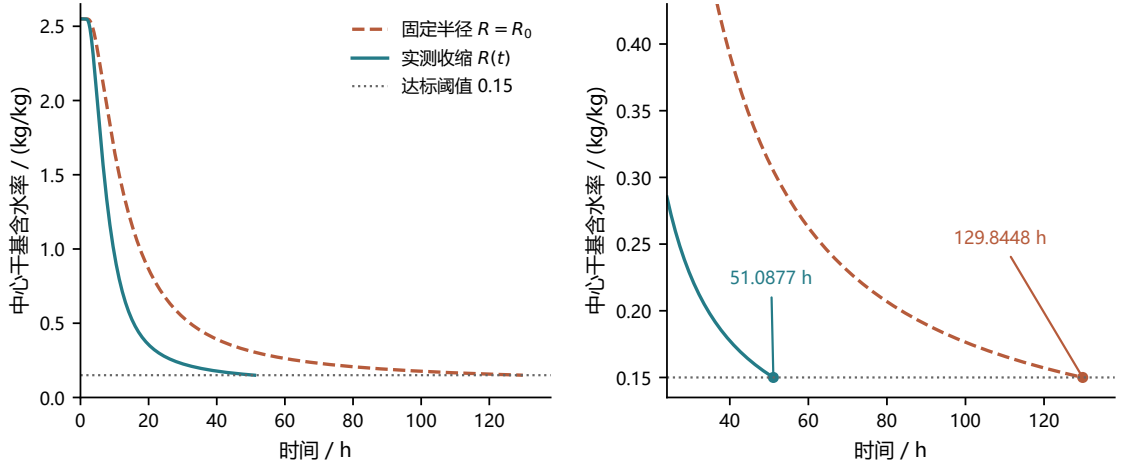


图8 同一附录4物性下，收缩组比固定半径组更早达标。左图为中心含水率全过程，右图放大后期；曲线止于各自报告时刻，点线为0.15 kg/kg 阈值。

收缩域的水分总量按干物质质量归一化后满足

$$\begin{aligned} \frac{M_w}{M_d} &= 2 \int_0^1 U \xi d\xi, \\ \frac{d}{dt} \left(\frac{M_w}{M_d} \right) &= -\frac{2h_m}{R(t)} [U_s - C_a(t)]. \end{aligned} \quad (39)$$

式(39)由材料域方程积分并使用表面条件得到，无需指定未经测量的 s_0 。令 $R(t) \equiv R_0$ 时，式(35)退化为同一附录4物性的固定域方程；关闭表面交换且初值均匀时，纯收缩不改变干基含水率。这两种退化及式(39)的数值收支均已检查，网格与时间加密结果见表13和表11。

九 模型检验与误差分析

9.1 水分收支与有效热方程检验

固定域对半离散方程求和，内部通量相消，得到

$$\sum_i W_i [C_i(t) - C_i(0)] = -Rh_m \int_0^t [C_s(\tau) - C_a(\tau)] d\tau. \quad (40)$$

式(40)按参考干密度归一化，不能直接以 kg 标记；收缩域使用式(39)。

问题一热容量恒定，总有效蓄热增量应与累计对流输入一致。问题二、三则令

$$E(t) = 2\pi L \sum_i W_i B(C_i)(T_i - T_{ini}),$$

由式(19)推导

$$\begin{aligned} E(t) - E(0) &= 2\pi L \int_0^t \sum_i W_i B'(C_i)(T_i - T_{ini}) \dot{C}_i d\tau \\ &= 2\pi LRh \int_0^t (T_a - T_s) d\tau. \end{aligned} \quad (41)$$

组成修正项不能省略。相对残差用收支差除以累计交换量尺度，近零时设正下限；第一问定义见式(15)。第三问在 BDF 接受步连续解上作 4 点、8 点 Gauss 积分对照。表 11 汇总结果，第四问热量仅检验加权半离散方程及温度收敛。残差较小支持离散通量与所选

表 11 各问收支检验：相对残差仅针对所选有效模型

问题	水分相对残差	有效热收支相对残差	检验范围
一	2.4254×10^{-11}	2.0484×10^{-9}	0 至 1800 s
二	4.312×10^{-7}	3.565×10^{-8}	0 至 10800 s
三	4.0221×10^{-11}	3.6640×10^{-8}	至报告时刻
四	4.0558×10^{-9}	不作同类累计比较	生产计算至 72 h

方程的一致性，不验证潜热、相平衡或端面简化的真实物理误差。

9.2 空间与时间加密

在相同物理时刻比较固定域共享节点及收缩域共享材料节点，以各配置的节点最大绝对差为指标。表 12 分别列出温度和含水率结果。前两问覆盖全部规定输出点，后两

表 12 主要网格与时间加密差异

问题	空间比较	ΔT_{space} °C	ΔC_{space} kg/kg	ΔT_{time} °C	ΔC_{time} kg/kg
一	G_4/G_8	5.895×10^{-7}	4.564×10^{-6}	2.874×10^{-8}	1.179×10^{-9}
二	G_2/G_4	2.452×10^{-6}	1.493×10^{-5}	4.108×10^{-7}	2.381×10^{-8}
三	G_8/G_{16}	9.545×10^{-7}	1.537×10^{-5}	1.002×10^{-6}	7.716×10^{-9}
四	G_8/G_{16}	1.012×10^{-5}	8.681×10^{-7}	1.188×10^{-5}	6.383×10^{-8}

问还覆盖内部节点，第四问比较至 72 h。第二问另以 G_4/G_8 复核，温度、含水率最大差为 6.244×10^{-7} °C、 3.732×10^{-6} kg/kg。配置差不等于真解误差；临界时间见表 13。六张题定表在相应加密下四位一致，但不能据此声称全部 Excel 单元格的末位均已稳定。

表 13 不同计算配置下的临界烘干时间 (h)

配置	问题三	问题四
G_4	57.4731739851	51.0872487284
G_8	57.4725195022	51.0871229810
G_{16}	57.4723569654	51.0870914570
G_8 时间收紧	57.4725194974	51.0871221801

9.3 独立对照与输出一致性

问题一的 Bessel–Duhamel 温度对照及冻结扩散系数辅助测试见第 5.5 节。问题二在短时制造解测试中保留变系数导数项，温度和含水率最大误差分别为 6.0949×10^{-8} °C

和 $8.9808 \times 10^{-9} \text{ kg/kg}$ 。制造解用于检验实现，不是药材实验数据。

第三问与第二问在 1800、3600、7200、9000 s 的重叠输出比较中，温度、水分最大差分别为 $4.66 \times 10^{-7} \text{ }^\circ\text{C}$ 和 $9.12 \times 10^{-7} \text{ kg/kg}$ ，支持独立起算的两个过程在共享区间内一致。第四问则采用同一附录 4 物性的固定半径退化检查及关闭边界交换的纯收缩检查，不把附录 3 结果直接作为附录 4 的退化参照。

源数组、题定表及 Excel 保持同源，数值非有限、含水率越界或必需检查失败均阻断正式输出；另核对第三问准确终时、第四问当前半径及域外标记。这些检查不替代物理验证。

9.4 同物性固定半径对照的检验

固定组通过独立物理环层算子检查。空间加密和时间收紧的临界时间差为 0.498231 s 和 0.013113 s；至 132 h 共同保存时刻的温度、水分空间差分别为 $1.3522 \times 10^{-5} \text{ }^\circ\text{C}$ 、 $1.9052 \times 10^{-6} \text{ kg/kg}$ ，时间差分别为 $1.4730 \times 10^{-5} \text{ }^\circ\text{C}$ 、 $6.4161 \times 10^{-8} \text{ kg/kg}$ 。三组累计水分相对残差均低于 10^{-6} 。129.8448 h 时三组最大含水率均低于 0.15，生产值加经验裕量 1.9437×10^{-7} 后仍达标，支持表 10 的数值可比性。

9.5 局部敏感性与适用边界

式(20)给出扩散系数对状态变量的局部敏感性。在 $T = 50 \text{ }^\circ\text{C}$ 、 $C = 0.15$ 时，问题三有

$$\frac{\partial \ln D}{\partial T} \approx 0.0369 \text{ K}^{-1}, \quad \frac{\partial \ln D}{\partial C} = 20 \text{ (kg/kg)}^{-1}.$$

局部升温 $1 \text{ }^\circ\text{C}$ 对应 D 约 3.69% 的一阶变化，不能等同于时长变化。问题四有 $\partial \ln D / \partial U = 0.30 / U^2$ ，阈值处为 $13.3333 \text{ (kg/kg)}^{-1}$ ；几何因子满足 $\partial \ln(R^{-2}) / \partial \ln R = -2$ ，其局部方向仍需全过程对照检验。

若事件附近最大值位置不变且 $M'(t_*) \neq 0$ ，一阶扰动给出

$$\delta t_* \approx -\frac{\delta M(t_*)}{M'(t_*)}. \quad (42)$$

后期曲线越平缓，相同含水率误差越可能放大为时间偏差，故需直接检验事件收敛。表 6 的有限输入扰动量化了指定情景的时长变化，不构成全局稳健性保证。

第三问径向近似的二维端面校核已在第 7.7 节给出；其作用是检验本题条件下的维度简化，与本章的离散收支和数值加密检验相互补充。

十 模型评价与结论

10.1 模型特点

四问以一致的变量和环层通量结构处理常热物性、双向耦合、全域阈值及径向收缩。有限体积保留变系数散度与收支关系，表面加密兼顾薄层分辨和题定输出；解析解、制造解、退化及二维对照分别检查实现与简化。达标时区分临界根、严格不等式与四位显示，收缩时以材料坐标保持干物质和水质量守恒，并按当前物理距离输出结果。

10.2 局限与可推广方向

第三问二维校核为指定情景下的径向简化提供了定量依据。第四问结合径向收缩时长径比增大、端侧面积比减小的几何特征沿用这一近似；其依据仍受各向同性及端面条件假设限制，收缩情景下的端面时间误差尚未单独量化。

等效水分边界缺少独立吸附等温关系，热方程也未显式描述汽化潜热和水分携焓。更完整的界面模型需要药材平衡含水率、相变和气固传质数据。仅凭当前方程的收支残差，不能证明这些机制对真实预测可忽略。

经验密度与几何干物质密度分开使用，是本模型的一项有效近似。以问题四为例，若同时把 $\rho(U)$ 视为真实湿密度，则初始干密度为 $(760 + 90 \times 2.55)/(1 + 2.55) \approx 278.73 \text{ kg/m}^3$ 。72 h 半径对应的体积比为 $(1.198/2)^2 = 0.358801$ ，固定长度且干物质守恒要求平均干密度约 776.84 kg/m^3 ；但对任意 $U \geq 0$ ，

$$\frac{760 + 90U}{1 + U} = 90 + \frac{670}{1 + U} \leq 760 \text{ kg/m}^3.$$

这说明上述解释不能同时精确成立，因此本文只将 ρc_p 用于有效热容量。该检查针对 72 h 数据和指定解释，不是 51.0877 h 报告时刻的误差估计，也不说明题目无法建模。

后期环境为恒定延续假设，尚无内部实测数据验证预测。问题三温度情景使时长变化约 6%，后期水分量 $\pm 10\%$ 使其变化约 0.2%；这些输入不确定性意味着数值时长末位不能解释为实际预测精度。

10.3 结论

在题给初值、经验物性和本文明确的等效假设下，问题一 1800 s 时中心温度 33.5758°C 、表面 36.7879°C ，水分损失主要集中在外层；问题二 3 h 时中心温度 49.8494°C 、含水率 1.7662 kg/kg ，温度接近平衡早于内部达到干燥标准。全域阈值计算给出附录 3 固定半径模型的报告时长 57.4731 h、附录 4 给定收缩模型的报告时长 51.0877 h。二者均经未舍入末态和时空加密核验，且移动边界计算未超出半径观测范围。补充的附录 4 固定半径对照报告时长为 129.8448 h，在相同物性与环境条件下，收缩使模型预测时长缩短 78.7571 h，约占固定组的 60.65%。

AI 工具使用声明

本文工作使用了 AI 工具，主要用于建模材料核对、论文起草、同物性固定半径对照与二维端面校核的程序编写和数值验证，以及图表绘制、公式一致性检查和 LaTeX 排版。具体使用范围、交互方式和核验记录见支撑材料中的《AI 工具使用详情》。

参考文献

- [1] Hussain M M, Dincer I. Two-dimensional heat and moisture transfer analysis of a cylindrical moist object subjected to drying: A finite-difference approach[J]. International Journal of Heat and Mass Transfer, 2003, 46(21): 4033–4039. DOI: 10.1016/S0017-9310(03)00229-1.
- [2] Kaya A, Aydin O, Dincer I. Numerical modeling of forced-convection drying of cylindrical moist objects[J]. Numerical Heat Transfer, Part A: Applications, 2007, 51(9): 843–854. DOI: 10.1080/10407780601112753.
- [3] The SciPy community. PchipInterpolator: SciPy v1.18.0 Manual[EB/OL]. [2026-09-12]. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.interpolate.PchipInterpolator.html>.
- [4] The SciPy community. BDF: SciPy v1.18.0 Manual[EB/OL]. [2026-09-12]. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.BDF.html>.
- [5] Brasiello A, Venditti C, Adrover A. Non-isothermal Moving-boundary Model for Food Drying[J]. Chemical Engineering Transactions, 2021, 87: 193–198. DOI: 10.3303/CET2187033.

附录 A 支撑材料清单与复现说明

支撑材料包含完整 Python 模型与验证程序、JavaScript 导出与检查脚本、四个正式 Excel 结果文件、题定结果 CSV、验证 JSON、主要报告、依赖清单、源文件哈希映射和 AI 工具使用详情。另附第三问二维校核的全部网格场与配对数据；原四问的大型内部数组可复算。

Python 依赖为 NumPy、SciPy、openpyxl 和 Matplotlib；原生产运行记录为 Python 3.14.6、NumPy 2.5.1、SciPy 1.18.0。代码采用相对项目根目录，需提供 **data/**附件/附件 1.xlsx、附件 2.xlsx 及附件 3/中的题给模板。所有初值、物性和时间单位与正文一致。

按下列次序可复算四问数值与所用加密对照。第三问使用 `--no-xlsx` 可独立复核数值而不依赖 Node；随附正式 Excel 不受该参数更新。

```
python code/problem1.py
python code/problem2.py
python code/problem3.py --no-xlsx
python code/problem4.py --level 2
python code/problem4.py --level 4
python code/problem4.py --level 4 --tight
python code/problem4.py --level 8
python code/problem4.py --level 16
python code/problem4.py --level 8 --tight
python code/verify_q4_results.py
python code/plot_q3_results.py
```

原第 3、4 问 Excel 导出脚本依赖 Node.js 与 @oai/artifact-tool。配置依赖后，第 3 问不加 `--no-xlsx` 运行，第 4 问执行 `node tmp/q4_export/build.mjs`。支撑材料附完整导出脚本。为使代码可迁移，仅将原计算机的 Node 路径改为从 PATH 查找，将第 3 问布局模板路径改为随附结果文件；数值方程与参数保持原实现。

以下列出本论文模型与验证所使用的完整源程序。部分文件保留历史诊断函数；正式入口及当前输出以以上复现顺序为准。各程序运行会在本地生成结果与诊断文件，宜在支撑材料副本中执行。

附录 B 完整源程序

B.1 code/utils.py

```
1 """Shared paths and attachment readers for Problem A."""
2 from __future__ import annotations
3
4 from pathlib import Path
5
6 import numpy as np
7 from openpyxl import load_workbook
8 from scipy.interpolate import PchipInterpolator
9
```

```

10 ROOT = Path(__file__).resolve().parents[1]
11 DATA_DIR = ROOT / "data"
12 ATTACH_DIR = DATA_DIR / " 附件"
13 RESULTS_DIR = ROOT / "results"
14 FIGURES_DIR = ROOT / "figures"
15 REPORTS_DIR = ROOT / "reports"
16
17 # Appendix 2 (Problem 1)
18 R = 0.02
19 L = 0.25
20 RHO = 820.0
21 CP = 2600.0
22 K = 0.36
23 H = 25.0
24 HM = 8.0e-7
25 T0 = 28.0
26 C0 = 2.55
27 D_PREFACTOR = 7.0e-9
28 D_EXP_COEF = 0.89
29 T_END = 1800.0
30 ALPHA = K / (RHO * CP)
31
32
33 def moisture_diffusivity(C: np.ndarray | float, *, protect: bool = True) -> np.ndarray | float:
34     """D(C) = 7e-9 exp(-0.89/C). Nonpositive C uses D=0 only as a solver-domain guard."""
35     C_arr = np.asarray(C, dtype=float)
36     out = np.zeros_like(C_arr, dtype=float)
37     positive = C_arr > 0.0
38     if np.any(positive):
39         out[positive] = D_PREFACTOR * np.exp(-D_EXP_COEF / C_arr[positive])
40     if not protect and np.any(~positive):
41         raise ValueError("moisture diffusivity evaluated at nonpositive C without protection")
42     if np.isscalar(C):
43         return float(out)
44     return out
45
46
47 def load_oven_table() -> tuple[np.ndarray, np.ndarray, np.ndarray]:
48     path = ATTACH_DIR / " 附件 1.xlsx"
49     wb = load_workbook(path, read_only=True, data_only=True)
50     rows = list(wb.active.values)
51     wb.close()
52     data = np.asarray(rows[1:], dtype=float)
53     t, Ta, Ca = data[:, 0], data[:, 1], data[:, 2]
54     if t[0] != 0.0:
55         raise ValueError(f"oven table must start at t=0, got {t[0]}")
56     return t, Ta, Ca
57
58
59 class PiecewiseLinear:
60     """Piecewise linear interpolant; kept for comparison, not used in Q1 production."""
61
62     def __init__(self, t: np.ndarray, y: np.ndarray):
63         self.t = np.asarray(t, dtype=float)
64         self.y = np.asarray(y, dtype=float)
65         self.kind = "linear"
66         if self.t.ndim != 1 or self.y.shape != self.t.shape:
67             raise ValueError("interpolant requires matching 1-D arrays")
68         if np.any(np.diff(self.t) <= 0):
69             raise ValueError("breakpoints must be strictly increasing")
70
71     def __call__(self, t: float | np.ndarray) -> float | np.ndarray:
72         return np.interp(t, self.t, self.y)
73
74     def slope(self, t: float) -> float:

```

```

75     t = float(t)
76     if t <= self.t[0] or t >= self.t[-1]:
77         if t >= self.t[-1]:
78             dt = self.t[-1] - self.t[-2]
79             return (self.y[-1] - self.y[-2]) / dt
80         return (self.y[1] - self.y[0]) / (self.t[1] - self.t[0])
81     i = int(np.searchsorted(self.t, t, side="right") - 1)
82     i = min(i, len(self.t) - 2)
83     return (self.y[i + 1] - self.y[i]) / (self.t[i + 1] - self.t[i])
84
85
86 class PchipOven:
87     """Monotone piecewise cubic Hermite interpolant (PCHIP) through attachment knots."""
88
89     def __init__(self, t: np.ndarray, y: np.ndarray):
90         self.t = np.asarray(t, dtype=float)
91         self.y = np.asarray(y, dtype=float)
92         self.kind = "pchip"
93         if self.t.ndim != 1 or self.y.shape != self.t.shape:
94             raise ValueError("interpolant requires matching 1-D arrays")
95         if np.any(np.diff(self.t) <= 0):
96             raise ValueError("breakpoints must be strictly increasing")
97         self._pchip = PchipInterpolator(self.t, self.y, extrapolate=False)
98         self._deriv = self._pchip.derivative()
99
100     def __call__(self, t: float | np.ndarray) -> float | np.ndarray:
101         tt = np.clip(np.asarray(t, dtype=float), self.t[0], self.t[-1])
102         val = self._pchip(tt)
103         if np.ndim(t) == 0:
104             return float(val)
105         return np.asarray(val, dtype=float)
106
107     def slope(self, t: float) -> float:
108         tt = float(np.clip(t, self.t[0], self.t[-1]))
109         return float(self._deriv(tt))

```

B.2 code/appendix3.py

```

1  """Appendix 3 effective properties for Problems 2 and 3."""
2  from __future__ import annotations
3
4  import numpy as np
5
6  RHO_A = 650.0
7  RHO_B = 128.0
8  CP_A = 1450.0
9  CP_B = 2736.0
10 K_A = 0.21
11 K_B = 0.38
12 D_PREFACTOR = 2.4e-3
13 D_C_COEF = 0.45
14 D_T_COEF = 3850.0
15
16
17 def rho(C: np.ndarray | float) -> np.ndarray | float:
18     return RHO_A + RHO_B * np.asarray(C, dtype=float)
19
20
21 def cp(C: np.ndarray | float) -> np.ndarray | float:
22     C_arr = np.asarray(C, dtype=float)
23     return CP_A + CP_B * C_arr / (1.0 + C_arr)
24
25
26 def conductivity(C: np.ndarray | float) -> np.ndarray | float:
27     C_arr = np.asarray(C, dtype=float)

```

```

28     return K_A + K_B * C_arr / (1.0 + C_arr)
29
30
31 def heat_capacity(C: np.ndarray | float) -> np.ndarray | float:
32     return rho(C) * cp(C)
33
34
35 def conductivity_deriv(C: np.ndarray | float) -> np.ndarray | float:
36     C_arr = np.asarray(C, dtype=float)
37     return K_B / (1.0 + C_arr) ** 2
38
39
40 def heat_capacity_deriv(C: np.ndarray | float) -> np.ndarray | float:
41     C_arr = np.asarray(C, dtype=float)
42     return RHO_B * cp(C_arr) + rho(C_arr) * CP_B / (1.0 + C_arr) ** 2
43
44
45 def moisture_diffusivity_q2(
46     C: np.ndarray | float,
47     T_celsius: np.ndarray | float,
48     *,
49     protect: bool = True,
50 ) -> np.ndarray | float:
51     """D(C,T) with T in Celsius; Kelvin is used only inside the Arrhenius factor."""
52     C_arr = np.asarray(C, dtype=float)
53     T_arr = np.asarray(T_celsius, dtype=float)
54     out = np.zeros(np.broadcast(C_arr, T_arr).shape, dtype=float)
55     C_b, T_b = np.broadcast_arrays(C_arr, T_arr)
56     positive = C_b > 0.0
57     if np.any(positive):
58         Tk = T_b[positive] + 273.15
59         out[positive] = D_PREFACTOR * np.exp(-D_C_COEF / C_b[positive]) * np.exp(-D_T_COEF / Tk)
60     if not protect and np.any(~positive):
61         raise ValueError("q2 diffusivity evaluated at nonpositive C without protection")
62     if np.ndim(C) == 0 and np.ndim(T_celsius) == 0:
63         return float(np.asarray(out))
64     return out
65
66
67 def diffusivity_partials(C: np.ndarray, T_celsius: np.ndarray) -> tuple[np.ndarray, np.ndarray,
↪ np.ndarray]:
68     D = np.asarray(moisture_diffusivity_q2(C, T_celsius, protect=True), dtype=float)
69     C_arr = np.asarray(C, dtype=float)
70     T_arr = np.asarray(T_celsius, dtype=float)
71     dD_dC = np.zeros_like(D)
72     dD_dT = np.zeros_like(D)
73     ok = C_arr > 0.0
74     dD_dC[ok] = D[ok] * (D_C_COEF / C_arr[ok] ** 2)
75     Tk = T_arr + 273.15
76     dD_dT[ok] = D[ok] * (D_T_COEF / Tk[ok] ** 2)
77     return D, dD_dC, dD_dT

```

B.3 code/radial_fvm.py

```

1  """Node-centered radial finite-volume operators for Problem 1."""
2  from __future__ import annotations
3
4  import numpy as np
5  from scipy.sparse import diags
6
7  from utils import CP, H, HM, K, R, RHO, moisture_diffusivity
8
9  OUTPUT_RADII_M = 0.001 * np.arange(21)
10
11

```

```

12 def graded_radial_nodes(level: int) -> np.ndarray:
13     """Nodes denser near the surface; every 0.1 cm output radius remains a node.
14
15     Interval subdivisions (before multiplying by ``level``):
16     0-1.5 cm: 8; 1.5-1.8 cm: 16; 1.8-1.9 cm: 32; 1.9-2.0 cm: 64.
17     Surface spacing at level L is 1e-3 / (64 L) metres.
18     """
19     level = int(level)
20     if level < 1:
21         raise ValueError("graded mesh level must be a positive integer")
22     n_sub = np.full(20, 8 * level, dtype=int)
23     n_sub[15:] = 16 * level
24     n_sub[18] = 32 * level
25     n_sub[19] = 64 * level
26     pts = [0.0]
27     for j in range(20):
28         r0 = 0.001 * j
29         r1 = 0.001 * (j + 1)
30         m = int(n_sub[j])
31         for k in range(1, m + 1):
32             pts.append(r0 + (r1 - r0) * k / m)
33     r = np.asarray(pts, dtype=float)
34     r[0] = 0.0
35     r[-1] = R
36     return r
37
38
39 class RadialFVM:
40     def __init__(self, N: int | None = None, r: np.ndarray | None = None):
41         if r is not None:
42             self.r = np.asarray(r, dtype=float)
43             if self.r.ndim != 1 or self.r.size < 3:
44                 raise ValueError("r must be a 1-D array with at least 3 nodes")
45             if abs(self.r[0]) > 1e-15 or abs(self.r[-1] - R) > 1e-14:
46                 raise ValueError("radial nodes must start at 0 and end at R")
47             if np.any(np.diff(self.r) <= 0.0):
48                 raise ValueError("radial nodes must be strictly increasing")
49         else:
50             if N is None or int(N) < 2:
51                 raise ValueError("N must be at least 2")
52             self.r = np.linspace(0.0, R, int(N) + 1)
53             self.n_nodes = int(self.r.size)
54             self.N = self.n_nodes - 1
55             self.dr_face = np.diff(self.r)
56             self.r_half = 0.5 * (self.r[:-1] + self.r[1:])
57             a = np.empty(self.n_nodes)
58             b = np.empty(self.n_nodes)
59             a[0] = 0.0
60             b[0] = self.r_half[0]
61             a[-1] = self.r_half[-1]
62             b[-1] = R
63             if self.n_nodes > 2:
64                 a[1:-1] = self.r_half[:-1]
65                 b[1:-1] = self.r_half[1:]
66             self.W = 0.5 * (b**2 - a**2)
67             self.dr = float(self.dr_face.min())
68             if abs(self.W.sum() - 0.5 * R**2) > 1e-14:
69                 raise RuntimeError("radial weights do not sum to R^2/2")
70
71             self.M_heat = self._heat_matrix()
72             self.M_heat_sparse = diags(
73                 [np.diag(self.M_heat, -1), np.diag(self.M_heat, 0), np.diag(self.M_heat, 1)],
74                 offsets=[-1, 0, 1],
75                 shape=self.M_heat.shape,
76                 format="csc",

```

```

77         )
78         self._heat_force_coef = (R * H) / (RHO * CP * self.W[-1])
79         self._heat_force_index = self.n_nodes - 1
80
81     def output_indices(self) -> np.ndarray:
82         """Indices of 0, 0.1, ..., 2.0 cm nodes."""
83         idx = np.empty(OUTPUT_RADII_M.size, dtype=int)
84         for j, rc in enumerate(OUTPUT_RADII_M):
85             k = int(np.argmin(np.abs(self.r - rc)))
86             if abs(self.r[k] - rc) > 1e-12:
87                 raise ValueError(f"grid misses output radius {rc}, nearest {self.r[k]}")
88             idx[j] = k
89         return idx
90
91     def _interior_face_coeff(self) -> np.ndarray:
92         return self.r_half / self.dr_face
93
94     def _heat_matrix(self) -> np.ndarray:
95         n = self.n_nodes
96         M = np.zeros((n, n))
97         face = K * self._interior_face_coeff()
98         den = RHO * CP * self.W
99         M[0, 0] = -face[0] / den[0]
100        M[0, 1] = face[0] / den[0]
101        for i in range(1, n - 1):
102            M[i, i - 1] = face[i - 1] / den[i]
103            M[i, i + 1] = face[i] / den[i]
104            M[i, i] = -(face[i - 1] + face[i]) / den[i]
105        i = n - 1
106        M[i, i - 1] = face[i - 1] / den[i]
107        M[i, i] = (-face[i - 1] - R * H) / den[i]
108        return M
109
110     def heat_rhs(self, t: float, T: np.ndarray, Ta: float) -> np.ndarray:
111         out = self.M_heat @ T
112         out[self._heat_force_index] += self._heat_force_coef * Ta
113         return out
114
115     def heat_jac(self, t: float, T: np.ndarray):
116         return self.M_heat_sparse
117
118     def heat_jac_sparsity(self):
119         n = self.n_nodes
120         ones_m = np.ones(n - 1, dtype=float)
121         ones_d = np.ones(n, dtype=float)
122         return diags([ones_m, ones_d, ones_m], offsets=[-1, 0, 1], shape=(n, n), format="csc")
123
124     def moisture_rhs(self, t: float, C: np.ndarray, Ca: float, D_const: float | None = None) -> np.ndarray:
125         if D_const is None:
126             D_node = moisture_diffusivity(C, protect=True)
127         else:
128             D_node = np.full_like(C, float(D_const))
129         D_face = harmonic_mean(D_node[:-1], D_node[1:])
130         F = -self.r_half * D_face * np.diff(C) / self.dr_face
131         Cdot = np.empty_like(C)
132         Cdot[0] = -F[0] / self.W[0]
133         Cdot[1:-1] = (F[:-1] - F[1:]) / self.W[1:-1]
134         F_surf = R * HM * (C[-1] - Ca)
135         Cdot[-1] = (F[-1] - F_surf) / self.W[-1]
136         return Cdot
137
138     def moisture_jac_sparsity(self):
139         return self.heat_jac_sparsity()
140
141     def surface_heat_influx_density(self, T_s: float, Ta: float) -> float:

```

```

142     """Heat into the solid per unit surface area, W/m^2."""
143     return H * (Ta - T_s)
144
145     def surface_moisture_outflux(self, C_s: float, Ca: float) -> float:
146         """Outward moisture flux in the normalized (no rho_d) units, m/s * kg/kg."""
147         return HM * (C_s - Ca)
148
149
150 def harmonic_mean(a: np.ndarray, b: np.ndarray) -> np.ndarray:
151     out = np.zeros_like(a, dtype=float)
152     both = (a > 0.0) & (b > 0.0)
153     out[both] = 2.0 * a[both] * b[both] / (a[both] + b[both])
154     return out

```

B.4 code/radial_coupled.py

```

1  """Coupled variable-property radial FVM for Problem 2."""
2  from __future__ import annotations
3
4  import numpy as np
5  from scipy.sparse import lil_matrix
6
7  from appendix3 import conductivity, heat_capacity, moisture_diffusivity_q2
8  from radial_fvm import RadialFVM, harmonic_mean
9  from utils import H, HM, R
10
11
12 class CoupledRadialFVM:
13     def __init__(self, r: np.ndarray):
14         geom = RadialFVM(r=r)
15         self.r = geom.r
16         self.W = geom.W
17         self.r_half = geom.r_half
18         self.dr_face = geom.dr_face
19         self.n_nodes = geom.n_nodes
20         self.N = geom.N
21         self._geom = geom
22
23     def output_indices(self) -> np.ndarray:
24         return self._geom.output_indices()
25
26     def pack(self, T: np.ndarray, C: np.ndarray) -> np.ndarray:
27         return np.concatenate([T, C])
28
29     def unpack(self, y: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
30         n = self.n_nodes
31         return y[:n], y[n:]
32
33     def rhs(
34         self,
35         t: float,
36         y: np.ndarray,
37         Ta: float,
38         Ca: float,
39         *,
40         D_const: float | None = None,
41         ST: np.ndarray | None = None,
42         SC: np.ndarray | None = None,
43     ) -> np.ndarray:
44         T, C = self.unpack(y)
45         kn = np.asarray(conductivity(C), dtype=float)
46         Bn = np.asarray(heat_capacity(C), dtype=float)
47         if D_const is None:
48             Dn = np.asarray(moisture_diffusivity_q2(C, T, protect=True), dtype=float)
49         else:

```

```

50         Dn = np.full_like(C, float(D_const))
51         k_face = harmonic_mean(kn[:-1], kn[1:])
52         D_face = harmonic_mean(Dn[:-1], Dn[1:])
53         FT = -self.r_half * k_face * np.diff(T) / self.dr_face
54         FC = -self.r_half * D_face * np.diff(C) / self.dr_face
55         Tdot = np.empty_like(T)
56         Cdot = np.empty_like(C)
57         Tdot[0] = -FT[0] / (Bn[0] * self.W[0])
58         Cdot[0] = -FC[0] / self.W[0]
59         Tdot[1:-1] = (FT[:-1] - FT[1:]) / (Bn[1:-1] * self.W[1:-1])
60         Cdot[1:-1] = (FC[:-1] - FC[1:]) / self.W[1:-1]
61         FTsurf = R * H * (T[-1] - Ta)
62         FCsurf = R * HM * (C[-1] - Ca)
63         Tdot[-1] = (FT[-1] - FTsurf) / (Bn[-1] * self.W[-1])
64         Cdot[-1] = (FC[-1] - FCsurf) / self.W[-1]
65         if ST is not None:
66             Tdot = Tdot + ST / Bn
67         if SC is not None:
68             Cdot = Cdot + SC
69         return self.pack(Tdot, Cdot)
70
71     def jac_sparsity(self):
72         n = self.n_nodes
73         J = lil_matrix((2 * n, 2 * n), dtype=float)
74         for i in range(n):
75             js = [i]
76             if i > 0:
77                 js.append(i - 1)
78             if i + 1 < n:
79                 js.append(i + 1)
80             for j in js:
81                 J[i, j] = 1.0
82                 J[i, n + j] = 1.0
83                 J[n + i, j] = 1.0
84                 J[n + i, n + j] = 1.0
85         return J.tocsc()

```

B.5 code/q1_analytic.py

```

1  """Independent Bessel-Duhamel reference for the linear radial heat (or frozen-D moisture) problem."""
2  from __future__ import annotations
3
4  from typing import Any
5
6  import numpy as np
7  from scipy.integrate import solve_ivp
8  from scipy.optimize import brentq
9  from scipy.special import j0, j1
10 from scipy.sparse import diags
11
12
13 def robin_eigenvalues(Bi: float, n_roots: int) -> np.ndarray:
14     """Positive roots of  $\lambda * J_1(\lambda) = Bi * J_0(\lambda)$ ."""
15     if Bi <= 0:
16         raise ValueError("Biot number must be positive")
17     n_roots = int(n_roots)
18
19     def f(lam: float) -> float:
20         return lam * j1(lam) - Bi * j0(lam)
21
22     lam_max = (n_roots + 4) * np.pi
23     grid = np.linspace(1e-10, lam_max, max(8000, 200 * n_roots))
24     vals = grid * j1(grid) - Bi * j0(grid)
25     roots: list[float] = []
26     for i in range(len(grid) - 1):

```

```

27     v0, v1 = vals[i], vals[i + 1]
28     if not np.isfinite(v0) or not np.isfinite(v1):
29         continue
30     a, b = float(grid[i]), float(grid[i + 1])
31     if v0 == 0.0:
32         cand = a
33     elif v0 * v1 < 0.0:
34         cand = float(brentq(f, a, b, xtol=1e-14, maxiter=200))
35     else:
36         continue
37     if not roots or abs(cand - roots[-1]) > 1e-10:
38         roots.append(cand)
39     if len(roots) >= n_roots:
40         break
41     if len(roots) < n_roots:
42         raise RuntimeError(f"found only {len(roots)} Robin Bessel roots, need {n_roots}")
43     return np.asarray(roots, dtype=float)
44
45
46 def expansion_coefficients(lam: np.ndarray) -> np.ndarray:
47     j0l = j0(lam)
48     j1l = j1(lam)
49     return 2.0 * j1l / (lam * (j0l**2 + j1l**2))
50
51
52 class BesselDuhamel:
53     def __init__(self, Bi: float, diffusivity: float, R: float, n_terms: int = 120):
54         self.Bi = float(Bi)
55         self.alpha = float(diffusivity)
56         self.R = float(R)
57         self.lam = robin_eigenvalues(self.Bi, n_terms)
58         self.A = expansion_coefficients(self.lam)
59         self.beta = self.alpha * self.lam**2 / self.R**2
60
61     def radial_modes(self, r: np.ndarray) -> np.ndarray:
62         """Return array shape (n_terms, n_r) of A_n J0(lambda_n r/R)."""
63         r = np.asarray(r, dtype=float)
64         return self.A[:, None] * j0(self.lam[:, None] * (r[None, :] / self.R))
65
66     def evaluate(self, r: np.ndarray, t: float, env: Any, u0: float) -> np.ndarray:
67         return self.evaluate_many(r, np.array([float(t)]), env, u0)[0]
68
69     def evaluate_many(self, r: np.ndarray, times: np.ndarray, env: Any, u0: float) -> np.ndarray:
70         """T = env(t) - sum A_n J0 I_n, with I' = -beta I + env'(t), I(0)=env(0)-u0.
71
72         Valid for any C1 interpolant that exposes env.y, env.slope, and env(t).
73         """
74         r = np.asarray(r, dtype=float)
75         times = np.asarray(times, dtype=float)
76         if times.size == 0:
77             return np.empty((0, r.size), dtype=float)
78         modes = self.radial_modes(r)
79         beta = self.beta
80         I0 = np.full(beta.size, float(env.y[0] - u0))
81         t_end = float(np.max(times))
82         out = np.empty((times.size, r.size), dtype=float)
83         zero = times == 0.0
84         if np.all(zero):
85             out[:] = u0
86             return out
87         t_eval = times[~zero]
88         sol = solve_ivp(
89             lambda t, I: -beta * I + env.slope(t),
90             (0.0, max(t_end, 1e-30)),
91             I0,

```

```

92         method="BDF",
93         jac=diags(-beta, format="csc"),
94         rtol=1e-11,
95         atol=1e-13,
96         t_eval=t_eval,
97         max_step=1.0,
98         vectorized=False,
99     )
100     if not sol.success:
101         raise RuntimeError(f"Bessel Duhamel ODE failed: {sol.message}")
102     Ihist = sol.y.T
103     env_t = np.array([float(env(t)) for t in t_eval], dtype=float)
104     nonzero_vals = env_t[:, None] - Ihist @ modes
105     out[zero] = u0
106     out[~zero] = nonzero_vals
107     return out
108
109
110 def converged_reference(Bi, diffusivity, R, r, times, env, u0, start_terms=320, tolerance=1e-8):
111     """Double the modal truncation and explicitly check every requested point."""
112     previous = BesselDuhamel(Bi, diffusivity, R, start_terms).evaluate_many(r, times, env, u0)
113     history = []
114     for terms in (start_terms * 2, start_terms * 4, start_terms * 8):
115         current = BesselDuhamel(Bi, diffusivity, R, terms).evaluate_many(r, times, env, u0)
116         error = float(np.max(np.abs(current - previous)))
117         history.append({"terms": terms, "previous_terms": terms // 2, "max_abs_difference": error})
118         if np.isfinite(error) and error <= tolerance:
119             return current, {"passed": True, "tolerance": tolerance, "history": history, "terms": terms}
120         previous = current
121     return current, {"passed": False, "tolerance": tolerance, "history": history, "terms": terms}

```

B.6 code/delivery.py

```

1  """Fail-closed numerical delivery checks shared by Q1 and Q2."""
2  import json
3  from pathlib import Path
4
5
6  def make_gate(checks):
7      checks = {name: bool(value) for name, value in checks.items()}
8      return {"checks": checks, "failed_checks": [k for k, v in checks.items() if not v],
9              "meets_delivery_gate": bool(checks) and all(checks.values())}
10
11
12 def require_delivery(validation, diagnostic_path):
13     """Persist diagnostics first; never reach any production writer on failure."""
14     path = Path(diagnostic_path)
15     path.parent.mkdir(parents=True, exist_ok=True)
16     path.write_text(json.dumps(validation, ensure_ascii=False, indent=2, default=float), encoding="utf-8")
17     gate = validation["delivery_gate"]
18     checks = gate.get("checks", {})
19     if not checks or not all(value is True for value in checks.values()) or gate.get("meets_delivery_gate")
20     ↪ is not True:
21         raise RuntimeError("Delivery blocked; see " + str(path) + ": " + str(gate.get("failed_checks",
22         ↪ [])))

```

B.7 code/problem1.py

```

1  """Problem 1: radial FVM + BDF for preheating temperature and moisture."""
2  from __future__ import annotations
3
4  import json
5  import sys
6  import time

```

```

7 from pathlib import Path
8
9 import numpy as np
10 from openpyxl import Workbook
11 from scipy.integrate import solve_ivp, quad
12
13 CODE_DIR = Path(__file__).resolve().parent
14 if str(CODE_DIR) not in sys.path:
15     sys.path.insert(0, str(CODE_DIR))
16
17 from q1_analytic import BesselDuhamel, converged_reference
18 from delivery import make_gate, require_delivery
19 from radial_fvm import RadialFVM
20 from radial_fvm import graded_radial_nodes
21 from utils import (
22     ALPHA,
23     C0,
24     CP,
25     FIGURES_DIR,
26     H,
27     HM,
28     K,
29     L,
30     R,
31     RESULTS_DIR,
32     RHO,
33     T0,
34     T_END,
35     PchipOven,
36     load_oven_table,
37     moisture_diffusivity,
38 )
39
40 OUTPUT_TIMES = np.arange(1.0, T_END + 0.5, 1.0)
41 TABLE_TIMES = np.array([100, 300, 600, 900, 1200, 1500, 1800], dtype=float)
42 TABLE_RADII_CM = np.array([0.0, 0.5, 1.0, 1.5, 2.0])
43 OUTPUT_RADII_M = 0.001 * np.arange(21) # 0, 0.1, ..., 2.0 cm
44 PAPER_TIMES = TABLE_TIMES.copy()
45
46
47 def load_q1_inputs() -> tuple[PchipOven, PchipOven, np.ndarray]:
48     t, Ta, Ca = load_oven_table()
49     mask = t <= T_END + 1e-12
50     t, Ta, Ca = t[mask], Ta[mask], Ca[mask]
51     if t.size != 31 or abs(t[-1] - T_END) > 1e-12:
52         raise ValueError(f"expected 31 oven records on [0, 1800], got {t.size}, t_end={t[-1]}")
53     if abs(Ta[0] - T0) > 1e-12:
54         raise ValueError(f"Ta(0)={Ta[0]} != T0={T0}")
55     Ta_fun = PchipOven(t, Ta)
56     Ca_fun = PchipOven(t, Ca)
57     for tj, yj in zip(t, Ta):
58         if abs(float(Ta_fun(tj)) - float(yj)) > 1e-12:
59             raise RuntimeError(f"PCHIP misses Ta knot t={tj}")
60     for tj, yj in zip(t, Ca):
61         if abs(float(Ca_fun(tj)) - float(yj)) > 1e-12:
62             raise RuntimeError(f"PCHIP misses Ca knot t={tj}")
63     return Ta_fun, Ca_fun, t
64
65
66 def integrate_mol(
67     rhs,
68     y0: np.ndarray,
69     t_breaks: np.ndarray,
70     *,
71     rtol: float,

```

```

72     atol: float,
73     max_step: float,
74     jac=None,
75     jac_sparsity=None,
76     field_name: str = "field",
77 ):
78     y = np.asarray(y0, dtype=float).copy()
79     times = [0.0]
80     states = [y.copy()]
81     pieces: list[tuple[float, float, object]] = []
82     t_end = float(t_breaks[-1])
83     for i in range(len(t_breaks) - 1):
84         t0 = float(t_breaks[i])
85         t1 = float(min(t_breaks[i + 1], t_end))
86         if t1 <= t0:
87             continue
88         t_eval = np.arange(np.floor(t0) + 1.0, np.floor(t1) + 1.0 + 1e-12, 1.0)
89         t_eval = t_eval[(t_eval > t0 + 1e-14) & (t_eval <= t1 + 1e-14)]
90         kwargs = dict(
91             fun=rhs,
92             t_span=(t0, t1),
93             y0=y,
94             method="BDF",
95             rtol=rtol,
96             atol=atol,
97             max_step=max_step,
98             dense_output=True,
99             vectorized=False,
100         )
101         if t_eval.size:
102             kwargs["t_eval"] = t_eval
103         if jac is not None:
104             kwargs["jac"] = jac
105         if jac_sparsity is not None:
106             kwargs["jac_sparsity"] = jac_sparsity
107         sol = solve_ivp(**kwargs)
108         if not sol.success:
109             raise RuntimeError(f"{field_name} BDF failed on [{t0}, {t1}]: {sol.message}")
110         pieces.append((t0, t1, sol))
111         if sol.t.size:
112             for k, tk in enumerate(sol.t):
113                 if times and abs(tk - times[-1]) < 1e-12:
114                     states[-1] = sol.y[:, k].copy()
115                     continue
116                 times.append(float(tk))
117                 states.append(sol.y[:, k].copy())
118             y = sol.y[:, -1].copy() if sol.y.size else y
119         t_arr = np.asarray(times, dtype=float)
120         Y = np.stack(states, axis=0)
121         return t_arr, Y, pieces
122
123
124 def dense_state(pieces, t: float) -> np.ndarray:
125     t = float(t)
126     for t0, t1, sol in pieces:
127         if t0 - 1e-12 <= t <= t1 + 1e-12:
128             return np.asarray(sol.sol(np.clip(t, t0, t1)), dtype=float)
129     raise ValueError(f"time {t} not covered by dense output")
130
131
132 def extract_output_grid(fvm: RadialFVM, Y: np.ndarray) -> np.ndarray:
133     idx = fvm.output_indices()
134     return Y[:, idx]
135
136

```

```

137 def conservation_heat(fvm: RadialFVM, pieces, Ta_fun: PchipOven, T_init: np.ndarray, t: float) -> dict:
138     T_t = dense_state(pieces, t)
139     E = 2.0 * np.pi * L * RHO * CP * np.dot(fvm.W, T_t - T_init)
140     flux = lambda x: float(Ta_fun(x)) - float(dense_state(pieces, x)[-1])
141     Qraw, qerr = quad(flux, 0, t, points=Ta_fun.t[(Ta_fun.t > 0) & (Ta_fun.t < t)], epsabs=1e-9,
142     ↪ epsrel=1e-10, limit=500)
143     Q = 2.0 * np.pi * R * L * H * Qraw
144     denom = max(abs(Q), abs(E), 1e-12)
145     return {
146         "E_relative_J": E,
147         "Q_surface_J": Q,
148         "relative_residual": abs(E - Q) / denom,
149         "quadrature_abs_error": qerr, "quadrature": "adaptive QUADPACK on PCHIP intervals",
150     }
151
152 def conservation_moisture(fvm: RadialFVM, pieces, Ca_fun: PchipOven, C_init: np.ndarray, t: float) -> dict:
153     C_t = dense_state(pieces, t)
154     MC = float(np.dot(fvm.W, C_t))
155     MC0 = float(np.dot(fvm.W, C_init))
156     flux = lambda x: HM * (float(dense_state(pieces, x)[-1]) - float(Ca_fun(x)))
157     I, qerr = quad(flux, 0, t, points=Ca_fun.t[(Ca_fun.t > 0) & (Ca_fun.t < t)], epsabs=1e-14,
158     ↪ epsrel=1e-10, limit=500)
159     rhs = MC0 - R * I
160     denom = max(abs(MC0 - MC), abs(R * I), 1e-18)
161     return {
162         "MC": MC,
163         "MC0": MC0,
164         "minus_R_int_hm": rhs,
165         "relative_residual": abs(MC - rhs) / denom,
166         "quadrature_abs_error": qerr, "quadrature": "adaptive QUADPACK on PCHIP intervals",
167     }
168
169 def compare_on_output(A: np.ndarray, B: np.ndarray, times: np.ndarray) -> dict:
170     diff = np.abs(A - B)
171     k = int(np.argmax(diff))
172     n_t, n_r = diff.shape
173     i, j = divmod(k, n_r)
174     return {
175         "max_abs": float(diff.max()),
176         "max_time_s": float(times[i]),
177         "max_radius_cm": 0.1 * j,
178         "mean_abs": float(diff.mean()),
179         "paper_table_max_abs": float(np.abs(A[np.isin(times, TABLE_TIMES)][:, ::5] - B[np.isin(times,
180     ↪ TABLE_TIMES)][:, ::5]).max())
181         if A.shape[1] == 21
182         else None,
183     }
184
185 def four_decimal_stats(A: np.ndarray, B: np.ndarray, times: np.ndarray) -> dict:
186     Ar = np.round(A, 4)
187     Br = np.round(B, 4)
188     changed = Ar != Br
189     paper_t = np.isin(times, TABLE_TIMES)
190     paper_r = slice(None, None, 5) if A.shape[1] == 21 else slice(None)
191     paper_changed = Ar[paper_t][:, paper_r] != Br[paper_t][:, paper_r]
192     early = times <= 100.0
193     return {
194         "changed_cells": int(np.sum(changed)),
195         "total_cells": int(A.size),
196         "paper_changed_cells": int(np.sum(paper_changed)),
197         "paper_total_cells": int(paper_changed.size),
198         "early100_changed_cells": int(np.sum(changed[early])),

```

```

199         "max_abs": float(np.max(np.abs(A - B))),
200         "paper_table_max_abs": float(np.max(np.abs(A[paper_t][:, paper_r] - B[paper_t][:, paper_r]))),
201     }
202
203
204 def solve_temperature(fvm: RadialFVM, Ta_fun: PchipOven, t_breaks, rtol, atol, max_step):
205     T_init = np.full(fvm.n_nodes, T0)
206     def rhs(t, T):
207         return fvm.heat_rhs(t, T, float(Ta_fun(t)))
208     def jac(t, T):
209         return fvm.heat_jac(t, T)
210     return integrate_mol(
211         rhs, T_init, t_breaks,
212         rtol=rtol, atol=atol, max_step=max_step,
213         jac=jac, jac_sparsity=fvm.heat_jac_sparsity(),
214         field_name="temperature",
215     )
216
217
218 def solve_moisture(fvm: RadialFVM, Ca_fun: PchipOven, t_breaks, rtol, atol, max_step, D_const=None):
219     C_init = np.full(fvm.n_nodes, C0)
220     def rhs(t, C):
221         return fvm.moisture_rhs(t, C, float(Ca_fun(t)), D_const=D_const)
222     return integrate_mol(
223         rhs, C_init, t_breaks,
224         rtol=rtol, atol=atol, max_step=max_step,
225         jac_sparsity=fvm.moisture_jac_sparsity(),
226         field_name="moisture",
227     )
228
229
230 def assert_physical(T_out, C_out, Ta_fun, Ca_fun) -> dict:
231     Ta_max = float(np.max(Ta_fun.y))
232     Ta_min = float(np.min(Ta_fun.y))
233     Ca_min = float(np.min(Ca_fun.y))
234     info = {
235         "T_min": float(T_out.min()),
236         "T_max": float(T_out.max()),
237         "C_min": float(C_out.min()),
238         "C_max": float(C_out.max()),
239         "T_below_28": float(max(0.0, 28.0 - T_out.min())),
240         "T_above-Ta_max": float(max(0.0, T_out.max() - Ta_max)),
241         "C_above_C0": float(max(0.0, C_out.max() - C0)),
242         "C_below_Ca_min": float(max(0.0, Ca_min - C_out.min())),
243         "Ta_range": [Ta_min, Ta_max],
244         "Ca_range": [Ca_min, float(np.max(Ca_fun.y))],
245     }
246     info["T_bounds_ok"] = info["T_below_28"] <= 1e-6 and info["T_above-Ta_max"] <= 1e-6
247     info["C_bounds_ok"] = info["C_above_C0"] <= 1e-8 and info["C_below_Ca_min"] <= 1e-8
248     info["C_nonnegative"] = bool(C_out.min() >= -1e-8)
249     return info
250
251
252 def write_result_workbook(path: Path, T_out: np.ndarray, C_out: np.ndarray, times: np.ndarray):
253     radii_cm = 0.1 * np.arange(21)
254     wb = Workbook()
255     for title, arr in ((" 温度", T_out), (" 水分浓度", C_out)):
256         ws = wb.create_sheet(title)
257         ws["A1"] = " 时间\\到药材中心的距离"
258         for j, rc in enumerate(radii_cm, start=2):
259             ws.cell(1, j, float(rc))
260         for i, t in enumerate(times):
261             ws.cell(i + 2, 1, int(round(float(t))))
262             for j in range(21):
263                 cell = ws.cell(i + 2, j + 2, float(arr[i, j]))

```

```

264         cell.number_format = "0.0000"
265     if "Sheet" in wb.sheetnames:
266         del wb["Sheet"]
267     path.parent.mkdir(parents=True, exist_ok=True)
268     wb.save(path)
269
270
271 def write_paper_csv(path: Path, arr_out: np.ndarray, times: np.ndarray, radii_cm=TABLE_RADII_CM):
272     t_idx = {float(t): i for i, t in enumerate(times)}
273     r_idx = {round(0.1 * j, 10): j for j in range(21)}
274     lines = ["t_s," + ",".join(f"r_{rc:g}cm" for rc in radii_cm)]
275     for t in TABLE_TIMES:
276         row = [f"{t:.0f}"]
277         i = t_idx[float(t)]
278         for rc in radii_cm:
279             j = r_idx[float(rc)]
280             row.append(f"{arr_out[i, j]:.8f}")
281         lines.append(",".join(row))
282     path.write_text("\n".join(lines) + "\n", encoding="utf-8")
283
284
285 def setup_mpl():
286     import matplotlib
287     matplotlib.use("Agg")
288     import matplotlib.pyplot as plt
289     plt.rcParams.update({
290         "font.sans-serif": ["Microsoft YaHei", "SimHei", "Noto Sans CJK SC", "DejaVu Sans"],
291         "axes.unicode_minus": False,
292         "font.size": 9,
293         "axes.labelsize": 10,
294         "legend.fontsize": 8,
295         "xtick.labelsize": 8,
296         "ytick.labelsize": 8,
297         "axes.linewidth": 0.8,
298         "pdf.fonttype": 42,
299         "ps.fonttype": 42,
300     })
301     return plt
302
303
304 def plot_figures(plt, fvm, times, T_full, C_full, T_out, C_out, Ta_fun, Ca_fun):
305     FIGURES_DIR.mkdir(parents=True, exist_ok=True)
306     r_cm = fvm.r * 100.0
307     r_out_cm = OUTPUT_RADII_M * 100.0
308     t_marks = PAPER_TIMES
309     idx = {float(t): i for i, t in enumerate(times)}
310
311     fig, ax = plt.subplots(figsize=(6.4, 3.8))
312     for t in t_marks:
313         ax.plot(r_cm, T_full[idx[float(t)]], lw=1.2, label=fr"$t={int(t)}\, \mathrm{{{s}}}$")
314     ax.set_xlabel("半径 $r$ (cm)")
315     ax.set_ylabel("温度 $T$ (°C)")
316     ax.set_xlim(0, 2)
317     ax.legend(frameon=False, ncol=2)
318     fig.tight_layout()
319     fig.savefig(FIGURES_DIR / "q1_T_profiles.pdf")
320     plt.close(fig)
321
322     fig, ax = plt.subplots(figsize=(6.4, 3.8))
323     for t in t_marks:
324         ax.plot(r_cm, C_full[idx[float(t)]], lw=1.2, label=fr"$t={int(t)}\, \mathrm{{{s}}}$")
325     ax.set_xlabel("半径 $r$ (cm)")
326     ax.set_ylabel("干基含水率 $C$ (kg/kg)")
327     ax.set_xlim(0, 2)
328     ax.legend(frameon=False, ncol=2)

```

```

329     fig.tight_layout()
330     fig.savefig(FIGURES_DIR / "q1_C_profiles.pdf")
331     plt.close(fig)
332
333     t_plot = np.concatenate([[0.0], times])
334     T_c = np.concatenate([[T0], T_out[:, 0]])
335     T_s = np.concatenate([[T0], T_out[:, -1]])
336     Ta = np.array([Ta_fun(t) for t in t_plot])
337     fig, ax = plt.subplots(figsize=(6.4, 3.8))
338     ax.plot(t_plot, Ta, lw=1.0, ls="--", label="烘房 $T_a$")
339     ax.plot(t_plot, T_s, lw=1.2, label="表面")
340     ax.plot(t_plot, T_c, lw=1.2, label="中心")
341     ax.set_xlabel("时间 $t$ (s)")
342     ax.set_ylabel("温度 (°C)")
343     ax.set_xlim(0, 1800)
344     ax.legend(frameon=False)
345     fig.tight_layout()
346     fig.savefig(FIGURES_DIR / "q1_T_center_surface.pdf")
347     plt.close(fig)
348
349     C_c = np.concatenate([[C0], C_out[:, 0]])
350     C_s = np.concatenate([[C0], C_out[:, -1]])
351     fig, ax = plt.subplots(figsize=(6.4, 3.8))
352     ax.plot(t_plot, C_s, lw=1.2, label="表面")
353     ax.plot(t_plot, C_c, lw=1.2, label="中心")
354     ax.set_xlabel("时间 $t$ (s)")
355     ax.set_ylabel(r"干基含水率 $C$ (kg/kg)")
356     ax.set_xlim(0, 1800)
357     ax.legend(frameon=False)
358     fig.tight_layout()
359     fig.savefig(FIGURES_DIR / "q1_C_center_surface.pdf")
360     plt.close(fig)
361
362     fig, ax1 = plt.subplots(figsize=(6.4, 3.6))
363     ax2 = ax1.twinx()
364     t_dense = np.linspace(0.0, T_END, 1801)
365     ax1.plot(t_dense, np.array([Ta_fun(t) for t in t_dense]), color="C0", lw=1.2, label="烘房温度")
366     ax2.plot(t_dense, np.array([Ca_fun(t) for t in t_dense]), color="C1", lw=1.2, ls="--",
367         ↪ label="烘房水分")
367     ax1.scatter(Ta_fun.t, Ta_fun.y, s=14, color="C0", zorder=3)
368     ax2.scatter(Ca_fun.t, Ca_fun.y, s=14, color="C1", zorder=3)
369     ax1.set_xlabel("时间 $t$ (s)")
370     ax1.set_ylabel("烘房温度 (°C)", color="C0")
371     ax2.set_ylabel("烘房水分 (kg/kg)", color="C1")
372     ax1.set_xlim(0, 1800)
373     h1, l1 = ax1.get_legend_handles_labels()
374     h2, l2 = ax2.get_legend_handles_labels()
375     ax1.legend(h1 + h2, l1 + l2, frameon=False)
376     fig.tight_layout()
377     fig.savefig(FIGURES_DIR / "q1_oven_input.pdf")
378     plt.close(fig)
379     _ = r_out_cm
380
381
382 def main():
383     t_wall0 = time.perf_counter()
384     RESULTS_DIR.mkdir(parents=True, exist_ok=True)
385     Ta_fun, Ca_fun, knots = load_q1_inputs()
386     # PCHIP is C1; integrate as one span rather than 60 s linear kinks.
387     t_breaks = np.array([0.0, T_END], dtype=float)
388     print(
389         "oven points", knots.size,
390         "kind", Ta_fun.kind,
391         "Ta(1800)=", Ta_fun(T_END),
392         "Ca(1800)=", Ca_fun(T_END),

```

```

393 )
394 print("alpha", ALPHA, "D(C0)", moisture_diffusivity(C0), "Bi_T", H * R / K)
395
396 mesh_levels = [2, 4, 8]
397 rtol, atol_T, atol_C, max_step = 1e-10, 1e-10, 1e-12, 0.5
398 solutions = {}
399 for level in mesh_levels:
400     r = graded_radial_nodes(level)
401     fvm = RadialFVM(r=r)
402     tag = f"G{level}"
403     print(
404         f"\n=== {tag} nodes={fvm.n_nodes} surface_dr={fvm.dr_face[-1]:.4e} m ==="
405     )
406     print(f"=== {tag} temperature ===")
407     tT, YT, piecesT = solve_temperature(fvm, Ta_fun, t_breaks, rtol, atol_T, max_step)
408     print(f"=== {tag} moisture ===")
409     tC, YC, piecesC = solve_moisture(fvm, Ca_fun, t_breaks, rtol, atol_C, max_step)
410     if not np.allclose(tT, tC):
411         raise RuntimeError("temperature and moisture output times differ")
412     times = tT[tT > 0]
413     YT_pos = YT[tT > 0]
414     YC_pos = YC[tC > 0]
415     T_out = extract_output_grid(fvm, YT_pos)
416     C_out = extract_output_grid(fvm, YC_pos)
417     solutions[level] = dict(
418         tag=tag,
419         level=level,
420         fvm=fvm,
421         times=times,
422         YT=YT_pos,
423         YC=YC_pos,
424         T_out=T_out,
425         C_out=C_out,
426         piecesT=piecesT,
427         piecesC=piecesC,
428         T_init=np.full(fvm.n_nodes, T0),
429         C_init=np.full(fvm.n_nodes, C0),
430         n_nodes=fvm.n_nodes,
431         surface_dr=float(fvm.dr_face[-1]),
432     )
433     print(
434         "done", tag,
435         "T range", T_out.min(), T_out.max(),
436         "C range", C_out.min(), C_out.max(),
437         "C(t=1,s)", C_out[0, -1],
438         "C(t=100,s)", C_out[99, -1],
439     )
440
441 conv = {}
442 fourdec = {}
443 for coarse, fine in ((2, 4), (4, 8)):
444     t = solutions[fine]["times"]
445     conv[f"T_G{coarse}_vs_G{fine}"] = compare_on_output(
446         solutions[coarse]["T_out"], solutions[fine]["T_out"], t
447     )
448     conv[f"C_G{coarse}_vs_G{fine}"] = compare_on_output(
449         solutions[coarse]["C_out"], solutions[fine]["C_out"], t
450     )
451     fourdec[f"T_G{coarse}_vs_G{fine}"] = four_decimal_stats(
452         solutions[coarse]["T_out"], solutions[fine]["T_out"], t
453     )
454     fourdec[f"C_G{coarse}_vs_G{fine}"] = four_decimal_stats(
455         solutions[coarse]["C_out"], solutions[fine]["C_out"], t
456     )
457 print("conv T", coarse, fine, conv[f"T_G{coarse}_vs_G{fine}"])

```

```

458     print("conv C", coarse, fine, conv[f"C_G{coarse}_vs_G{fine}"])
459     print("4dp T", coarse, fine, fourdec[f"T_G{coarse}_vs_G{fine}"])
460     print("4dp C", coarse, fine, fourdec[f"C_G{coarse}_vs_G{fine}"])
461
462     fd_T = fourdec["T_G4_vs_G8"]
463     fd_C = fourdec["C_G4_vs_G8"]
464     paper_4dp_stable = (
465         fd_T["paper_changed_cells"] == 0 and fd_C["paper_changed_cells"] == 0
466     )
467     full_4dp_stable = fd_T["changed_cells"] == 0 and fd_C["changed_cells"] == 0
468     abs_2e5 = (
469         conv["T_G4_vs_G8"]["max_abs"] <= 2e-5
470         and conv["C_G4_vs_G8"]["max_abs"] <= 2e-5
471     )
472     pub_level = 4 if paper_4dp_stable and full_4dp_stable else 8
473     pub = solutions[pub_level]
474     fvm = pub["fvm"]
475     times = pub["times"]
476     print(
477         "publishing", pub["tag"],
478         "n_nodes", pub["n_nodes"],
479         "paper_4dp_stable", paper_4dp_stable,
480         "full_4dp_stable", full_4dp_stable,
481         "abs_2e5", abs_2e5,
482     )
483
484     print("time refinement on published mesh")
485     tT2, YT2, _ = solve_temperature(fvm, Ta_fun, t_breaks, rtol / 5, atol_T / 5, 0.25)
486     tC2, YC2, _ = solve_moisture(fvm, Ca_fun, t_breaks, rtol / 5, atol_C / 5, 0.25)
487     T2 = extract_output_grid(fvm, YT2[tT2 > 0])
488     C2 = extract_output_grid(fvm, YC2[tC2 > 0])
489     time_sens = {
490         "T": compare_on_output(pub["T_out"], T2, times),
491         "C": compare_on_output(pub["C_out"], C2, times),
492     }
493     time_4dp = {
494         "T": four_decimal_stats(pub["T_out"], T2, times),
495         "C": four_decimal_stats(pub["C_out"], C2, times),
496     }
497     print("time sens", time_sens)
498     print("time 4dp", time_4dp)
499
500     print("Bessel temperature reference")
501     Bi = H * R / K
502     T_ref, heat_truncation = converged_reference(Bi, ALPHA, R, OUTPUT_RADII_M, times, Ta_fun, T0,
503         ↪ start_terms=120)
504     bessel_T = compare_on_output(pub["T_out"], T_ref, times)
505     print("Bessel T", bessel_T)
506
507     print("frozen-D moisture probe")
508     D0 = float(moisture_diffusivity(C0))
509     Bi_m = HM * R / D0
510     C_ref, moisture_truncation = converged_reference(Bi_m, D0, R, OUTPUT_RADII_M, times, Ca_fun, C0)
511     tCf, YCf, _ = solve_moisture(fvm, Ca_fun, t_breaks, rtol, atol_C, max_step, D_const=D0)
512     C_frozen = extract_output_grid(fvm, YCf[tCf > 0])
513     frozen = compare_on_output(C_frozen, C_ref, times)
514     nl_vs_frozen = compare_on_output(pub["C_out"], C_frozen, times)
515     print("nonlinear vs frozen D", nl_vs_frozen)
516     D_field = moisture_diffusivity(pub["YC"])
517     D_stats = {
518         "D0": D0,
519         "D_min": float(np.min(D_field)),
520         "D_max": float(np.max(D_field)),
521         "D_min_over_D0": float(np.min(D_field) / D0),

```

```

522     "D_max_over_D0": float(np.max(D_field) / D0),
523     "C_min_field": float(np.min(pub["YC"])),
524     "C_max_field": float(np.max(pub["YC"])),
525     "surface_C_min": float(np.min(pub["C_out"][:, -1])),
526     "center_C_end": float(pub["C_out"][-1, 0]),
527     "surface_C_end": float(pub["C_out"][-1, -1]),
528     "surface_C_t1": float(pub["C_out"][0, -1]),
529     "surface_C_t100": float(pub["C_out"][99, -1]),
530 }
531 print("D stats", D_stats)
532
533 bounds = assert_physical(pub["YT"], pub["YC"], Ta_fun, Ca_fun)
534 print("bounds", bounds)
535
536 cons_T = conservation_heat(fvm, pub["piecesT"], Ta_fun, pub["T_init"], T_END)
537 cons_C = conservation_moisture(fvm, pub["piecesC"], Ca_fun, pub["C_init"], T_END)
538 print("conservation T", cons_T)
539 print("conservation C", cons_C)
540
541 delivery = make_gate({
542     "space_abs": abs_2e5,
543     "space_paper": paper_4dp_stable,
544     "all_finite": np.isfinite(pub["YT"]).all() and np.isfinite(pub["YC"]).all(),
545     "heat_reference_truncation": heat_truncation["passed"],
546     "heat_reference_error": bessel_T["max_abs"] <= 2e-5,
547     "time_abs": time_sens["T"]["max_abs"] <= 2e-5 and time_sens["C"]["max_abs"] <= 2e-5,
548     "time_paper": time_4dp["T"]["paper_changed_cells"] == 0 and time_4dp["C"]["paper_changed_cells"] ==
    ↪ 0,
549     "temperature_bounds": bounds["T_bounds_ok"],
550     "moisture_bounds": bounds["C_bounds_ok"],
551     "moisture_reference_truncation": moisture_truncation["passed"],
552     "frozen_reference_error": frozen["max_abs"] <= 2e-5,
553     "heat_balance": cons_T["relative_residual"] <= 1e-6,
554     "moisture_balance": cons_C["relative_residual"] <= 1e-6,
555 })
556 print("delivery", delivery)
557
558 paper_T = {
559     f"t{int(t)}_r{rc:g}": float(pub["T_out"][int(t) - 1, int(round(rc / 0.1))])
560     for t in TABLE_TIMES for rc in TABLE_RADII_CM
561 }
562 paper_C = {
563     f"t{int(t)}_r{rc:g}": float(pub["C_out"][int(t) - 1, int(round(rc / 0.1))])
564     for t in TABLE_TIMES for rc in TABLE_RADII_CM
565 }
566
567 validation = {
568     "result_version": "q1-closeout-v1",
569     "interpolation": Ta_fun.kind,
570     "published_mesh": pub["tag"],
571     "published_n_nodes": pub["n_nodes"],
572     "published_surface_dr_m": pub["surface_dr"],
573     "mesh_levels": mesh_levels,
574     "mesh_n_nodes": {f"G{lv}": solutions[lv]["n_nodes"] for lv in mesh_levels},
575     "solver": "FVM+BDF",
576     "rtol": rtol,
577     "atol_T": atol_T,
578     "atol_C": atol_C,
579     "max_step": max_step,
580     "t_breaks": t_breaks.tolist(),
581     "grid_convergence": conv,
582     "four_decimal_grid": fourdec,
583     "time_sensitivity": time_sens,
584     "four_decimal_time": time_4dp,
585     "delivery_gate": delivery,

```

```

586     "reference_truncation_T": heat_truncation,
587     "bessel_temperature": bessel_T,
588     "reference_truncation_C": moisture_truncation,
589     "frozen_D_moisture_vs_analytic": frozen,
590     "nonlinear_C_vs_frozen_D": nl_vs_frozen,
591     "D_variation": D_stats,
592     "conservation_T_1800s": cons_T,
593     "conservation_C_1800s": cons_C,
594     "physical_bounds": bounds,
595     "paper_table_T": paper_T,
596     "paper_table_C": paper_C,
597     "elapsed_s": time.perf_counter() - t_wall0,
598     "python": sys.version,
599 }
600 require_delivery(validation, RESULTS_DIR / "diagnostics/q1_delivery/latest.json")
601 np.savez(
602     RESULTS_DIR / "q1_solution.npz",
603     N=pub["n_nodes"] - 1,
604     mesh_level=pub_level,
605     n_nodes=pub["n_nodes"],
606     times=times,
607     r=fvm.r,
608     checkpoint_times=np.r_[0., TABLE_TIMES],
609     T_checkpoints=np.vstack([pub["T_init"], pub["YT"][TABLE_TIMES.astype(int)-1]]),
610     C_checkpoints=np.vstack([pub["C_init"], pub["YC"][TABLE_TIMES.astype(int)-1]]),
611     T_full=pub["YT"],
612     C_full=pub["YC"],
613     T_out=pub["T_out"],
614     C_out=pub["C_out"],
615     output_radii_m=OUTPUT_RADII_M,
616     interpolation=np.array(Ta_fun.kind),
617     rtol=rtol,
618     atol_T=atol_T,
619     atol_C=atol_C,
620     max_step=max_step,
621     surface_dr=pub["surface_dr"],
622 )
623 write_result_workbook(RESULTS_DIR / "result1.xlsx", pub["T_out"], pub["C_out"], times)
624 write_paper_csv(RESULTS_DIR / "q1_table_temperature.csv", pub["T_out"], times)
625 write_paper_csv(RESULTS_DIR / "q1_table_moisture.csv", pub["C_out"], times)
626
627 plt = setup_mpl()
628 plot_figures(plt, fvm, times, pub["YT"], pub["YC"], pub["T_out"], pub["C_out"], Ta_fun, Ca_fun)
629
630 (RESULTS_DIR / "q1_validation.json").write_text(
631     json.dumps(validation, ensure_ascii=False, indent=2, default=float),
632     encoding="utf-8",
633 )
634 print("elapsed_s", validation["elapsed_s"])
635 print("wrote results to", RESULTS_DIR)
636
637
638
639 if __name__ == "__main__":
640     main()

```

B.8 code/problem2.py

```

1 """Problem 2: coupled variable-property radial drying for 0-3 h."""
2 from __future__ import annotations
3
4 import json
5 import sys
6 import time
7 from pathlib import Path

```

```

8
9 import numpy as np
10 from openpyxl import Workbook
11 from scipy.integrate import solve_ivp
12
13 CODE_DIR = Path(__file__).resolve().parent
14 if str(CODE_DIR) not in sys.path:
15     sys.path.insert(0, str(CODE_DIR))
16
17 from appendix3 import (
18     conductivity,
19     conductivity_deriv,
20     diffusivity_partials,
21     heat_capacity,
22     heat_capacity_deriv,
23     moisture_diffusivity_q2,
24 )
25 from q1_analytic import BesselDuhamel, converged_reference
26 from delivery import make_gate, require_delivery
27 from radial_coupled import CoupledRadialFVM
28 from radial_fvm import graded_radial_nodes
29 from utils import (
30     C0,
31     FIGURES_DIR,
32     H,
33     HM,
34     L,
35     R,
36     RESULTS_DIR,
37     T0,
38     PchipOven,
39     load_oven_table,
40 )
41
42 T_END = 10800.0
43 OUTPUT_RADII_M = 0.001 * np.arange(21)
44 TABLE_HOURS = np.array([0.5, 1.0, 1.5, 2.0, 2.5, 3.0])
45 TABLE_TIMES = 3600.0 * TABLE_HOURS
46 TABLE_RADII_CM = np.array([0.0, 0.5, 1.0, 1.5, 2.0])
47 TREF = 28.0
48
49
50 def load_q2_inputs() -> tuple[PchipOven, PchipOven, np.ndarray]:
51     t, Ta, Ca = load_oven_table()
52     mask = t <= T_END + 1e-12
53     t, Ta, Ca = t[mask], Ta[mask], Ca[mask]
54     if t.size != 181 or abs(t[-1] - T_END) > 1e-12:
55         raise ValueError(f"expected 181 oven records on [0, 10800], got {t.size}, t_end={t[-1]}")
56     if abs(Ta[0] - T0) > 1e-12:
57         raise ValueError(f"Ta(0)={Ta[0]} != T0={T0}")
58     Ta_fun = PchipOven(t, Ta)
59     Ca_fun = PchipOven(t, Ca)
60     for tj, yj in zip(t, Ta):
61         if abs(float(Ta_fun(tj)) - float(yj)) > 1e-12:
62             raise RuntimeError(f"PCHIP misses Ta knot t={tj}")
63     for tj, yj in zip(t, Ca):
64         if abs(float(Ca_fun(tj)) - float(yj)) > 1e-12:
65             raise RuntimeError(f"PCHIP misses Ca knot t={tj}")
66     return Ta_fun, Ca_fun, t
67
68
69 def integrate_coupled(fvm: CoupledRadialFVM, y0, t_end, Ta_fun, Ca_fun, rtol, atol, max_step,
70     D_const=None, source=None, t_eval=None):
71     y0 = np.asarray(y0, dtype=float)
72     n = fvm.n_nodes

```

```

73     atol_arr = np.asarray(atol, dtype=float)
74     if atol_arr.ndim == 0:
75         atol_arr = np.concatenate([np.full(n, float(atol_arr)), np.full(n, float(atol_arr) * 0.01)])
76
77     def rhs(t, y):
78         ST = SC = None
79         if source is not None:
80             ST, SC = source(t, fvm.r)
81         return fvm.rhs(t, y, float(Ta_fun(t)), float(Ca_fun(t)), D_const=D_const, ST=ST, SC=SC)
82
83     if t_eval is None:
84         t_eval = np.arange(1.0, t_end + 0.5, 1.0)
85     sol = solve_ivp(
86         rhs,
87         (0.0, float(t_end)),
88         y0,
89         method="BDF",
90         rtol=rtol,
91         atol=atol_arr,
92         max_step=max_step,
93         t_eval=t_eval,
94         dense_output=False,
95         vectorized=False,
96         jac_sparsity=fvm.jac_sparsity(),
97     )
98     if not sol.success:
99         raise RuntimeError(f"coupled BDF failed: {sol.message}")
100     times = np.concatenate([[0.0], sol.t])
101     Y = np.vstack([y0, sol.y.T])
102     return times, Y
103
104
105 def extract_output(fvm: CoupledRadialFVM, Y: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
106     n = fvm.n_nodes
107     idx = fvm.output_indices()
108     T = Y[:, :n][:, idx]
109     C = Y[:, n:][:, idx]
110     return T, C
111
112
113 def compare_on_output(A, B, times) -> dict:
114     diff = np.abs(A - B)
115     k = int(np.argmax(diff))
116     n_t, n_r = diff.shape
117     i, j = divmod(k, n_r)
118     paper_t = np.isin(times, TABLE_TIMES)
119     paper_r = slice(None, None, 5) if n_r == 21 else slice(None)
120     return {
121         "max_abs": float(diff.max()),
122         "max_time_s": float(times[i]),
123         "max_radius_cm": 0.1 * j,
124         "mean_abs": float(diff.mean()),
125         "paper_table_max_abs": float(np.max(np.abs(A[paper_t][:, paper_r] - B[paper_t][:, paper_r]))),
126     }
127
128
129 def four_decimal_stats(A, B, times) -> dict:
130     Ar, Br = np.round(A, 4), np.round(B, 4)
131     changed = Ar != Br
132     paper_t = np.isin(times, TABLE_TIMES)
133     paper_r = slice(None, None, 5) if A.shape[1] == 21 else slice(None)
134     paper_changed = Ar[paper_t][:, paper_r] != Br[paper_t][:, paper_r]
135     return {
136         "changed_cells": int(np.sum(changed)),
137         "total_cells": int(A.size),

```

```

138     "paper_changed_cells": int(np.sum(paper_changed)),
139     "paper_total_cells": int(paper_changed.size),
140     "max_abs": float(np.max(np.abs(A - B))),
141     "paper_table_max_abs": float(np.max(np.abs(A[paper_t][:, paper_r] - B[paper_t][:, paper_r]))),
142 }
143
144
145 def manufactured_fields(r: np.ndarray, t: float) -> tuple[np.ndarray, np.ndarray]:
146     x2 = (r / R) ** 2
147     T = 28.0 + 2.0 * np.exp(-t / 600.0) * (1.0 + x2)
148     C = 1.5 + 0.2 * np.exp(-t / 500.0) * (1.0 + x2)
149     return T, C
150
151
152 def manufactured_env(r_surface: float, t: float) -> tuple[float, float]:
153     T, C = manufactured_fields(np.array([r_surface]), t)
154     Ts, Cs = float(T[0]), float(C[0])
155     u = np.exp(-t / 600.0)
156     v = np.exp(-t / 500.0)
157     Tr = 4.0 / R * u
158     Cr = 4.0 / R * v * 0.1 # C amplitude 0.2 vs T amplitude 2: Cr = 0.2*v*2r/R^2 at r=R -> 0.4 v / R
159     Cr = 0.4 / R * v
160     kn = float(conductivity(Cs))
161     Dn = float(moisture_diffusivity_q2(Cs, Ts))
162     Ta = Ts + kn * Tr / H
163     Ca = Cs + Dn * Cr / HM
164     return Ta, Ca
165
166
167 class ManufacturedOven:
168     def __init__(self, kind: str):
169         self.kind = kind
170
171     def __call__(self, t):
172         Ta, Ca = manufactured_env(R, float(t))
173         return Ta if self.kind == "T" else Ca
174
175
176 def manufactured_sources(t: float, r: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
177     T, C = manufactured_fields(r, t)
178     x2 = (r / R) ** 2
179     u = np.exp(-t / 600.0)
180     v = np.exp(-t / 500.0)
181     Tt = 2.0 * (-1.0 / 600.0) * u * (1.0 + x2)
182     Ct = 0.2 * (-1.0 / 500.0) * v * (1.0 + x2)
183     Tr = 4.0 * r / (R ** 2) * u
184     Trr = np.full_like(r, 4.0 / (R ** 2) * u)
185     Cr = 0.4 * r / (R ** 2) * v
186     Crr = np.full_like(r, 0.4 / (R ** 2) * v)
187     lapT = np.empty_like(r)
188     lapC = np.empty_like(r)
189     center = r < 1e-16
190     lapT[center] = 2.0 * Trr[center]
191     lapC[center] = 2.0 * Crr[center]
192     away = ~center
193     lapT[away] = Trr[away] + Tr[away] / r[away]
194     lapC[away] = Crr[away] + Cr[away] / r[away]
195     kn = np.asarray(conductivity(C), dtype=float)
196     kp = np.asarray(conductivity_deriv(C), dtype=float)
197     Bn = np.asarray(heat_capacity(C), dtype=float)
198     D, dD_dC, dD_dT = diffusivity_partials(C, T)
199     ST = Bn * Tt - (kn * lapT + kp * Cr * Tr)
200     SC = Ct - (D * lapC + dD_dC * Cr ** 2 + dD_dT * Tr * Cr)
201     return ST, SC
202

```

```

203
204 def conservation_moisture(fvm, times, Y, Ca_fun) -> dict:
205     n = fvm.n_nodes
206     C = Y[:, n:]
207     M = C @ fvm.W
208     M0 = float(M[0])
209     CN = C[:, -1]
210     Ca = np.array([float(Ca_fun(t)) for t in times])
211     I = float(np.trapezoid(HM * (CN - Ca), times))
212     rhs = M0 - R * I
213     Mt = float(M[-1])
214     denom = max(abs(M0 - Mt), abs(R * I), 1e-18)
215     return {"M0": M0, "M_end": Mt, "rhs": rhs, "relative_residual": abs(Mt - rhs) / denom}
216
217
218 def conservation_heat(fvm, times, Y, Ta_fun) -> dict:
219     n = fvm.n_nodes
220     T = Y[:, :n]
221     C = Y[:, n:]
222     Bn = np.asarray(heat_capacity(C), dtype=float)
223     Bp = np.asarray(heat_capacity_deriv(C), dtype=float)
224     E = 2.0 * np.pi * L * np.sum(fvm.W[None, :] * Bn * (T - TREF), axis=1)
225     TN = T[:, -1]
226     Ta = np.array([float(Ta_fun(t)) for t in times])
227     Q = 2.0 * np.pi * R * L * H * float(np.trapezoid(Ta - TN, times))
228     dC = np.gradient(C, times, axis=0)
229     corr = 2.0 * np.pi * L * np.sum(fvm.W[None, :] * Bp * (T - TREF) * dC, axis=1)
230     Icorr = float(np.trapezoid(corr, times))
231     lhs = float(E[-1] - E[0] - Icorr)
232     denom = max(abs(Q), abs(lhs), 1e-12)
233     return {
234         "E_end_minus_E0": float(E[-1] - E[0]),
235         "composition_correction": Icorr,
236         "Q_surface": Q,
237         "relative_residual": abs(lhs - Q) / denom,
238     }
239
240
241 def assert_physical(T_out, C_out, Ta_fun, Ca_fun) -> dict:
242     Ta_max = float(np.max(Ta_fun.y))
243     Ca_min = float(np.min(Ca_fun.y))
244     info = {
245         "T_min": float(T_out.min()),
246         "T_max": float(T_out.max()),
247         "C_min": float(C_out.min()),
248         "C_max": float(C_out.max()),
249         "T_below_28": float(max(0.0, 28.0 - T_out.min())),
250         "T_above-Ta_max": float(max(0.0, T_out.max() - Ta_max)),
251         "C_above_C0": float(max(0.0, C_out.max() - C0)),
252         "C_below-Ca_min": float(max(0.0, Ca_min - C_out.min())),
253         "Ta_range": [float(np.min(Ta_fun.y)), Ta_max],
254         "Ca_range": [Ca_min, float(np.max(Ca_fun.y))],
255     }
256     info["T_bounds_ok"] = info["T_below_28"] <= 1e-6 and info["T_above-Ta_max"] <= 1e-4
257     info["C_bounds_ok"] = info["C_above_C0"] <= 1e-8 and info["C_below-Ca_min"] <= 1e-8
258     return info
259
260
261 def write_result_workbook(path: Path, T_out, C_out, times):
262     radii_cm = 0.1 * np.arange(21)
263     wb = Workbook()
264     for title, arr in ((" 温度", T_out), (" 水分浓度", C_out)):
265         ws = wb.create_sheet(title)
266         ws["A1"] = " 时间\\到药材中心的距离"
267         for j, rc in enumerate(radii_cm, start=2):

```

```

268         ws.cell(1, j, float(rc))
269     for i, t in enumerate(times):
270         ws.cell(i + 2, 1, int(round(float(t))))
271         for j in range(21):
272             cell = ws.cell(i + 2, j + 2, float(arr[i, j]))
273             cell.number_format = "0.0000"
274 if "Sheet" in wb.sheetnames:
275     del wb["Sheet"]
276 path.parent.mkdir(parents=True, exist_ok=True)
277 wb.save(path)
278
279
280 def write_paper_csv(path: Path, arr_out, times):
281     t_idx = {float(t): i for i, t in enumerate(times)}
282     r_idx = {round(0.1 * j, 10): j for j in range(21)}
283     lines = ["t_h," + ",".join(f"r_{rc:g}cm" for rc in TABLE_RADII_CM)]
284     for th, t in zip(TABLE_HOURS, TABLE_TIMES):
285         row = [f"{th:g}"]
286         i = t_idx[float(t)]
287         for rc in TABLE_RADII_CM:
288             j = r_idx[float(rc)]
289             row.append(f"{arr_out[i, j]:.8f}")
290         lines.append(",".join(row))
291     path.write_text("\n".join(lines) + "\n", encoding="utf-8")
292
293
294 def setup_mpl():
295     import matplotlib
296     matplotlib.use("Agg")
297     import matplotlib.pyplot as plt
298     plt.rcParams.update({
299         "font.sans-serif": ["Microsoft YaHei", "SimHei", "Noto Sans CJK SC", "DejaVu Sans"],
300         "axes.unicode_minus": False,
301         "font.size": 9,
302         "axes.labelsize": 10,
303         "legend.fontsize": 8,
304         "xtick.labelsize": 8,
305         "ytick.labelsize": 8,
306         "axes.linewidth": 0.8,
307         "pdf.fonttype": 42,
308         "ps.fonttype": 42,
309     })
310     return plt
311
312
313 def plot_figures(plt, fvm, times, Y, T_out, C_out, Ta_fun, Ca_fun):
314     FIGURES_DIR.mkdir(parents=True, exist_ok=True)
315     n = fvm.n_nodes
316     r_cm = fvm.r * 100.0
317     idx = {float(t): i for i, t in enumerate(times)}
318     marks = TABLE_TIMES
319
320     fig, ax = plt.subplots(figsize=(6.4, 3.8))
321     for t in marks:
322         ax.plot(r_cm, Y[idx[float(t)], :n], lw=1.2, label=fr"$t={t/3600:g}\$, \mathrm{{{h}}}$")
323     ax.set_xlabel(" 半径 $r$ (cm)")
324     ax.set_ylabel(" 温度 $T$ (°C)")
325     ax.set_xlim(0, 2)
326     ax.legend(frameon=False, ncol=3, loc="lower center", bbox_to_anchor=(0.5, 1.02))
327     fig.tight_layout()
328     fig.savefig(FIGURES_DIR / "q2_T_profiles.pdf")
329     plt.close(fig)
330
331     fig, ax = plt.subplots(figsize=(6.4, 3.8))
332     for t in marks:

```

```

333     ax.plot(r_cm, Y[idx[float(t)], n:], lw=1.2, label=fr"$t={t/3600:g}\,\mathrm{{{h}}}$")
334 ax.set_xlabel(" 半径 $r$ (cm)")
335 ax.set_ylabel(r" 干基含水率 $C$ (kg/kg)")
336 ax.set_xlim(0, 2)
337 ax.legend(frameon=False, ncol=2)
338 fig.tight_layout()
339 fig.savefig(FIGURES_DIR / "q2_C_profiles.pdf")
340 plt.close(fig)
341
342 t_plot = times
343 fig, ax = plt.subplots(figsize=(6.4, 3.8))
344 ax.plot(t_plot / 3600.0, [Ta_fun(t) for t in t_plot], lw=1.0, ls="--", label=" 烘房 $T_a$")
345 ax.plot(t_plot / 3600.0, T_out[:, -1], lw=1.2, label=" 表面")
346 ax.plot(t_plot / 3600.0, T_out[:, 0], lw=1.2, label=" 中心")
347 ax.set_xlabel(" 时间 $t$ (h)")
348 ax.set_ylabel(" 温度 ($^{\circ}$C)")
349 ax.set_xlim(0, 3)
350 ax.legend(frameon=False)
351 fig.tight_layout()
352 fig.savefig(FIGURES_DIR / "q2_T_center_surface.pdf")
353 plt.close(fig)
354
355 fig, ax = plt.subplots(figsize=(6.4, 3.8))
356 ax.plot(t_plot / 3600.0, C_out[:, -1], lw=1.2, label=" 表面")
357 ax.plot(t_plot / 3600.0, C_out[:, 0], lw=1.2, label=" 中心")
358 ax.set_xlabel(" 时间 $t$ (h)")
359 ax.set_ylabel(r" 干基含水率 $C$ (kg/kg)")
360 ax.set_xlim(0, 3)
361 ax.legend(frameon=False)
362 fig.tight_layout()
363 fig.savefig(FIGURES_DIR / "q2_C_center_surface.pdf")
364 plt.close(fig)
365
366 D_c = moisture_diffusivity_q2(C_out[:, 0], T_out[:, 0])
367 D_s = moisture_diffusivity_q2(C_out[:, -1], T_out[:, -1])
368 fig, ax = plt.subplots(figsize=(6.4, 3.6))
369 ax.plot(t_plot / 3600.0, D_s, lw=1.2, label=" 表面 $D_s$")
370 ax.plot(t_plot / 3600.0, D_c, lw=1.2, label=" 中心 $D_s$")
371 ax.set_xlabel(" 时间 $t$ (h)")
372 ax.set_ylabel(r"$D$ (m$^2$/s)")
373 ax.set_xlim(0, 3)
374 ax.legend(frameon=False)
375 fig.tight_layout()
376 fig.savefig(FIGURES_DIR / "q2_D_center_surface.pdf")
377 plt.close(fig)
378
379
380 def run_mms(level: int = 4) -> dict:
381     r = graded_radial_nodes(level)
382     fvm = CoupledRadialFVM(r)
383     T0f, C0f = manufactured_fields(fvm.r, 0.0)
384     y0 = fvm.pack(T0f, C0f)
385     Ta_fun = ManufacturedOven("T")
386     Ca_fun = ManufacturedOven("C")
387     times, Y = integrate_coupled(
388         fvm, y0, 100.0, Ta_fun, Ca_fun,
389         rtol=1e-8, atol=1e-9, max_step=0.5,
390         source=manufactured_sources,
391         t_eval=np.arange(1.0, 100.5, 1.0),
392     )
393     n = fvm.n_nodes
394     T_num, C_num = Y[:, :n], Y[:, n:]
395     T_ex = np.stack([manufactured_fields(fvm.r, t)[0] for t in times])
396     C_ex = np.stack([manufactured_fields(fvm.r, t)[1] for t in times])
397     return {

```

```

398     "level": level,
399     "n_nodes": n,
400     "T_max_abs": float(np.max(np.abs(T_num - T_ex))),
401     "C_max_abs": float(np.max(np.abs(C_num - C_ex))),
402     "T_end_max_abs": float(np.max(np.abs(T_num[-1] - T_ex[-1]))),
403     "C_end_max_abs": float(np.max(np.abs(C_num[-1] - C_ex[-1]))),
404 }
405
406
407 def main():
408     t_wall0 = time.perf_counter()
409     RESULTS_DIR.mkdir(parents=True, exist_ok=True)
410     Ta_fun, Ca_fun, knots = load_q2_inputs()
411     print("oven points", knots.size, "kind", Ta_fun.kind, "Ta(10800)=", Ta_fun(T_END), "Ca(10800)=",
412           ↪ Ca_fun(T_END))
413     D0 = float(moisture_diffusivity_q2(C0, T0))
414     print("initial D", D0, "k", float(conductivity(C0)), "B", float(heat_capacity(C0)))
415
416     print("manufactured-solution probe")
417     mms = run_mms(4)
418     print("MMS", mms)
419
420     mesh_levels = [2, 4]
421     rtol, atol_T, atol_C, max_step = 1e-9, 1e-8, 1e-10, 1.0
422     solutions = {}
423     for level in mesh_levels:
424         r = graded_radial_nodes(level)
425         fvm = CoupledRadialFVM(r)
426         tag = f"G{level}"
427         print(f"\n=== {tag} nodes={fvm.n_nodes} ===")
428         y0 = fvm.pack(np.full(fvm.n_nodes, T0), np.full(fvm.n_nodes, C0))
429         atol = np.concatenate([np.full(fvm.n_nodes, atol_T), np.full(fvm.n_nodes, atol_C)])
430         times, Y = integrate_coupled(fvm, y0, T_END, Ta_fun, Ca_fun, rtol, atol, max_step)
431         times_pos = times[times > 0]
432         Y_pos = Y[times > 0]
433         T_out, C_out = extract_output(fvm, Y_pos)
434         solutions[level] = dict(
435             tag=tag, fvm=fvm, times=times_pos, Y=Y_pos, Y_full=Y, times_full=times,
436             T_out=T_out, C_out=C_out, n_nodes=fvm.n_nodes,
437         )
438         print("done", tag, "T", T_out.min(), T_out.max(), "C", C_out.min(), C_out.max(),
439               ↪ "C(t=1,s)", C_out[0, -1], "C(3h,s)", C_out[-1, -1], "C(3h,c)", C_out[-1, 0])
440
441     t = solutions[4]["times"]
442     conv = {
443         "T_G2_vs_G4": compare_on_output(solutions[2]["T_out"], solutions[4]["T_out"], t),
444         "C_G2_vs_G4": compare_on_output(solutions[2]["C_out"], solutions[4]["C_out"], t),
445     }
446     fourdec = {
447         "T_G2_vs_G4": four_decimal_stats(solutions[2]["T_out"], solutions[4]["T_out"], t),
448         "C_G2_vs_G4": four_decimal_stats(solutions[2]["C_out"], solutions[4]["C_out"], t),
449     }
450     print("conv", conv)
451     print("4dp", fourdec)
452     paper_ok = fourdec["T_G2_vs_G4"]["paper_changed_cells"] == 0 and
453           ↪ fourdec["C_G2_vs_G4"]["paper_changed_cells"] == 0
454     abs_ok = conv["T_G2_vs_G4"]["max_abs"] <= 2e-5 and conv["C_G2_vs_G4"]["max_abs"] <= 2e-5
455     need_g8 = not (paper_ok and abs_ok)
456     if need_g8:
457         print("G2 vs G4 not enough; running G8")
458         level = 8
459         r = graded_radial_nodes(level)
460         fvm = CoupledRadialFVM(r)
461         y0 = fvm.pack(np.full(fvm.n_nodes, T0), np.full(fvm.n_nodes, C0))
462         atol = np.concatenate([np.full(fvm.n_nodes, atol_T), np.full(fvm.n_nodes, atol_C)])

```

```

461     times, Y = integrate_coupled(fvm, y0, T_END, Ta_fun, Ca_fun, rtol, atol, max_step)
462     times_pos = times[times > 0]
463     Y_pos = Y[times > 0]
464     T_out, C_out = extract_output(fvm, Y_pos)
465     solutions[8] = dict(
466         tag="G8", fvm=fvm, times=times_pos, Y=Y_pos, Y_full=Y, times_full=times,
467         T_out=T_out, C_out=C_out, n_nodes=fvm.n_nodes,
468     )
469     t = solutions[8]["times"]
470     conv["T_G4_vs_G8"] = compare_on_output(solutions[4]["T_out"], solutions[8]["T_out"], t)
471     conv["C_G4_vs_G8"] = compare_on_output(solutions[4]["C_out"], solutions[8]["C_out"], t)
472     fourdec["T_G4_vs_G8"] = four_decimal_stats(solutions[4]["T_out"], solutions[8]["T_out"], t)
473     fourdec["C_G4_vs_G8"] = four_decimal_stats(solutions[4]["C_out"], solutions[8]["C_out"], t)
474     print("conv G4 G8", conv["T_G4_vs_G8"], conv["C_G4_vs_G8"])
475     pub_level = 8
476     paper_ok = fourdec["T_G4_vs_G8"]["paper_changed_cells"] == 0 and
477     ↪ fourdec["C_G4_vs_G8"]["paper_changed_cells"] == 0
478     abs_ok = conv["T_G4_vs_G8"]["max_abs"] <= 2e-5 and conv["C_G4_vs_G8"]["max_abs"] <= 2e-5
479     else:
480         pub_level = 4 if fourdec["C_G2_vs_G4"]["changed_cells"] or fourdec["T_G2_vs_G4"]["changed_cells"]
481         ↪ else 2
482         if pub_level == 2:
483             pub_level = 4
484     pub = solutions[pub_level]
485     fvm = pub["fvm"]
486     times = pub["times"]
487     print("publishing", pub["tag"], "paper_ok", paper_ok, "abs_ok", abs_ok)
488
489     print("time refinement")
490     y0 = fvm.pack(np.full(fvm.n_nodes, T0), np.full(fvm.n_nodes, C0))
491     atol = np.concatenate([np.full(fvm.n_nodes, atol_T / 5), np.full(fvm.n_nodes, atol_C / 5)])
492     t2, Y2 = integrate_coupled(fvm, y0, T_END, Ta_fun, Ca_fun, rtol / 5, atol, 0.5)
493     T2, C2 = extract_output(fvm, Y2[t2 > 0])
494     time_sens = {"T": compare_on_output(pub["T_out"], T2, times), "C": compare_on_output(pub["C_out"], C2,
495     ↪ times)}
496     time_4dp = {"T": four_decimal_stats(pub["T_out"], T2, times), "C": four_decimal_stats(pub["C_out"], C2,
497     ↪ times)}
498     print("time sens", time_sens)
499
500     print("frozen-D moisture probe")
501     D_freeze = D0
502     Bi_m = HM * R / D_freeze
503     C_ref, moisture_truncation = converged_reference(Bi_m, D_freeze, R, OUTPUT_RADII_M, times, Ca_fun, C0)
504     y0f = fvm.pack(np.full(fvm.n_nodes, T0), np.full(fvm.n_nodes, C0))
505     atol = np.concatenate([np.full(fvm.n_nodes, atol_T), np.full(fvm.n_nodes, atol_C)])
506     tf, Yf = integrate_coupled(fvm, y0f, T_END, Ta_fun, Ca_fun, rtol, atol, max_step, D_const=D_freeze)
507     _, C_frozen = extract_output(fvm, Yf[tf > 0])
508     frozen = compare_on_output(C_frozen, C_ref, times)
509     nl_vs_frozen = compare_on_output(pub["C_out"], C_frozen, times)
510     print("frozen vs analytic", frozen)
511     print("nonlinear vs frozen", nl_vs_frozen)
512
513     n = fvm.n_nodes
514     D_field = moisture_diffusivity_q2(pub["Y"][:, n:], pub["Y"][:, :n])
515     D_stats = {
516         "D0": D0,
517         "D_min": float(np.min(D_field)),
518         "D_max": float(np.max(D_field)),
519         "D_min_over_D0": float(np.min(D_field) / D0),
520         "D_max_over_D0": float(np.max(D_field) / D0),
521         "center_C_end": float(pub["C_out"][-1, 0]),
522         "surface_C_end": float(pub["C_out"][-1, -1]),
523         "center_T_end": float(pub["T_out"][-1, 0]),
524         "surface_T_end": float(pub["T_out"][-1, -1]),
525     }
526 }

```

```

522     print("D stats", D_stats)
523
524     bounds = assert_physical(pub["Y_full"][:, :n], pub["Y_full"][:, n:], Ta_fun, Ca_fun)
525     cons_C = conservation_moisture(fvm, pub["times_full"], pub["Y_full"], Ca_fun)
526     cons_T = conservation_heat(fvm, pub["times_full"], pub["Y_full"], Ta_fun)
527     print("bounds", bounds)
528     print("conservation C", cons_C)
529     print("conservation T", cons_T)
530
531     delivery = make_gate({
532         "space_abs": abs_ok,
533         "space_paper": paper_ok,
534         "all_finite": np.isfinite(pub["Y_full"]).all(),
535         "manufactured_solution": mms["T_max_abs"] <= 2e-5 and mms["C_max_abs"] <= 2e-5,
536         "time_abs": time_sens["T"]["max_abs"] <= 2e-5 and time_sens["C"]["max_abs"] <= 2e-5,
537         "time_paper": time_4dp["T"]["paper_changed_cells"] == 0 and time_4dp["C"]["paper_changed_cells"] ==
538             ↪ 0,
539         "temperature_bounds": bounds["T_bounds_ok"],
540         "moisture_bounds": bounds["C_bounds_ok"],
541         "moisture_reference_truncation": moisture_truncation["passed"],
542         "frozen_reference_error": frozen["max_abs"] <= 2e-5,
543         "heat_balance": cons_T["relative_residual"] <= 1e-6,
544         "moisture_balance": cons_C["relative_residual"] <= 1e-6,
545     })
546     print("delivery", delivery)
547
548     paper_T = {f"h{th:g}_r{rc:g}": float(pub["T_out"][int(t) - 1, int(round(rc / 0.1))])
549         for th, t in zip(TABLE_HOURS, TABLE_TIMES) for rc in TABLE_RADII_CM}
550     paper_C = {f"h{th:g}_r{rc:g}": float(pub["C_out"][int(t) - 1, int(round(rc / 0.1))])
551         for th, t in zip(TABLE_HOURS, TABLE_TIMES) for rc in TABLE_RADII_CM}
552
553     validation = {
554         "result_version": "q2-closeout-v1",
555         "interpolation": Ta_fun.kind,
556         "published_mesh": pub["tag"],
557         "published_n_nodes": pub["n_nodes"],
558         "mesh_levels": list(solutions.keys()),
559         "solver": "coupled-FVM+BDF",
560         "rtol": rtol, "atol_T": atol_T, "atol_C": atol_C, "max_step": max_step,
561         "mms": mms,
562         "grid_convergence": conv,
563         "four_decimal_grid": fourdec,
564         "time_sensitivity": time_sens,
565         "four_decimal_time": time_4dp,
566         "delivery_gate": delivery,
567         "reference_truncation_C": moisture_truncation,
568         "frozen_D_moisture_vs_analytic": frozen,
569         "nonlinear_C_vs_frozen_D": nl_vs_frozen,
570         "D_variation": D_stats,
571         "conservation_T_10800s": cons_T,
572         "conservation_C_10800s": cons_C,
573         "physical_bounds": bounds,
574         "paper_table_T": paper_T,
575         "paper_table_C": paper_C,
576         "elapsed_s": time.perf_counter() - t_wall0,
577         "python": sys.version,
578         "end_face_2d": "skipped_by_plan",
579     }
580     require_delivery(validation, RESULTS_DIR / "diagnostics/q2_delivery/latest.json")
581     np.savez(
582         RESULTS_DIR / "q2_solution.npz",
583         mesh_level=pub_level,
584         n_nodes=pub["n_nodes"],
585         times=times,
586         r=fvm.r,

```

```

586     output_radai_m=OUTPUT_RADII_M,
587     checkpoint_times=np.r_[0., 1., 60., 100., TABLE_TIMES],
588     T_checkpoints=pub["Y_full"][np.r_[0., 1., 60., 100., TABLE_TIMES].astype(int), :n],
589     C_checkpoints=pub["Y_full"][np.r_[0., 1., 60., 100., TABLE_TIMES].astype(int), n:],
590     T_out=pub["T_out"],
591     C_out=pub["C_out"],
592     interpolation=np.array(Ta_fun.kind),
593     rtol=rtol, atol_T=atol_T, atol_C=atol_C, max_step=max_step,
594 )
595 write_result_workbook(RESULTS_DIR / "result2.xlsx", pub["T_out"], pub["C_out"], times)
596 write_paper_csv(RESULTS_DIR / "q2_table_temperature.csv", pub["T_out"], times)
597 write_paper_csv(RESULTS_DIR / "q2_table_moisture.csv", pub["C_out"], times)
598 plt = setup_mpl()
599 plot_figures(plt, fvm, times, pub["Y"], pub["T_out"], pub["C_out"], Ta_fun, Ca_fun)
600
601 (RESULTS_DIR / "q2_validation.json").write_text(
602     json.dumps(validation, ensure_ascii=False, indent=2, default=float),
603     encoding="utf-8",
604 )
605 print("elapsed_s", validation["elapsed_s"])
606 print("wrote results to", RESULTS_DIR)
607
608
609
610 if __name__ == "__main__":
611     main()

```

B.9 code/problem3.py

```

1 """Problem 3: full coupled drying until every interior node is below 0.15 kg/kg."""
2 from __future__ import annotations
3
4 import argparse
5 import json
6 import sys
7 import time
8 from pathlib import Path
9
10 import numpy as np
11 from scipy.integrate import solve_ivp
12 from scipy.optimize import brentq
13
14 CODE_DIR = Path(__file__).resolve().parent
15 if str(CODE_DIR) not in sys.path:
16     sys.path.insert(0, str(CODE_DIR))
17
18 from appendix3 import moisture_diffusivity_q2
19 from delivery import make_gate, require_delivery
20 from problem2 import conservation_heat, conservation_moisture, extract_output, setup_mpl
21 from radial_coupled import CoupledRadialFVM
22 from radial_fvm import graded_radial_nodes
23 from utils import (
24     C0,
25     FIGURES_DIR,
26     RESULTS_DIR,
27     T0,
28     PchipOven,
29     load_oven_table,
30 )
31
32 T_SWITCH = 14400.0
33 TA_AFTER = 50.0
34 CA_AFTER = 0.05
35 C_THRESH = 0.15
36 T_HARD_CAP = 21.0 * 24.0 * 3600.0

```

```

37 OUTPUT_RADII_M = 0.001 * np.arange(21)
38 TABLE_RADII_CM = np.array([0.0, 0.5, 1.0, 1.5, 2.0])
39 EXCEL_DT = 60.0
40 TABLE_DT_H = 6.0
41 OVEN_N_EXPECTED = 241
42
43 # First-round baseline; recorded after the run, not treated as a prior user decision.
44 RTOL = 1e-9
45 ATOL_T = 1e-8
46 ATOL_C = 1e-10
47 MAX_STEP_EARLY = 2.0
48 MAX_STEP_LATE = 60.0
49
50
51 class ExtendedOven:
52     """PCHIP through attachment 1 on [0, 4 h]; constant value strictly after 4 h."""
53
54     def __init__(self, pchip: PchipOven, after_value: float, t_switch: float = T_SWITCH):
55         self.pchip = pchip
56         self.after = float(after_value)
57         self.t_switch = float(t_switch)
58         self.kind = pchip.kind
59         self.t = pchip.t
60         self.y = pchip.y
61
62     def __call__(self, t):
63         if np.ndim(t) == 0:
64             tt = float(t)
65             if tt <= self.t_switch:
66                 return float(self.pchip(tt))
67             return self.after
68         tt = np.asarray(t, dtype=float)
69         out = np.empty_like(tt, dtype=float)
70         inside = tt <= self.t_switch
71         if np.any(inside):
72             out[inside] = np.asarray(self.pchip(tt[inside]), dtype=float)
73         out[~inside] = self.after
74         return out
75
76
77 def load_q3_inputs() -> tuple[ExtendedOven, ExtendedOven, np.ndarray]:
78     t, Ta, Ca = load_oven_table()
79     if t.size != OVEN_N_EXPECTED or abs(t[-1] - T_SWITCH) > 1e-12:
80         raise ValueError(f"expected {OVEN_N_EXPECTED} oven records ending at {T_SWITCH}, got {t.size},
81             ↳ t_end={t[-1]}")
82     if abs(Ta[0] - T0) > 1e-12:
83         raise ValueError(f"Ta(0)={Ta[0]} != T0={T0}")
84     Ta_pchip = PchipOven(t, Ta)
85     Ca_pchip = PchipOven(t, Ca)
86     for tj, yj in zip(t, Ta):
87         if abs(float(Ta_pchip(tj)) - float(yj)) > 1e-12:
88             raise RuntimeError(f"PCHIP misses Ta knot t={tj}")
89     for tj, yj in zip(t, Ca):
90         if abs(float(Ca_pchip(tj)) - float(yj)) > 1e-12:
91             raise RuntimeError(f"PCHIP misses Ca knot t={tj}")
92     Ta_fun = ExtendedOven(Ta_pchip, TA_AFTER)
93     Ca_fun = ExtendedOven(Ca_pchip, CA_AFTER)
94     jump = {
95         "Ta_at_4h": float(Ta_fun(T_SWITCH)),
96         "Ca_at_4h": float(Ca_fun(T_SWITCH)),
97         "Ta_after": float(Ta_fun(T_SWITCH + 1e-9)),
98         "Ca_after": float(Ca_fun(T_SWITCH + 1e-9)),
99     }
100     print("oven 4h jump", jump, flush=True)
101     return Ta_fun, Ca_fun, t

```

```

101
102
103 def atol_vector(n: int, atol_T: float = ATOL_T, atol_C: float = ATOL_C) -> np.ndarray:
104     return np.concatenate([np.full(n, float(atol_T)), np.full(n, float(atol_C))])
105
106
107 def max_moisture_event(n_nodes: int):
108     def event(t, y):
109         return float(np.max(y[n_nodes:]) - C_THRESH)
110
111     event.terminal = True
112     event.direction = -1.0
113     return event
114
115
116 def integrate_segment(
117     fvm: CoupledRadialFVM,
118     y0,
119     t0: float,
120     t1: float,
121     Ta_fun,
122     Ca_fun,
123     rtol: float,
124     atol: np.ndarray,
125     max_step: float,
126     *,
127     t_eval=None,
128     events=None,
129     dense_output: bool = False,
130     first_step=None,
131     label: str = "",
132     balance_audit: bool = False,
133 ):
134     n = fvm.n_nodes
135     y0 = np.asarray(y0, dtype=float)
136     t0 = float(t0)
137     t1 = float(t1)
138     if t1 <= t0 + 1e-15:
139         raise ValueError(f"empty interval [{t0}, {t1}]")
140     nfev = [0]
141     t_print = [t0]
142     t_wall0 = time.perf_counter()
143
144     def rhs(t, y):
145         nfev[0] += 1
146         interval = 3600.0 if t < T_SWITCH + 12.0 * 3600.0 else 6.0 * 3600.0
147         if t - t_print[0] >= interval:
148             C = y[n:]
149             imax = int(np.argmax(C))
150             elapsed = time.perf_counter() - t_wall0
151             print(
152                 f"    {label}t={t/3600.0:.3f} h    maxC={float(C[imax]):.6f} "
153                 f"rmax={fvm.r[imax]*100:.4f} cm    nfev={nfev[0]}    wall={elapsed:.1f}s",
154                 flush=True,
155             )
156             t_print[0] = float(t)
157         return fvm.rhs(t, y, float(Ta_fun(t)), float(Ca_fun(t)))
158
159     kwargs = dict(
160         fun=rhs,
161         t_span=(t0, t1),
162         y0=y0,
163         method="BDF",
164         rtol=rtol,
165         atol=np.asarray(atol, dtype=float),

```

```

166         max_step=max_step,
167         dense_output=dense_output or balance_audit,
168         vectorized=False,
169         jac_sparsity=fvm.jac_sparsity(),
170     )
171     if t_eval is not None:
172         te = np.asarray(t_eval, dtype=float)
173         te = te[(te > t0 + 1e-14) & (te <= t1 + 1e-12)]
174         if te.size:
175             kwargs["t_eval"] = te
176     if events is not None:
177         kwargs["events"] = events
178     if first_step is not None:
179         kwargs["first_step"] = min(float(first_step), 0.5 * max_step, t1 - t0)
180
181     sol = solve_ivp(**kwargs)
182     if not sol.success:
183         raise RuntimeError(f"coupled BDF failed on [{t0}, {t1}] {label}: {sol.message}")
184     if balance_audit:
185         from q3_balance import audit_segment
186         sol.balance_audit = audit_segment(fvm, sol, Ta_fun, Ca_fun)
187     if not dense_output:
188         sol.sol = None
189     return sol
190
191
192 def excel_times(t_rep: float) -> np.ndarray:
193     t_rep = float(t_rep)
194     grid = np.arange(EXCEL_DT, np.floor(t_rep / EXCEL_DT) * EXCEL_DT + 0.5 * EXCEL_DT, EXCEL_DT)
195     if grid.size == 0 or grid[-1] < t_rep - 1e-12:
196         times = np.concatenate([grid, [t_rep]]) if grid.size else np.array([t_rep])
197     elif abs(grid[-1] - t_rep) <= 1e-9:
198         times = grid
199     else:
200         times = np.concatenate([grid, [t_rep]])
201     return times
202
203
204 def table_hours(t_rep_h: float) -> np.ndarray:
205     t_rep_h = float(t_rep_h)
206     hrs = np.arange(TABLE_DT_H, t_rep_h + 1e-12, TABLE_DT_H)
207     if hrs.size == 0 or abs(hrs[-1] - t_rep_h) > 1e-10:
208         hrs = np.concatenate([hrs, [t_rep_h]]) if hrs.size else np.array([t_rep_h])
209     return hrs
210
211
212 def monitor_from_Y(fvm: CoupledRadialFVM, times: np.ndarray, Y: np.ndarray) -> dict:
213     n = fvm.n_nodes
214     C = Y[:, n:]
215     imax = np.argmax(C, axis=1)
216     M = C[np.arange(C.shape[0]), imax]
217     rmax = fvm.r[imax]
218     dC = np.diff(C, axis=1)
219     # Allow tiny positive bumps from rounding; count clear interior increases.
220     n_increase = int(np.sum(dC > 1e-10))
221     center_is_max = imax == 0
222     return {
223         "times": times,
224         "M": M,
225         "rmax_m": rmax,
226         "imax": imax,
227         "center_is_max_frac": float(np.mean(center_is_max)),
228         "n_radial_increases": n_increase,
229         "C_center": C[:, 0].copy(),
230         "C_surface": C[:, -1].copy(),

```

```

231         "T_center": Y[:, 0].copy(),
232         "T_surface": Y[:, n - 1].copy(),
233     }
234
235
236 def locate_threshold(
237     fvm: CoupledRadialFVM,
238     Ta_fun,
239     Ca_fun,
240     t_left: float,
241     y_left: np.ndarray,
242     t_right: float,
243     rtol: float,
244     atol: np.ndarray,
245 ):
246     """Bracketed root of max_i C_i(t) - 0.15 on a locally refined dense interpolant."""
247     n = fvm.n_nodes
248     event = max_moisture_event(n)
249     t_pad = max(30.0, 2.0 * (t_right - t_left))
250     t_hi = t_right + t_pad
251     sol = integrate_segment(
252         fvm, y_left, t_left, t_hi, Ta_fun, Ca_fun, rtol, atol,
253         max_step=0.5, events=event, dense_output=True, first_step=0.05,
254         label="zoom ",
255     )
256     if sol.t_events is None or len(sol.t_events[0]) == 0:
257         C_end = sol.y[n:, -1]
258         raise RuntimeError(
259             f"zoom missed threshold on [{t_left}, {t_hi}]; "
260             f"maxC(end)={float(np.max(C_end))}"
261         )
262     t_solver = float(sol.t_events[0][0])
263     y_solver = np.asarray(sol.y_events[0][0], dtype=float)
264
265     def g(t):
266         return float(np.max(sol.sol(float(t))[n:]) - C_THRESH)
267
268     g_left = g(t_left)
269     t_lo = t_left
270     t_hi_root = float(sol.t[-1])
271     g_hi = g(t_hi_root)
272     if g_left <= 0.0:
273         raise RuntimeError(f"zoom left end already <= threshold: t={t_left}, g={g_left}")
274     if g_hi > 0.0 and abs(g_hi) <= 1e-8:
275         t_star = t_solver
276     elif g_hi > 0.0:
277         raise RuntimeError(f"zoom right end still above threshold: t={t_hi_root}, g={g_hi}")
278     else:
279         t_star = float(brentq(g, t_lo, t_hi_root, xtol=1e-4, rtol=1e-12, maxiter=200))
280     y_star = np.asarray(sol.sol(t_star), dtype=float)
281     C_star = y_star[n:]
282     imax = int(np.argmax(C_star))
283     info = {
284         "t_star_s": t_star,
285         "t_star_h": t_star / 3600.0,
286         "t_solver_event_s": t_solver,
287         "event_solver_minus_brent_s": t_solver - t_star,
288         "M_star": float(C_star[imax]),
289         "g_star": float(C_star[imax] - C_THRESH),
290         "rmax_star_m": float(fvm.r[imax]),
291         "rmax_star_cm": float(fvm.r[imax] * 100.0),
292         "imax_star": imax,
293         "center_is_max": bool(imax == 0),
294         "C_center_star": float(C_star[0]),
295         "C_surface_star": float(C_star[-1]),

```

```

296         "T_center_star": float(y_star[0]),
297         "T_surface_star": float(y_star[n - 1]),
298         "bracket_s": [float(t_left), float(t_hi_root)],
299         "zoom_nfev": int(sol.nfev),
300     }
301     return t_star, y_star, sol, info, y_solver
302
303
304 def find_strict_report(eval_M, t_star: float, t_hi: float, margin: float = 0.0) -> dict:
305     target = C_THRESH - float(margin)
306     t_star = float(t_star)
307     t_hi = float(t_hi)
308     if not (np.isfinite([t_star, t_hi, margin]).all() and 0 <= margin < C_THRESH and t_hi > t_star):
309         raise ValueError('invalid strict-report interval or margin')
310
311     original_eval = eval_M
312     def eval_M(t):
313         if not t_star <= t <= t_hi:
314             raise RuntimeError('strict report requested outside verified continuation')
315         value = float(original_eval(t))
316         if not np.isfinite(value):
317             raise RuntimeError('nonfinite moisture in strict report')
318         return value
319
320     def h(t):
321         return float(eval_M(t) - target)
322
323     h0 = h(t_star)
324     h1 = h(t_hi)
325     if h1 >= 0.0:
326         raise RuntimeError(f"M(t_hi)={eval_M(t_hi)} still >= {target}")
327     if h0 < 0.0:
328         t_cross = t_star
329     else:
330         t_cross = float(brentq(h, t_star, t_hi, xtol=1e-4, rtol=1e-12, maxiter=200))
331     t_rep = float(np.floor(max(t_cross, t_star)) + 1.0)
332     # Walk forward in seconds until unrounded moisture is strictly below the target.
333     for _ in range(10000):
334         if float(eval_M(t_rep)) < target:
335             break
336         t_rep += 1.0
337     else:
338         raise RuntimeError("failed to find integer-second report time below threshold")
339     t_hour4 = np.ceil((t_rep / 3600.0) * 10000.0) / 10000.0
340     t_hour4_s = 3600.0 * t_hour4
341     if float(eval_M(t_hour4_s)) >= target:
342         raise RuntimeError("four-decimal hour report does not satisfy the target")
343     t_grid60 = EXCEL_DT * np.ceil(t_star / EXCEL_DT)
344     if t_grid60 < t_rep:
345         t_grid60 = EXCEL_DT * np.ceil(t_rep / EXCEL_DT)
346     return {
347         "t_cross_s": t_cross,
348         "t_rep_s": float(t_rep),
349         "t_rep_h": float(t_rep) / 3600.0,
350         "M_rep": float(eval_M(t_rep)),
351         "t_hour4_h": float(t_hour4),
352         "t_hour4_s": float(t_hour4_s),
353         "M_hour4": float(eval_M(t_hour4_s)),
354         "t_grid60_s": float(t_grid60),
355         "M_grid60": float(eval_M(t_grid60)),
356         "margin": float(margin),
357         "strict_below_rep": bool(float(eval_M(t_rep)) < C_THRESH),
358         "strict_below_hour4": bool(float(eval_M(t_hour4_s)) < C_THRESH),
359     }
360

```

```

361
362 def slope_at(eval_M, t: float, h: float = 1.0) -> float:
363     return float(eval_M(t + h) - eval_M(t - h)) / (2.0 * h)
364
365
366 def run_case(
367     level: int,
368     Ta_fun,
369     Ca_fun,
370     t_max: float,
371     rtol: float,
372     atol_T: float,
373     atol_C: float,
374     max_step_early: float,
375     max_step_late: float,
376     tag: str,
377     audit_balances: bool = False,
378 ):
379     r = graded_radial_nodes(level)
380     fvm = CoupledRadialFVM(r)
381     n = fvm.n_nodes
382     print(f"\n=== {tag} nodes={n} t_max={t_max/3600:.2f} h ===", flush=True)
383     y0 = fvm.pack(np.full(n, T0), np.full(n, C0))
384     atol = atol_vector(n, atol_T, atol_C)
385     t_eval_early = np.arange(EXCEL_DT, T_SWITCH + 0.5 * EXCEL_DT, EXCEL_DT)
386     t_wall0 = time.perf_counter()
387     sol1 = integrate_segment(
388         fvm, y0, 0.0, T_SWITCH, Ta_fun, Ca_fun, rtol, atol,
389         max_step=max_step_early, t_eval=t_eval_early, first_step=0.05, label=f"{tag} early ",
390         ↪ balance_audit=audit_balances,
391     )
392     y_switch = sol1.y[:, -1]
393     times1 = np.concatenate([[0.0], sol1.t])
394     Y1 = np.vstack([y0, sol1.y.T])
395     M_switch = float(np.max(y_switch[n:]))
396     print(f" {tag} 4h maxC={M_switch:.6f} C_center={y_switch[n]:.6f}
397     ↪ wall={time.perf_counter()-t_wall0:.1f}s", flush=True)
398     if M_switch <= C_THRESH:
399         raise RuntimeError("threshold already reached at 4 h; unexpected for this model")
400
401     event = max_moisture_event(n)
402     t_eval_late = np.unique(np.r_[np.arange(T_SWITCH + EXCEL_DT, t_max, EXCEL_DT), t_max])
403     sol2 = integrate_segment(
404         fvm, y_switch, T_SWITCH, t_max, Ta_fun, Ca_fun, rtol, atol,
405         max_step=max_step_late, t_eval=t_eval_late, events=event,
406         first_step=0.2, label=f"{tag} late ", dense_output=audit_balances,
407     )
408     if sol2.t_events is None or len(sol2.t_events[0]) == 0:
409         M_end = float(np.max(sol2.y[n:, -1]))
410         print(f" {tag} no event by {t_max/3600:.2f} h, maxC={M_end:.6f}; extending to hard cap",
411             ↪ flush=True)
412         if t_max >= T_HARD_CAP - 1.0:
413             raise RuntimeError(f"{tag} still above threshold at {T_HARD_CAP/3600:.1f} h, maxC={M_end}")
414         y_cont = sol2.y[:, -1]
415         t_cont = float(sol2.t[-1])
416         t_eval_cap = np.arange(t_cont + EXCEL_DT, T_HARD_CAP + 0.5 * EXCEL_DT, EXCEL_DT)
417         sol2b = integrate_segment(
418             fvm, y_cont, t_cont, T_HARD_CAP, Ta_fun, Ca_fun, rtol, atol,
419             max_step=max_step_late, t_eval=t_eval_cap, events=event,
420             first_step=0.2, label=f"{tag} cap ", dense_output=audit_balances,
421         )
422         if sol2b.t_events is None or len(sol2b.t_events[0]) == 0:
423             raise RuntimeError(
424                 f"{tag} not dry at {T_HARD_CAP/3600:.1f} h, maxC={float(np.max(sol2b.y[n:, -1]))}"
425             )

```

```

423     times2 = np.concatenate([sol2.t[sol2.t > T_SWITCH + 1e-12], sol2b.t[sol2b.t > t_cont + 1e-12]])
424     Y2 = np.vstack([sol2.y[:, sol2.t > T_SWITCH + 1e-12].T, sol2b.y[:, sol2b.t > t_cont + 1e-12].T])
425     t_event_raw = float(sol2b.t_events[0][0])
426     y_event_raw = np.asarray(sol2b.y_events[0][0], dtype=float)
427     else:
428         times2 = sol2.t[sol2.t > T_SWITCH + 1e-12]
429         Y2 = sol2.y[:, sol2.t > T_SWITCH + 1e-12].T
430         t_event_raw = float(sol2.t_events[0][0])
431         y_event_raw = np.asarray(sol2.y_events[0][0], dtype=float)
432
433     times = np.concatenate([times1, times2])
434     Y = np.vstack([Y1, Y2])
435     order = np.argsort(times)
436     times, Y = times[order], Y[order]
437     uniq = np.concatenate([[True], np.diff(times) > 1e-12])
438     times, Y = times[uniq], Y[uniq]
439
440     M_hist = np.max(Y[:, n:], axis=1)
441     above = np.where(M_hist > C_THRESH + 1e-14)[0]
442     if above.size == 0:
443         raise RuntimeError(f"{tag} never above threshold")
444     i_left = int(above[-1])
445     t_left = float(times[i_left])
446     y_left = Y[i_left]
447     if t_left >= t_event_raw:
448         i_left = max(0, i_left - 1)
449         t_left = float(times[i_left])
450         y_left = Y[i_left]
451     t_star, y_star, sol_zoom, event_info, y_solver = locate_threshold(
452         fvm, Ta_fun, Ca_fun, t_left, y_left, t_event_raw, rtol, atol,
453     )
454     event_info["t_event_raw_s"] = t_event_raw
455     event_info["M_event_raw"] = float(np.max(y_event_raw[n:]))
456
457     balance_parts = []
458     if audit_balances:
459         from q3_balance import audit_segment
460         # Follow the same trajectory used for reporting, including the zoom restart.
461         balance_parts = [sol1.balance_audit]
462         if 'sol2b' in locals():
463             balance_parts.append(audit_segment(fvm, sol2, Ta_fun, Ca_fun))
464             late_sol = sol2b
465         else:
466             late_sol = sol2
467         balance_parts.append(audit_segment(fvm, late_sol, Ta_fun, Ca_fun, t_end=t_left))
468         balance_parts.append(audit_segment(fvm, sol_zoom, Ta_fun, Ca_fun, t_end=t_star))
469         sol2.sol = None
470         if 'sol2b' in locals():
471             sol2b.sol = None
472
473     t_pad = 3600.0
474     sol_after = integrate_segment(
475         fvm, y_star, t_star, t_star + t_pad, Ta_fun, Ca_fun, rtol, atol,
476         max_step=min(max_step_late, 5.0), dense_output=True, first_step=0.05,
477         label=f"{tag} after ",
478     )
479
480     def eval_M(t):
481         return float(np.max(eval_y(float(t))[n:]))
482
483     def eval_y(t):
484         tt = float(t)
485         if abs(tt - t_star) <= 1e-12:
486             return y_star.copy()
487         if tt < t_star:

```

```

488         if tt < times[0]:
489             raise ValueError(f"state requested before initial time: {tt}")
490         if tt >= t_left:
491             return np.asarray(sol_zoom.sol(tt), dtype=float)
492         k = max(0, int(np.searchsorted(times, tt, side="right"))-1)
493         if abs(times[k]-tt)<1e-8:
494             return Y[k].copy()
495         short = integrate_segment(fvm,Y[k],float(times[k]),tt,Ta_fun,Ca_fun,
496                                 min(rtol,1e-10),atol/10,max_step=1.0,first_step=0.05,label="matched time ")
497         return short.y[:,-1]
498     if tt > float(sol_after.t[-1])+1e-9:
499         raise ValueError(f"state requested beyond continuation: {tt}")
500     return np.asarray(sol_after.sol(tt), dtype=float)
501
502     dMdt = slope_at(eval_M, min(t_star + 2.0, t_star + 0.5 * t_pad), h=1.0)
503     event_info["dMdt_per_s"] = dMdt
504     event_info["dMdt_per_h"] = dMdt * 3600.0
505
506     T_out, C_out = extract_output(fvm, Y)
507     mon = monitor_from_Y(fvm, times, Y)
508     elapsed = time.perf_counter() - t_wall0
509     print(
510         f" {tag} t*={t_star/3600:.6f} h  rmax={event_info['rmax_star_cm']:.4f} cm  "
511         f"M*={event_info['M_star']:.8f}  dM/dt={dMdt:.3e}/s  wall={elapsed:.1f}s",
512         flush=True,
513     )
514     return dict(
515         tag=tag, level=level, fvm=fvm, n_nodes=n, times=times, Y=Y,
516         T_out=T_out, C_out=C_out, t_star=t_star, y_star=y_star,
517         eval_M=eval_M, eval_y=eval_y, sol_after=sol_after, event=event_info,
518         monitor=mon, elapsed_s=elapsed, rtol=rtol, atol_T=atol_T, atol_C=atol_C,
519         max_step_early=max_step_early, max_step_late=max_step_late, t_max=t_max,
520         y_switch=y_switch,
521         balance_parts=balance_parts,
522     )
523
524
525 def run_probe(Ta_fun, Ca_fun) -> dict:
526     """Cheap uniform-mesh estimate of t* to size the production interval."""
527     r = np.linspace(0.0, 0.02, 41)
528     fvm = CoupledRadialFVM(r)
529     n = fvm.n_nodes
530     y0 = fvm.pack(np.full(n, T0), np.full(n, C0))
531     atol = atol_vector(n, 1e-6, 1e-8)
532     print("\n=== probe N=41 ===", flush=True)
533     sol1 = integrate_segment(
534         fvm, y0, 0.0, T_SWITCH, Ta_fun, Ca_fun, 1e-6, atol, max_step=30.0,
535         first_step=0.2, label="probe early ",
536     )
537     event = max_moisture_event(n)
538     sol2 = integrate_segment(
539         fvm, sol1.y[:, -1], T_SWITCH, T_HARD_CAP, Ta_fun, Ca_fun, 1e-6, atol,
540         max_step=120.0, events=event, first_step=1.0, label="probe late ",
541     )
542     if sol2.t_events is None or len(sol2.t_events[0]) == 0:
543         raise RuntimeError(f"probe not dry by 14 d, maxC={float(np.max(sol2.y[n:, -1]))}")
544     t_star = float(sol2.t_events[0][0])
545     y_e = np.asarray(sol2.y_events[0][0], dtype=float)
546     imax = int(np.argmax(y_e[n:]))
547     info = {
548         "n_nodes": n,
549         "t_star_s": t_star,
550         "t_star_h": t_star / 3600.0,
551         "M_star": float(y_e[n + imax]),
552         "rmax_cm": float(fvm.r[imax] * 100.0),

```

```

553         "C_center": float(y_e[n]),
554         "C_surface": float(y_e[-1]),
555         "nfev": int(sol1.nfev + sol2.nfev),
556     }
557     print("probe", info, flush=True)
558     return info
559
560
561 def overlap_q2(case) -> dict:
562     path = RESULTS_DIR / "q2_solution.npz"
563     if not path.exists():
564         return {"available": False}
565     z = np.load(path)
566     fvm = case["fvm"]
567     n = fvm.n_nodes
568     idx = fvm.output_indices()
569     t_check = np.array([1800.0, 3600.0, 7200.0, 9000.0])
570     q2_times = np.asarray(z["times"], dtype=float)
571     T_q2 = np.asarray(z["T_out"], dtype=float)
572     C_q2 = np.asarray(z["C_out"], dtype=float)
573     rows = {}
574     for t in t_check:
575         k = int(np.argmin(np.abs(case["times"] - t)))
576         if abs(case["times"][k] - t) > 1e-8:
577             rows[f"t{int(t)}"] = {"matched": False}
578             continue
579         jq = int(np.argmin(np.abs(q2_times - t)))
580         if abs(q2_times[jq] - t) > 1e-8:
581             rows[f"t{int(t)}"] = {"matched": False}
582             continue
583         T = case["Y"][k, :n][idx]
584         C = case["Y"][k, n:][idx]
585         rows[f"t{int(t)}"] = {
586             "matched": True,
587             "T_max_abs": float(np.max(np.abs(T - T_q2[jq]))),
588             "C_max_abs": float(np.max(np.abs(C - C_q2[jq]))),
589             "T_center_diff": float(T[0] - T_q2[jq, 0]),
590             "C_center_diff": float(C[0] - C_q2[jq, 0]),
591             "C_surface_diff": float(C[-1] - C_q2[jq, -1]),
592         }
593     return {"available": True, "note": "PCHIP stencils differ near 3 h endpoint", "checks": rows}
594
595
596 def sample_state(case, t: float) -> tuple[np.ndarray, np.ndarray]:
597     fvm = case["fvm"]
598     n = fvm.n_nodes
599     y = case["eval_y"](float(t))
600     T, C = y[:n], y[n:]
601     return T, C
602
603
604 def output_row(fvm, C: np.ndarray) -> np.ndarray:
605     return np.asarray(C[fvm.output_indices()], dtype=float)
606
607
608 def build_output_tables(case, t_rep: float) -> dict:
609     fvm = case["fvm"]
610     times_x = excel_times(t_rep)
611     C_excel = np.empty((times_x.size, 21), dtype=float)
612     T_excel = np.empty_like(C_excel)
613     n = fvm.n_nodes
614     idx = fvm.output_indices()
615     t_star = case["t_star"]
616     times = case["times"]
617     Y = case["Y"]

```

```

618     for i, t in enumerate(times_x):
619         if t <= times[-1] + 1e-12 and t < t_star - 1e-9:
620             j = int(np.argmin(np.abs(times - t)))
621             if abs(times[j] - t) > 1.0:
622                 raise RuntimeError(f"missing 60 s sample at t={t}, nearest {times[j]}")
623             T_excel[i] = Y[j, :n][idx]
624             C_excel[i] = Y[j, n:][idx]
625         else:
626             y = case["eval_y"](float(t))
627             T_excel[i] = y[:n][idx]
628             C_excel[i] = y[n:][idx]
629     hrs = table_hours(t_rep / 3600.0)
630     C_table = np.empty((hrs.size, TABLE_RADII_CM.size), dtype=float)
631     paper_map = {}
632     r_idx = {round(0.1 * j, 10): j for j in range(21)}
633     for i, th in enumerate(hrs):
634         t = 3600.0 * float(th)
635         k = int(np.argmin(np.abs(times_x - t)))
636         if abs(times_x[k] - t) > 1.0 and abs(t - t_rep) > 1.0:
637             # 6 h marks lie on the 60 s grid; end time may be the extra row.
638             y = case["eval_y"](t) if t >= t_star - 1e-9 else None
639             if y is None:
640                 j = int(np.argmin(np.abs(times - t)))
641                 row = Y[j, n:][idx]
642             else:
643                 row = y[n:][idx]
644         else:
645             row = C_excel[k]
646         for rc, col in zip(TABLE_RADII_CM, range(5)):
647             C_table[i, col] = float(row[r_idx[float(rc)]]])
648             paper_map[f"h{th:.6g}_r{rc:g}"] = float(row[r_idx[float(rc)]]])
649     return {
650         "excel_times": times_x,
651         "C_excel": C_excel,
652         "T_excel": T_excel,
653         "table_hours": hrs,
654         "C_table": C_table,
655         "paper_C": paper_map,
656     }
657
658
659 def write_result3_xlsx(path: Path, C_out: np.ndarray, times: np.ndarray, *, validation):
660     """All numerical checks must pass before calling the workbook author."""
661     from q3_closeout import export_workbook, require_q3_delivery
662     require_q3_delivery(validation, RESULTS_DIR / "diagnostics/q3_closeout/pre_export.json")
663     export_workbook(path, C_out, times, validation)
664
665
666 def write_table5_csv(path: Path, hours: np.ndarray, C_table: np.ndarray):
667     lines = ["t_h," + ",".join(f"r_{rc:g}cm" for rc in TABLE_RADII_CM)]
668     for th, row in zip(hours, C_table):
669         bits = [f"{float(th):.8f}"] + [f"{float(v):.10f}" for v in row]
670         lines.append(",".join(bits))
671     path.write_text("\n".join(lines) + "\n", encoding="utf-8")
672
673
674 def physical_bounds(case, Ta_fun, Ca_fun) -> dict:
675     n = case["n_nodes"]
676     T = case["Y"][:, :n]
677     C = case["Y"][:, n:]
678     Ta_max = max(float(np.max(Ta_fun.y)), TA_AFTER)
679     info = {
680         "T_min": float(T.min()),
681         "T_max": float(T.max()),
682         "C_min": float(C.min()),

```

```

683     "C_max": float(C.max()),
684     "T_below_28": float(max(0.0, 28.0 - T.min())),
685     "T_above-Ta_max": float(max(0.0, T.max() - Ta_max)),
686     "C_above_C0": float(max(0.0, C.max() - C0)),
687     "C_negative": float(max(0.0, -C.min())),
688     "Ta_max": Ta_max,
689     "Ca_range_knots": [float(np.min(Ca_fun.y)), float(np.max(Ca_fun.y))],
690 }
691 info["T_bounds_ok"] = info["T_below_28"] <= 1e-6 and info["T_above-Ta_max"] <= 0.05
692 info["C_bounds_ok"] = info["C_above_C0"] <= 1e-8 and info["C_negative"] <= 1e-12
693 return info
694
695
696 def plot_figures(plt, pub, tables, t_rep: float, Ta_fun):
697     FIGURES_DIR.mkdir(parents=True, exist_ok=True)
698     fvm = pub["fvm"]
699     n = fvm.n_nodes
700     mon = pub["monitor"]
701     t_h = mon["times"] / 3600.0
702     t_star_h = pub["t_star"] / 3600.0
703     t_rep_h = t_rep / 3600.0
704
705     fig, ax = plt.subplots(figsize=(6.4, 3.8))
706     ax.axhline(C_THRESH, color="0.4", lw=1.0, ls="--", label=r" 阈值 $0.15$")
707     ax.plot(t_h, mon["M"], lw=1.2, label=r" 全网格 $\max C$")
708     ax.plot(t_h, mon["C_center"], lw=1.2, ls="-. ", label=" 中心")
709     ax.plot(t_h, mon["C_surface"], lw=1.2, ls=":", label=" 表面")
710     ax.axvline(t_star_h, color="0.2", lw=0.8, ls=":")
711     ax.set_xlabel(" 时间 $t$ (h)")
712     ax.set_ylabel(r" 干基含水率 $C$ (kg/kg)")
713     ax.set_xlim(0, t_rep_h)
714     ax.legend(frameon=False)
715     fig.tight_layout()
716     fig.savefig(FIGURES_DIR / "q3_C_max_history.pdf")
717     plt.close(fig)
718
719     fig, ax = plt.subplots(figsize=(6.4, 3.6))
720     ax.plot(t_h, mon["rmax_m"] * 100.0, lw=1.2)
721     ax.set_xlabel(" 时间 $t$ (h)")
722     ax.set_ylabel(r" 最大含水率位置 $r$ (cm)")
723     ax.set_xlim(0, t_rep_h)
724     ax.set_ylim(-0.02, 2.05)
725     fig.tight_layout()
726     fig.savefig(FIGURES_DIR / "q3_max_location.pdf")
727     plt.close(fig)
728
729     fig, ax = plt.subplots(figsize=(6.4, 3.8))
730     r_cm = fvm.r * 100.0
731     hrs = tables["table_hours"]
732     t_end_h = float(hrs[-1])
733     if t_end_h > 72.0:
734         show = np.arange(24.0, t_end_h + 1e-9, 24.0)
735         if abs(show[-1] - t_end_h) > 1e-6:
736             show = np.concatenate([show, [t_end_h]])
737     elif hrs.size > 10:
738         show = np.unique(np.concatenate([hrs[:,2], hrs[-1:]]))
739     else:
740         show = hrs
741     for th in show:
742         t = 3600.0 * float(th)
743         if t < pub["t_star"] - 1e-9:
744             j = int(np.argmin(np.abs(pub["times"] - t)))
745             C = pub["Y"][j, n:]
746         else:
747             C = pub["eval_y"](t)[n:]

```

```

748     ax.plot(r_cm, C, lw=1.2, label=fr"$t={th:g}\,\mathrm{{{h}}}$")
749     ax.set_xlabel(" 半径 $r$ (cm)")
750     ax.set_ylabel(r" 干基含水率 $C$ (kg/kg)")
751     ax.set_xlim(0, 2)
752     ax.legend(frameon=False, ncol=2)
753     fig.tight_layout()
754     fig.savefig(FIGURES_DIR / "q3_C_profiles.pdf")
755     plt.close(fig)
756
757     fig, ax = plt.subplots(figsize=(6.4, 3.6))
758     mask = mon["times"] >= pub["t_star"] - 6 * 3600.0
759     if np.count_nonzero(mask) < 5:
760         mask = np.ones_like(mon["times"], dtype=bool)
761     ax.axhline(C_THRESH, color="0.4", lw=1.0, ls="--", label=r" 阈值 $0.15$")
762     ax.plot(t_h[mask], mon["M"][mask], lw=1.2, label=r"$\max C$")
763     ax.axvline(t_star_h, color="0.2", lw=0.8, ls=":", label=fr"$t_*={t_star_h:.3f}\,\mathrm{{{h}}}$")
764     ax.axvline(t_rep_h, color="0.2", lw=0.8, ls="-.",
765         ↪ label=fr"$t_{\mathrm{{{rep}}}}={t_rep_h:.3f}\,\mathrm{{{h}}}$")
766     ax.set_xlabel(" 时间 $t$ (h)")
767     ax.set_ylabel(r" 干基含水率 $C$ (kg/kg)")
768     ax.legend(frameon=False)
769     fig.tight_layout()
770     fig.savefig(FIGURES_DIR / "q3_event_zoom.pdf")
771     plt.close(fig)
772
773     fig, ax = plt.subplots(figsize=(6.4, 3.6))
774     ax.plot(t_h, [Ta_fun(t) for t in mon["times"]], lw=1.0, ls="--", label=" 烘房 $T_a$")
775     ax.plot(t_h, mon["T_surface"], lw=1.2, label=" 表面")
776     ax.plot(t_h, mon["T_center"], lw=1.2, label=" 中心")
777     ax.set_xlabel(" 时间 $t$ (h)")
778     ax.set_ylabel(" 温度 (°C)")
779     ax.set_xlim(0, t_rep_h)
780     ax.legend(frameon=False)
781     fig.tight_layout()
782     fig.savefig(FIGURES_DIR / "q3_T_center_surface.pdf")
783     plt.close(fig)
784
785     def json_safe(obj):
786         if isinstance(obj, dict):
787             return {str(k): json_safe(v) for k, v in obj.items()}
788         if isinstance(obj, (list, tuple)):
789             return [json_safe(v) for v in obj]
790         if isinstance(obj, np.ndarray):
791             return obj.tolist()
792         if isinstance(obj, (np.floating, np.integer)):
793             return obj.item()
794         if isinstance(obj, (float, int, str, bool)) or obj is None:
795             return obj
796         return str(obj)
797
798
799     def main(argv=None):
800         from q3_closeout import main as closeout_main
801         return closeout_main(argv)
802
803
804     if __name__ == '__main__':
805         main()

```

B.10 code/q3_balance.py

```

1 """Continuous-solution balance integrals for the accepted Q3 effective model."""
2 import numpy as np
3 from numpy.polynomial.legendre import leggauss

```

```

4 from appendix3 import heat_capacity, heat_capacity_deriv, moisture_diffusivity_q2
5 from utils import R, L, H, HM
6
7
8 def audit_segment(fvm, sol, Ta_fun, Ca_fun, t_end=None, orders=(4,8)):
9     if sol.sol is None:
10         raise ValueError("balance audit requires a dense solution")
11     n = fvm.n_nodes
12     start = float(sol.sol.ts[0])
13     end = float(sol.sol.ts[-1]) if t_end is None else float(t_end)
14     if not start < end <= sol.sol.ts[-1]+1e-9:
15         raise ValueError("audit outside dense-solution interval")
16     edges = np.r_[sol.sol.ts[sol.sol.ts < end],end]
17     edges = np.unique(edges)
18     z0, z1 = sol.sol(start), sol.sol(end)
19     factor = 2*np.pi*L
20     def energy(z):
21         return float(factor*np.dot(fvm.W,heat_capacity(z[n:]))*(z[:n]-28.)))
22     def quadrature(order):
23         gx,gw=leggauss(order)
24         total=np.zeros(3)
25         for j in range(0,len(edges)-1,64):
26             aa,bb=edges[j:-1][:64],edges[j+1:][:64]
27             tt=((aa+bb)[:,None]/2+(bb-aa)[:,None]/2*gx).ravel()
28             Y=sol.sol(tt);T,C=Y[:n],Y[n:]
29             Ta,Ca=np.asarray(Ta_fun(tt)),np.asarray(Ca_fun(tt))
30             D=moisture_diffusivity_q2(C,T)
31             Df=2*D[:,-1]*D[1:]/np.maximum(D[:,-1]+D[1:],1e-300)
32             F=-fvm.r_half[:,-1]*Df*np.diff(C,axis=0)/fvm.dr_face[:,-1]
33             flows=np.vstack([np.zeros((1,len(tt))),F,R*HM*(C[-1]-Ca)])
34             Cdot=-np.diff(flows,axis=0)/fvm.W[:,-1]
35             vals=np.vstack([R*HM*(C[-1]-Ca),factor*R*H*(Ta-T[-1]),
36                             factor*np.sum(fvm.W[:,-1]*heat_capacity_deriv(C)*(T-28.)*Cdot,axis=0)])
37             weights=((bb-aa)[:,-1]/2*gw).ravel()
38             total+=vals@weights
39         return total
40     integrals={str(o):quadrature(o).tolist() for o in orders}
41     return {"start_s":start,"end_s":end,"accepted_intervals":len(edges)-1,
42           "M_start":float(fvm.W@z0[n:]),"M_end":float(fvm.W@z1[n:]),
43           "E_start":energy(z0),"E_end":energy(z1),"integrals_by_order":integrals}
44
45
46 def combine(parts):
47     if not parts:
48         raise ValueError('no balance segments')
49     for left, right in zip(parts, parts[1:]):
50         if abs(left['end_s']-right['start_s']) > 1e-9:
51             raise ValueError('balance segments have a time gap or overlap')
52         for field in ('M', 'E'):
53             if not np.isclose(left[field+'_end'], right[field+'_start'], rtol=1e-12, atol=1e-14):
54                 raise ValueError('balance segments have a state discontinuity')
55     I=np.sum([p['integrals_by_order']['8'] for p in parts],axis=0)
56     I4=np.sum([p['integrals_by_order']['4'] for p in parts],axis=0)
57     M0,M1=parts[0]['M_start'],parts[-1]['M_end']
58     E0,E1=parts[0]['E_start'],parts[-1]['E_end']
59     mass_denom=max(abs(M0-M1),abs(I[0]),1e-18)
60     heat_denom=max(abs(E1-E0-I[2]),abs(I[1]),1e-12)
61     return {
62         'moisture':{'M0':M0,'M_end':M1,'surface_integral':float(I[0]),
63                     'relative_residual':float(abs(M1-M0+I[0])/mass_denom)},
64         'heat':{'E_end_minus_E0':E1-E0,'composition_correction':float(I[2]),'Q_surface':float(I[1]),
65                 'relative_residual':float(abs(E1-E0-I[2]-I[1])/heat_denom)},
66         'quadrature_relative_difference':{'moisture':float(abs(I[0]-I4[0])/mass_denom),
67                                           'heat_and_composition':float((abs(I[1]-I4[1])+abs(I[2]-I4[2]))/heat_denom)},
68         'parts':parts,

```

```

69         'interpretation':'Integrated effective equation with B_prime(C)*(T-28)*Cdot correction; not full
        ↪ multiphase energy conservation.'}

```

B.11 code/q3_closeout.py

```

1  """Q3 closeout: matched-time convergence, continuous balances and gated delivery."""
2  from __future__ import annotations
3  import argparse
4  import hashlib
5  import json
6  import math
7  import subprocess
8  import sys
9  import time
10 from pathlib import Path
11 import numpy as np
12 from delivery import make_gate,require_delivery
13 from q3_balance import audit_segment,combine
14 from utils import RESULTS_DIR,REPORTS_DIR,FIGURES_DIR,ROOT
15
16 DIAG=RESULTS_DIR/'diagnostics/q3_closeout'
17 NODE=Path(__import__('shutil').which('node') or 'node')
18 BUILDER=ROOT/'tmp/q3_export/build.mjs'
19 REQUIRED_CHECKS = frozenset({
20     'event_found', 'finite', 'strict_below_all_verified_grids',
21     'strict_below_with_empirical_margin', 'moisture_bounds', 'temperature_bounds',
22     'moisture_balance_1e-6', 'heat_balance_1e-6', 'quadrature_convergence_1e-7',
23     'space_event_60s', 'time_event_60s', 'space_full_fields_2e-5',
24     'time_full_fields_2e-5', 'table_four_decimals', 'q2_overlap',
25 })
26
27
28 def require_q3_delivery(validation, diagnostic_path):
29     require_delivery(validation, diagnostic_path)
30     if validation.get('result_version') != 'q3-closeout-v1' or not
31     ↪ REQUIRED_CHECKS.issubset(validation['delivery_gate']['checks']):
32         raise RuntimeError('Delivery blocked: missing Q3 checks or wrong result version')
33
34 def export_workbook(path,C_out,times,validation):
35     require_q3_delivery(validation,DIAG/'pre_export.json')
36     data={'sheet':' 水分浓度','headers':[' 时间\\到药材中心的距离']+[round(.1*j,1) for j in range(21)],
37         'rows':[[float(t)]+list(map(float,row)) for t,row in
38     ↪ zip(times,C_out)],'path':str(path.resolve())}
39     (DIAG/'excel_payload.json').write_text(json.dumps(data,ensure_ascii=False,allow_nan=False),encoding='u'
40     ↪ tf-8')
41     subprocess.run([str(NODE),str(BUILDER)],cwd=ROOT,check=True)
42
43 def save_case(case):
44     f=case['fvm'];n=f.n_nodes
45     np.savez_compressed(DIAG/(case['tag']+'.npz'),r=f.r,times=case['times'],Y=case['Y'],
46         y_star=case['y_star'],t_star=case['t_star'])
47     info={k:case[k] for k in ('tag','level','n_nodes','rtol','atol_T','atol_C','max_step_early','max_step_
48     ↪ late','elapsed_s','event')}
49     (DIAG/(case['tag']+'.json')).write_text(json.dumps(info,indent=2),encoding='utf-8')
50
51 def compare_fields(a,b):
52     """Shared physical nodes and exactly matching stored times."""
53     times,ia,ib=np.intersect1d(a['times'],b['times'],return_indices=True)
54     ra,rb=a['fvm'].r,b['fvm'].r
55     j=np.searchsorted(rb,ra)
56     if np.max(abs(rb[j]-ra))>1e-13:
57         raise RuntimeError('spatial meshes are not nested')

```

```

57     na,nb=a['n_nodes'],b['n_nodes']
58     # Batch to bound temporary array sizes for fine meshes.
59     maxT=maxC=0.
60     for start in range(0,len(times),128):
61         ai,bi=ia[start:start+128],ib[start:start+128]
62         maxT=max(maxT,float(np.max(abs(a['Y'][ai,:na]-b['Y'][bi,nb][:,j]))))
63         maxC=max(maxC,float(np.max(abs(a['Y'][ai,na:]-b['Y'][bi,nb][:,j]))))
64     return {'matched_times':len(times), 'max_T':maxT, 'max_C':maxC}
65
66
67 def main(argv=None):
68     from problem3 import (load_q3_inputs,run_case,run_probe,RTOL,ATOL_T,ATOL_C,MAX_STEP_EARLY,
69         MAX_STEP_LATE,find_strict_report,build_output_tables,physical_bounds,overlap_q2,
70         write_result3_xlsx,write_table5_csv,plot_figures,setup_mpl,conservation_heat,conservation_moisture)
71     parser=argparse.ArgumentParser();parser.add_argument('--probe-only',action='store_true')
72     parser.add_argument('--no-xlsx',action='store_true',help='Save gated numerical evidence; export
73     ↪ separately later.')
74     args=parser.parse_args(argv)
75     DIAG.mkdir(parents=True,exist_ok=True)
76     start=time.perf_counter()
77     Ta,Ca,knots=load_q3_inputs()
78     if args.probe_only:
79         print(json.dumps(run_probe(Ta,Ca)));return
80     cases={}
81     for level in (4,8,16):
82         cases[level]=run_case(level,Ta,Ca,72*3600.,RTOL,ATOL_T,ATOL_C,MAX_STEP_EARLY,MAX_STEP_LATE,
83             tag=f'G{level}',audit_balances=(level==8))
84         save_case(cases[level])
85     pub,ref=cases[8],cases[16]
86     tight=run_case(8,Ta,Ca,72*3600.,RTOL/5,ATOL_T/5,ATOL_C/5,MAX_STEP_EARLY/2,MAX_STEP_LATE/2,tag='G8t')
87     save_case(tight)
88     space=compare_fields(pub,ref);temporal=compare_fields(pub,tight)
89     coarse_space=compare_fields(cases[4],pub)
90     critical=pub['t_star']
91     same={name:float(case['eval_M'](critical)) for name,case in
92     ↪ [(('G4',cases[4]),('G8',pub)),('G16',ref)),('G8t',tight)]}
93     # The maximum differences are measured at the same physical time.
94     err=max(abs(same['G8']-same['G16']),abs(same['G8']-same['G8t']))
95     margin=2*err+1e-8
96     zero_report=find_strict_report(pub['eval_M'],critical,critical+3500,margin=0.)
97     report=find_strict_report(pub['eval_M'],critical,critical+3500,margin=margin)
98     t_rep=report['t_hour4_s'] # Upward four-decimal-hour rounding retains the safety margin.
99     report_M={name:float(case['eval_M'](t_rep)) for name,case in [(('G8',pub)),('G16',ref)),('G8t',tight)]}
100     tables=build_output_tables(pub,t_rep);other=build_output_tables(ref,t_rep)
101     tight_tables=build_output_tables(tight,t_rep)
102     table_changed=int(np.count_nonzero(np.round(tables['C_table'],4)!=np.round(other['C_table'],4)))
103     table_time_changed=int(np.count_nonzero(np.round(tables['C_table'],4)!=np.round(tight_tables['C_table']
104     ↪ ),4)))
105     output_space=float(np.max(abs(tables['C_excel']-other['C_excel'])))
106     tail=audit_segment(pub['fvm'],pub['sol_after'],Ta,Ca,t_end=t_rep)
107     balances=combine(pub['balance_parts']+tail)
108     checked_states = np.vstack([pub['Y'], pub['y_star'], pub['eval_y'](t_rep)])
109     bounds=physical_bounds(dict(pub,Y=checked_states),Ta,Ca);overlap=overlap_q2(pub)
110     overlap_ok=overlap.get('available',False) and all(c.get('matched',False) and c['T_max_abs']<2e-5 and
111     ↪ c['C_max_abs']<2e-5 for c in overlap['checks'].values())
112     event_space=abs(pub['t_star']-ref['t_star']);event_time=abs(pub['t_star']-tight['t_star'])
113     gates={
114         'event_found':abs(pub['event']['g_star'])<1e-8,
115         'finite':bool(np.isfinite(checked_states).all() and np.isfinite(tables['C_excel']).all()
116             and np.isfinite(tables['T_excel']).all()),
117         'strict_below_all_verified_grids':all(v<.15 for v in report_M.values()),
118         'strict_below_with_empirical_margin':report_M['G8']+margin<.15,
119         'moisture_bounds':bounds['C_bounds_ok'],'temperature_bounds':bounds['T_bounds_ok'],
120         'moisture_balance_1e-6':balances['moisture']['relative_residual']<1e-6,
121         'heat_balance_1e-6':balances['heat']['relative_residual']<1e-6,

```

```

118     'quadrature_convergence_1e-7':max(balances['quadrature_relative_difference'].values())<1e-7,
119     'space_event_60s':event_space<=60., 'time_event_60s':event_time<=60.,
120     'space_full_fields_2e-5':max(space['max_T'],space['max_C'],output_space)<2e-5,
121     'time_full_fields_2e-5':max(temporal['max_T'],temporal['max_C'])<2e-5,
122     'table_four_decimals':table_changed==0 and table_time_changed==0,
123     'q2_overlap':overlap_ok,
124 }
125 v={'result_version':'q3-closeout-v1','model':'q3-model-v1','result_status':'PENDING',
126   'published_mesh':'G8','published_n_nodes':pub['n_nodes'],'rtol':RTOL,'atol_T':ATOL_T,'atol_C':ATOL_
    ↪ C,
127   'max_step_early':MAX_STEP_EARLY,'max_step_late':MAX_STEP_LATE,
128   't_star_s':critical,'t_star_h':critical/3600., 't_rep_s':t_rep,'t_rep_h':t_rep/3600.,
129   'M_star':pub['event']['M_star'],'M_rep':report_M['G8'],'report_maxima':report_M,
130   'matched_critical_time_maxima':same,'moisture_margin':margin,'margin_type':'empirical, not a
    ↪ rigorous PDE error bound',
131   'report_zero_margin':zero_report,'report_with_margin':report,
132   'space_fields':space,'time_fields':temporal,'coarse_space_fields':coarse_space,'output_space_max_C'
    ↪ :output_space,
133   'dt_space_s':event_space,'dt_time_s':event_time,
134   'conservation_C':balances['moisture'],'conservation_T':balances['heat'],'continuous_balances':balan
    ↪ ces,
135   'old_sampled_conservation_C':conservation_moisture(pub['fvm'],pub['times'],pub['Y'],Ca),
136   'old_sampled_conservation_T':conservation_heat(pub['fvm'],pub['times'],pub['Y'],Ta),
137   'physical_bounds':bounds,'q2_overlap':overlap,'table_four_decimal_changes':table_changed,
138   'events':{f'G{k}':c['event'] for k,c in cases.items()}|{'G8t':tight['event']}},
139   'delivery_gate':make_gate(gates),'table_hours':tables['table_hours'].tolist(),
140   'excel_nrows':len(tables['excel_times']), 'paper_table_C':tables['paper_C'],
141   'max_location_cm':pub['event']['rmax_star_cm'],'center_is_max_at_event':pub['event']['center_is_max
    ↪ '],
142   'n_radial_increases':pub['monitor']['n_radial_increases'],
143   'elapsed_s':time.perf_counter()-start,'python':sys.version,
144   'source_sha256':{p:hashlib.sha256((ROOT/p).read_bytes()).hexdigest() for p in
145     ['code/problem3.py','code/q3_balance.py','code/q3_closeout.py','code/delivery.py',
146     'tmp/q3_export/build.mjs','tmp/q3_export/gate.mjs',
147     'code/radial_coupled.py','code/appendix3.py','data/附件/附件 1.xlsx']}}
148 print('CLOSEOUT',json.dumps({k:v[k] for k in ('t_star_h','t_rep_h','M_rep','dt_space_s','dt_time_s','c
    ↪ nservation_C','conservation_T','delivery_gate')},ensure_ascii=False),flush=True)
149 publish_results(pub, tables, v, Ta, no_xlsx=args.no_xlsx)
150 print('Q3 CLOSEOUT NUMERICAL PASS',flush=True)
151
152
153 def publish_results(pub, tables, v, Ta, *, no_xlsx=False):
154     from problem3 import write_result3_xlsx, write_table5_csv, plot_figures, setup_mpl
155     require_q3_delivery(v,DIAG/'latest.json')
156     t_rep, critical = v['t_rep_s'], v['t_star_s']
157     # Commit production outputs only after every required numerical check passed.
158     yend=pub['eval_y'](t_rep)
159     np.savez_compressed(RESULTS_DIR/'q3_solution.npz',mesh_level=8,n_nodes=pub['n_nodes'],r=pub['fvm'].r,
160       times=pub['times'],Y=pub['Y'],excel_times=tables['excel_times'],T_out=tables['T_excel'],C_out=tabl
    ↪ es['C_excel'],
161     table_hours=tables['table_hours'],C_table=tables['C_table'],t_star=critical,t_rep=t_rep,
162     y_star=pub['y_star'],y_end=yend,M_hist=pub['monitor']['M'],rmax_m=pub['monitor']['rmax_m'],
163     C_center=pub['monitor']['C_center'],C_surface=pub['monitor']['C_surface'],
164     T_center=pub['monitor']['T_center'],T_surface=pub['monitor']['T_surface'],
165     rtol=v['rtol'],atol_T=v['atol_T'],atol_C=v['atol_C'],max_step_early=v['max_step_early'],max_step_l
    ↪ ate=v['max_step_late'],
166     interpolation=np.array('pchip_then_constant'))
167     (RESULTS_DIR/'q3_validation.json').write_text(json.dumps(v,ensure_ascii=False,indent=2),encoding='utf-
    ↪ 8')
168     write_table5_csv(RESULTS_DIR/'q3_table_moisture.csv',tables['table_hours'],tables['C_table'])
169     if not no_xlsx:
170         write_result3_xlsx(RESULTS_DIR/'result3.xlsx',tables['C_excel'],tables['excel_times'],validation=v)
171     plot_figures(setup_mpl(),pub,tables,t_rep,Ta)
172

```

```

173
174 if __name__ == '__main__':
175     main()

```

B.12 code/problem4.py

```

1  """Q4: prescribed radial shrinkage, material-coordinate FVM and BDF.
2
3  Run a bounded, source-preserving baseline with --level 2/4/8 [--tight].
4  No radius extrapolation; output here is internal numerical evidence, not Excel.
5  """
6  from __future__ import annotations
7
8  import argparse
9  import hashlib
10 import json
11 import platform
12 import time
13 from pathlib import Path
14
15 import numpy as np
16 import scipy
17 from numpy.polynomial.legendre import leggauss
18 from openpyxl import load_workbook
19 from scipy.integrate import solve_ivp
20 from scipy.interpolate import PchipInterpolator
21 from scipy.sparse import lil_matrix
22
23 from radial_fvm import graded_radial_nodes, harmonic_mean
24 from utils import ATTACH_DIR, RESULTS_DIR, C0, T0, H, HM, R, load_oven_table
25
26 CAP = 259200.0
27 SWITCH = 14400.0
28 THRESHOLD = 0.15
29 OUT = RESULTS_DIR / "q4_baseline"
30
31
32 def properties(C, T):
33     C, T = np.asarray(C), np.asarray(T)
34     # Only protect nonphysical implicit-iteration probes; stored states are audited.
35     Cp = np.maximum(C, 1e-12)
36     rho = 760.0 + 90.0 * Cp
37     cp = 1850.0 + 2150.0 * Cp / (1.0 + Cp)
38     k = 0.12 + 0.20 * Cp / (1.0 + Cp)
39     D = 4.2e-4 * np.exp(-0.30 / Cp) * np.exp(-3850.0 / (T + 273.15))
40     return rho * cp, k, D
41
42
43 class Inputs:
44     def __init__(self):
45         t, Ta, Ca = load_oven_table()
46         if t.size != 241 or t[0] != 0 or t[-1] != SWITCH:
47             raise ValueError("unexpected oven input coverage")
48         self.Ta = PchipInterpolator(t, Ta, extrapolate=False)
49         self.Ca = PchipInterpolator(t, Ca, extrapolate=False)
50         wb = load_workbook(ATTACH_DIR / "附件 2.xlsx", read_only=True, data_only=True)
51         radius_rows = list(wb.active.values)
52         wb.close()
53         a = np.asarray(radius_rows[1:], dtype=float)
54         self.rt, self.rv = a[:, 0], a[:, 1] / 100.0
55         if (a.shape != (145, 2) or not np.all(np.isfinite(a))
56             or np.any(np.diff(self.rt) <= 0) or np.any(np.diff(self.rv) > 0)
57             or self.rt[0] != 0 or self.rt[-1] != CAP or self.rv[0] != R):
58             raise ValueError("unexpected radius input")
59         self.radius = PchipInterpolator(self.rt, self.rv, extrapolate=False)

```

```

60     self.audit = {"radius_records": len(a), "radius_range_m": [float(self.rv[0]), float(self.rv[-1])],
61                  "radius_cap_s": CAP, "oven_records": len(t), "radius_input_headers":
        ↳ list(radius_rows[0]),
62                  "radius_node_error_m": float(np.max(abs(self.radius(self.rt) - self.rv))),
63                  "oven_node_error": float(max(np.max(abs(self.Ta(t)-Ta)),
        ↳ np.max(abs(self.Ca(t)-Ca))))}
64     self.audit["source_sha256"] = {
65         name: hashlib.sha256((ATTACH_DIR / name).read_bytes()).hexdigest()
66         for name in (" 附件 1.xlsx", " 附件 2.xlsx")}
67
68     def environment(self, t, late=False):
69         if late or t > SWITCH:
70             return 50.0, 0.05
71         return float(self.Ta(t)), float(self.Ca(t))
72
73
74 class ShrinkingFVM:
75     def __init__(self, xi):
76         self.xi = np.asarray(xi, dtype=float)
77         if self.xi[0] != 0 or self.xi[-1] != 1 or np.any(np.diff(self.xi) <= 0):
78             raise ValueError("invalid material mesh")
79         self.n = len(xi)
80         self.faces = (self.xi[:-1] + self.xi[1:]) / 2
81         edges = np.r_[0., self.faces, 1.]
82         self.w = np.diff(edges**2) / 2
83         self.dx = np.diff(self.xi)
84
85     def sparsity(self):
86         J = lil_matrix((2*self.n, 2*self.n))
87         for i in range(self.n):
88             for j in range(max(0, i-1), min(self.n, i+2)):
89                 J[i, j] = J[i, self.n+j] = J[self.n+i, j] = J[self.n+i, self.n+j] = 1
90         return J.tocsc()
91
92     def rhs(self, y, radius, Ta, Ca, hm=HM, ht=H):
93         T, C = y[:self.n], y[self.n:]
94         B, k, D = properties(C, T)
95         FT = np.r_[0., -self.faces*harmonic_mean(k[:-1], k[1:])*np.diff(T)/self.dx,
96                  radius*ht*(T[-1]-Ta)]
97         FC = np.r_[0., -self.faces*harmonic_mean(D[:-1], D[1:])*np.diff(C)/self.dx,
98                  radius*hm*(C[-1]-Ca)]
99         Tdot = -np.diff(FT) / (radius**2 * self.w * B)
100         Cdot = -np.diff(FC) / (radius**2 * self.w)
101         return np.r_[Tdot, Cdot]
102
103
104 def model_checks(inputs):
105     fvm = ShrinkingFVM(np.linspace(0., 1., 41))
106     x = fvm.xi
107     y = np.r_[35.+10*x*x, 1.7-1.2*x*x]
108     rr = float(inputs.radius(21600.))
109     dy = fvm.rhs(y, rr, 50., 0.05)
110     mass_error = abs(2*np.dot(fvm.w, dy[fvm.n:])) + 2*HM/rr*(y[-1]-0.05)
111     B, k, D = properties(y[fvm.n:], y[fvm.n:])
112     heat_error = abs(np.dot(fvm.w*B, dy[:fvm.n])) + H/rr*(y[fvm.n-1]-50.)
113     # Independent dimensional-volume assembly on the same physical annuli.
114     rp = rr*x
115     rf = (rp[:-1]+rp[1:])/2
116     volumes = np.diff(np.r_[0., rf, rr]**2)/2
117     flows_T = np.r_[0., -rf*harmonic_mean(k[:-1], k[1:])*np.diff(y[:fvm.n])/np.diff(rp),
        ↳ rr*H*(y[fvm.n-1]-50.)]
118     flows_C = np.r_[0., -rf*harmonic_mean(D[:-1], D[1:])*np.diff(y[fvm.n:])/np.diff(rp),
        ↳ rr*HM*(y[-1]-0.05)]
119     reference = np.r_[ -np.diff(flows_T)/(volumes*B), -np.diff(flows_C)/volumes]
120     fixed_error = float(np.max(abs(dy-reference)))

```

```

121 uniform = np.r_[np.full(fvm.n, T0), np.full(fvm.n, C0)]
122 pure = solve_ivp(lambda t,z: fvm.rhs(z, float(inputs.radius(t)), T0, C0, hm=0., ht=0.),
123                 (0., CAP), uniform, method="BDF", jac_sparsity=fvm.sparsity(),
124                 rtol=1e-9, atol=1e-11)
125 pure_error = float(np.max(abs(pure.y-uniform[:,None])))
126 checks = {"instant_mass_abs": float(mass_error), "instant_heat_abs": float(heat_error),
127          "fixed_physical_operator_abs": fixed_error, "pure_shrinkage_uniform_abs": pure_error,
128          "pure_shrinkage_success": bool(pure.success)}
129 checks["pass"] = bool(mass_error < 1e-12 and heat_error < 1e-6 and fixed_error < 1e-10
130                     and pure_error < 1e-10 and pure.success)
131 return checks
132
133
134 def run(level=2, tight=False):
135     OUT.mkdir(parents=True, exist_ok=True)
136     inputs = Inputs()
137     basic = model_checks(inputs)
138     if not basic["pass"]:
139         raise RuntimeError(f"model checks failed: {basic}")
140     fvm = ShrinkingFVM(graded_radial_nodes(level)/R)
141     n = fvm.n
142     rtol = 1e-9 if tight else 1e-8
143     atol = np.r_[np.full(n, 1e-9 if tight else 1e-8), np.full(n, 1e-11 if tight else 1e-10)]
144     label = f"G{level}" + ("_tight" if tight else "")
145     y0 = np.r_[np.full(n, T0), np.full(n, C0)]
146     times, states = [np.array([0.])], [y0[None,:]]
147     segments, events, event_states = [], [], []
148     mass_loss = 0.
149     max_mass_error = 0.
150     bounds = [float("inf"), -float("inf"), float("inf"), -float("inf")]
151     radial_increase = 0.
152     heat_residual_max = 0.
153     wall = time.perf_counter()
154     event = lambda t,y: float(np.max(y[n:])-THRESHOLD)
155     event.direction = -1.
156     event.terminal = False
157     breaks = np.unique(np.r_[0., SWITCH, np.arange(21600., CAP+1., 21600.)])
158     gx, gw = leggauss(4)
159     for a,b in zip(breaks[:-1], breaks[1:]):
160         late = a >= SWITCH
161         def rhs(t,y):
162             Ta,Ca = inputs.environment(t, late=late)
163             return fvm.rhs(y, float(inputs.radius(t)), Ta, Ca)
164         max_step = (5. if tight else 10.) if not late else (30. if tight else 60.)
165         sol = solve_ivp(rhs, (a,b), y0, method="BDF", rtol=rtol, atol=atol,
166                       jac_sparsity=fvm.sparsity(), max_step=max_step,
167                       dense_output=True, events=event)
168         if not sol.success:
169             raise RuntimeError(sol.message)
170         ts = np.arange(a+60., b+0.1, 60.)
171         ys = sol.sol(ts).T
172         times.append(ts)
173         states.append(ys)
174         if len(sol.t_events[0]):
175             events.extend(sol.t_events[0].tolist())
176             event_states.extend(sol.y_events[0].tolist())
177         # Quadrature on accepted BDF step intervals; 4-point Gaussian rule.
178         integral = 0.
179         for start in range(0, len(sol.t)-1, 128):
180             aa, bb = sol.t[start:-1][:128], sol.t[start+1:][:128]
181             qq = ((aa+bb)[:,:None]/2 + (bb-aa)[:,:None]/2*gx).ravel()
182             surface = sol.sol(qq)[-1]
183             ca = np.full_like(qq, 0.05) if late else inputs.Ca(qq)
184             loss = 2*HM/inputs.radius(qq)*(surface-ca)
185             integral += float(np.sum(loss.reshape(-1,4)*gw*(bb-aa)[:,:None]/2))

```

```

186     mass_loss += integral
187     y0 = sol.y[:,-1]
188     mass_error = abs(float(2*np.dot(fvm.w,y0[n:])-C0+mass_loss))
189     max_mass_error = max(max_mass_error, mass_error)
190     TT, CC = sol.y[:n], sol.y[n:]
191     bounds = [min(bounds[0],float(TT.min())), max(bounds[1],float(TT.max())),
192               min(bounds[2],float(CC.min())), max(bounds[3],float(CC.max()))]
193     radial_increase = max(radial_increase,float(np.max(np.diff(CC,axis=0))))
194     # Audit sampled heat-equation weighted residual, not d(rho*cp*T)/dt.
195     for tcheck in (a,(a+b)/2,b):
196         z = sol.sol(tcheck)
197         dz = rhs(tcheck,z)
198         B,_,_ = properties(z[n:],z[:n])
199         Ta,_ = inputs.environment(tcheck,late=late)
200         heat_error = abs(float(np.dot(fvm.w*dz[:n])+H/float(inputs.radius(tcheck))*(z[n-1]-Ta)))
201         heat_residual_max = max(heat_residual_max,heat_error)
202         segments.append({"start_s":float(a),"end_s":float(b),"steps":len(sol.t)-1,
203                          "nfev":sol.nfev,"nlu":sol.nlu,"mass_abs_error":mass_error})
204     print(f"{label}: {b/3600:g}h maxC={y0[n:].max():.8f} mass_err={mass_error:.2e}
205     ↪ wall={time.perf_counter()-wall:.1f}s",flush=True)
206     tt, yy = np.concatenate(times), np.vstack(states)
207     meta = {"label":label,"level":level,"tight":tight,"nodes":n,"rtol":rtol,
208            "atol_T":float(atol[0]),"atol_C":float(atol[n]),"early_max_step_s":5. if tight else 10.,
209            "late_max_step_s":30. if tight else 60.,"event_s":events[0] if events else None,
210            "event_h":events[0]/3600 if events else None,"end_maxC":float(yy[-1,n:].max()),
211            "max_mass_abs_error":max_mass_error,"mass_relative_error":max_mass_error/C0,
212            "heat_equation_residual_max":heat_residual_max,"bounds_Tmin_Tmax_Cmin_Cmax":bounds,
213            "max_outward_C_increase":radial_increase,"segments":segments,"input_audit":inputs.audit,
214            "model_checks":basic,"wall_s":time.perf_counter()-wall,
215            "python":platform.python_version(),"numpy":np.__version__,"scipy":scipy.__version__,
216            "model":"q4-model-v1","radius_extrapolation":False,
217            "source_code_sha256":hashlib.sha256(Path(__file__).read_bytes()).hexdigest()}
218     np.savez_compressed(OUT/f"{label}.npz",times_s=tt,Y=yy,xi=fvm.xi,w=fvm.w,
219                        radius_m=inputs.radius(tt),events_s=np.array(events),event_states=np.array(event_s)
220                        ↪ tates))
221     (OUT/f"{label}.json").write_text(json.dumps(meta,ensure_ascii=False,indent=2),encoding="utf-8")
222     print(json.dumps({k:meta[k] for k in
223                       ↪ ("label","event_h","end_maxC","mass_relative_error","wall_s")}),flush=True)
224     return meta
225
226
227 if __name__ == "__main__":
228     parser = argparse.ArgumentParser()
229     parser.add_argument("--level",type=int,default=2)
230     parser.add_argument("--tight",action="store_true")
231     args = parser.parse_args()
232     run(args.level,args.tight)

```

B.13 code/verify_q4_results.py

```

1  """Validate Q4 at matched times; produce source tables and figures after gates."""
2  from __future__ import annotations
3  import csv
4  import json
5  import math
6  from pathlib import Path
7  import numpy as np
8  from scipy.integrate import solve_ivp
9  from scipy.interpolate import PchipInterpolator
10 from problem4 import OUT, Inputs, ShrinkingFVM, THRESHOLD, SWITCH
11 from problem2 import setup_mpl
12 from utils import RESULTS_DIR, FIGURES_DIR, REPORTS_DIR
13
14

```

```

15 def load(label):
16     return np.load(OUT/f"{label}.npz"), json.loads((OUT/f"{label}.json").read_text(encoding="utf-8"))
17
18
19 def state_at(data, meta, t, inputs):
20     """Match a new time via short BDF restart from a saved full state, not nearest-time comparison."""
21     times = data["times_s"]
22     i = int(np.searchsorted(times,t,side="right")-1)
23     if abs(times[i]-t) < 1e-8:
24         return data["Y"][i].copy()
25     f = ShrinkingFVM(data["xi"])
26     n = f.n
27     assert times[i] >= SWITCH # New event/report times are in the late constant environment.
28     sol = solve_ivp(lambda tt,y:f.rhs(y,float(inputs.radius(tt)),50.,0.05),
29                     (times[i],t),data["Y"][i],method="BDF",jac_sparsity=f.sparsity(),
30                     rtol=min(meta["rtol"],1e-10),atol=np.r_[np.full(n,1e-10),np.full(n,1e-12)],
31                     max_step=1.)
32     if not sol.success:
33         raise RuntimeError(sol.message)
34     return sol.y[:,-1]
35
36
37 def physical_values(xi, Y, times, inputs, spacing=.001):
38     n = len(xi)
39     fixed = np.arange(21)*.001 if spacing == .001 else np.arange(5)*.005
40     radii = inputs.radius(times)
41     out = np.full((len(times),len(fixed)+1),np.nan)
42     for i,(row,rr) in enumerate(zip(Y,radii)):
43         good = fixed <= rr+1e-14
44         out[i,:len(fixed)][good] =
45             ↳ PchipInterpolator(xi,row[n:],extrapolate=False)(np.minimum(fixed[good]/rr,1.))
46     out[i,-1] = row[-1]
47     return out, radii, fixed
48
49 def main():
50     inputs = Inputs()
51     labels = ["G2","G4","G8","G4_tight","G16","G8_tight"]
52     datasets = {k:load(k) for k in labels}
53     production, reference, tightened = "G8", "G16", "G8_tight"
54     a,ma = datasets[production]
55     b,mb = datasets[reference]
56     c,mc = datasets[tightened]
57     n,m = len(a["xi"]),len(b["xi"])
58     if not np.array_equal(a["times_s"],b["times_s"]) or not
59         ↳ np.allclose(a["xi"],b["xi"][:2],atol=1e-14,rtol=0):
60         raise RuntimeError("grids or sampling times are not nested")
61     spatial_T = float(np.max(abs(a["Y"][:,n]-b["Y"][:,m:2])))
62     spatial_C = float(np.max(abs(a["Y"][:,n]-b["Y"][:,m:2])))
63     temporal_T = float(np.max(abs(a["Y"][:,n]-c["Y"][:,n])))
64     temporal_C = float(np.max(abs(a["Y"][:,n]-c["Y"][:,n])))
65     # Production was refined after one table cell straddled a rounding boundary.
66     critical = ma["event_s"]
67     t_rep = math.ceil((max(ma["event_s"],mb["event_s"],mc["event_s"])+2.)/.36)*.36
68     checked = (production,reference,tightened)
69     at_critical = {k:state_at(d,md,critical,inputs) for k,(d,md) in datasets.items() if k in checked}
70     at_report = {k:state_at(d,md,t_rep,inputs) for k,(d,md) in datasets.items() if k in checked}
71     maxima = {k:float(z[len(datasets[k][0]["xi"]):].max()) for k,z in at_report.items()}
72     critical_max = {k:float(z[len(datasets[k][0]["xi"]):].max()) for k,z in at_critical.items()}
73     event_C_error = max(abs(critical_max[production]-critical_max[reference]),
74                         abs(critical_max[production]-critical_max[tightened]))
75     empirical_margin = 2*event_C_error + 1e-8
76     idx = a["times_s"] < t_rep
77     tt = np.r_[a["times_s"][idx],t_rep]
78     YY = np.vstack([a["Y"][idx],at_report[production]))

```

```

78 C_phys,radii,fixed = physical_values(a["xi"],YY,tt,inputs)
79 C8,_ = physical_values(b["xi"],np.vstack([b["Y"][idx],at_report[reference]]),tt,inputs)
80 physical_C_error = float(np.nanmax(abs(C_phys-C8)))
81 table_times = np.r_[np.arange(21600.,t_rep,21600.),t_rep]
82 table_ids = [int(np.argmin(abs(tt-t))) for t in table_times]
83 table_C,table_R,table_fixed = physical_values(a["xi"],YY[table_ids],table_times,inputs,spacing=.005)
84 # Dedicated table comparison includes the actual moving surface and ignores paired domain blanks.
85 table8,_ = physical_values(b["xi"],np.vstack([b["Y"][idx],at_report[reference]])[table_ids],table_ti
↪ mes,inputs,spacing=.005)
86 finite_table = np.isfinite(table_C)
87 table_changed =
↪ int(np.count_nonzero(np.round(table_C[finite_table],4)!=np.round(table8[finite_table],4)))
88 bounds_pass = all(md["bounds_Tmin_Tmax_Cmin_Cmax"][2]>=-1e-10 and
↪ md["bounds_Tmin_Tmax_Cmin_Cmax"][3]<2.550000001 for _,md in datasets.values())
89 gates = {
90     "input_and_model_tests":all(md["model_checks"]["pass"] for _,md in datasets.values()),
91     "spatial_fields_2e-5":max(spatial_T,spatial_C,physical_C_error)<2e-5,
92     "temporal_fields_2e-5":max(temporal_T,temporal_C)<2e-5,
93     "spatial_event_1s":abs(ma["event_s"]-mb["event_s"])<1.,
94     "temporal_event_1s":abs(ma["event_s"]-mc["event_s"])<1.,
95     "mass_relative_1e-6":all(md["mass_relative_error"]<1e-6 for _,md in datasets.values()),
96     "heat_equation_residual_1e-6":all(md["heat_equation_residual_max"]<1e-6 for _,md in
↪ datasets.values()),
97     "physical_bounds":bounds_pass,
98     "strict_report_all_checked_grids":all(v<THRESHOLD for v in maxima.values()),
99     "strict_report_empirical_margin":maxima[production]+empirical_margin<THRESHOLD,
100     "event_within_observed_radius":t_rep<=259200.,
101     "table_four_decimals":table_changed==0,
102 }
103 summary = {"version":"q4-baseline-v1","model":"q4-model-v1","production":production,
↪ "result_status":"PENDING",
104     "critical_s":critical,"critical_h":critical/3600.,"report_s":t_rep,"report_h":t_rep/3600.,
105     "report_radius_cm":float(inputs.radius(t_rep)*100),"report_maxima":maxima,"same_time_critic
↪ al_maxima":critical_max,
106     "empirical_C_margin":empirical_margin,"event_spatial_difference_s":abs(ma["event_s"]-mb["ev
↪ ent_s"])),
107     "event_time_difference_s":abs(ma["event_s"]-mc["event_s"])),
108     "max_spatial_T_C":[spatial_T,spatial_C],"max_temporal_T_C":[temporal_T,temporal_C],
109     "max_output_physical_C_difference":physical_C_error,"table_changed_four_decimal_cells":tabl
↪ e_changed,
110     "checks":gates,"pass":all(gates.values()),"runs":{k:md for k,(_,md) in datasets.items()},
111     "heat_validation_scope":"weighted equation residual, not full multiphase energy
↪ conservation",
112     "strict_margin_scope":"empirical grid/time differences, not a rigorous PDE error bound"}
113 (RESULTS_DIR/"q4_validation.json").write_text(json.dumps(summary,ensure_ascii=False,indent=2),encoding=
↪ "utf-8")
114 if not summary["pass"]:
115     raise RuntimeError(f"Q4 output blocked: {gates}")
116 np.savez_compressed(RESULTS_DIR/"q4_solution.npz",times_s=tt,xi=a["xi"],Y=YY,
117     radius_m=radii,physical_radii_m=fixed,C_physical=C_phys,
118     table_times_s=table_times,table_C=table_C,critical_state=at_critical[production],r
↪ eport_state=at_report[production])
119 def safe(v):
120     return None if not np.isfinite(v) else float(v)
121 rows = [[float(t)]+[safe(v) for v in row] for t,row in zip(tt[1:],C_phys[1:])]
122 payload = {"sheet":"Sheet1","headers":[" 时间/s\\距离/cm"]+[float(r*100) for r in fixed]+[" 药材表面"],
123     "rows":rows,"report_h":t_rep/3600.}
124 (OUT/"excel_payload.json").write_text(json.dumps(payload,ensure_ascii=False,allow_nan=False),encoding=
↪ "utf-8")
125 with (RESULTS_DIR/"q4_table6.csv").open("w",newline="",encoding="utf-8-sig") as f:
126     writer=csv.writer(f);writer.writerow(["时间/h"]+[f"{r*100:g} cm" for r in
↪ table_fixed]+["药材表面","表面半径/cm"])
127     for t,row,rr in zip(table_times,table_C,table_R):

```

```

128         writer.writerow([f"{t/3600:.4f}"]+[f"" if np.isnan(v) else f"{v:.4f}" for v in
        ↪ row]+[f"{rr*100:.6f}"])
129 with (RESULTS_DIR/"q4_radius_output.csv").open("w",newline="",encoding="utf-8-sig") as f:
130     writer=csv.writer(f);writer.writerow([" 时间/s"," 半径/cm"]);writer.writerows(zip(tt,radii*100))
131 plt=setup_mpl();FIGURES_DIR.mkdir(exist_ok=True)
132 fig,ax=plt.subplots(figsize=(6.2,3.8))
133 ax.plot(tt/3600,YY[:,n],label=" 中心 (全域最大值) ")
134 ax.plot(tt/3600,YY[:, -1],label=" 表面")
135 ax.axhline(.15,color="0.3",linestyle="--",label=" 达标阈值")
136 ax.axvline(critical/3600,color="0.6",linestyle=":")
137 ax.set(xlabel=" 时间 / h",ylabel=" 干基含水率 / (kg/kg)",xlim=(0,t_rep/3600))
138 ax.legend(frameon=False);fig.tight_layout()
139 fig.savefig(FIGURES_DIR/"q4_C_history.pdf");fig.savefig(FIGURES_DIR/"q4_C_history.png",dpi=160);plt.cla
    ↪ ose(fig)
140 fig,ax=plt.subplots(figsize=(6.2,3.8))
141 for t,i,rr in zip(table_times,table_ids,table_R):
142     ax.plot(a["xi"]*rr*100,YY[i,n:],label=f"{t/3600:.2f} h")
143 ax.set(xlabel=" 到药材中心的距离 / cm",ylabel=" 干基含水率 / (kg/kg)")
144 ax.legend(frameon=False,ncol=3,fontsize=7);fig.tight_layout()
145 fig.savefig(FIGURES_DIR/"q4_C_profiles.pdf");fig.savefig(FIGURES_DIR/"q4_C_profiles.png",dpi=160);plt.cla
    ↪ ose(fig)
146 # Small visible table companion; this is a result record, not paper prose.
147 headers=[" 时间/h","0 cm","0.5 cm","1.0 cm","1.5 cm","2.0 cm"," 表面"," 表面半径/cm"]
148 lines=["# 第四问表6 (基线结果, 待审) ", "", " | "+" | ".join(headers)+" | ",
    ↪ " | "+" | ".join(["---"*len(headers)+" | "]
149 for t,row,rr in zip(table_times,table_C,table_R):
150     lines.append(" | "+" | ".join([f"{t/3600:.4f}"]+[f"—" if np.isnan(v) else f"{v:.4f}" for v in
    ↪ row]+[f"{rr*100:.6f}"])+ " | ")
151 lines += ["", "—表示该固定半径在当前药材区域之外。表面列对应各时刻真实外半径。末行以未舍入值通过严格阈值检查;
    ↪ 四位显示可能仍为0.1500。"]
152 (REPORTS_DIR/"Q4_TABLE6.md").write_text("\n".join(lines)+"\n",encoding="utf-8")
153 print(json.dumps({k:summary[k] for k in ("critical_h","report_h","report_radius_cm","report_maxima","e
    ↪ vent_spatial_difference_s","pass")),ensure_ascii=False),flush=True)
154
155
156 if __name__=="__main__":
157     main()

```

B.14 code/plot_q3_results.py

```

1 """Redraw Q3 profiles and the event window using validated saved states only."""
2 import hashlib
3 import json
4 from pathlib import Path
5
6 import numpy as np
7
8 from problem3 import setup_mpl
9 from q3_closeout import require_q3_delivery
10 from utils import ROOT, RESULTS_DIR, FIGURES_DIR
11
12
13 def main():
14     out = RESULTS_DIR / 'diagnostics/q3_delivery'
15     out.mkdir(parents=True, exist_ok=True)
16     validation = json.loads((RESULTS_DIR / 'q3_validation.json').read_text(encoding='utf-8'))
17     require_q3_delivery(validation, out / 'pre_plot.json')
18     z = np.load(RESULTS_DIR / 'q3_solution.npz')
19     n = int(z['n_nodes'])
20     plt = setup_mpl()
21     fig, ax = plt.subplots(figsize=(6.4, 4.8))
22     for hour in z['table_hours']:
23         if abs(hour * 3600 - float(z['t_rep'])) < 1e-7:
24             state = z['y_end']

```

```

25     else:
26         indices = np.flatnonzero(abs(z['times'] - hour * 3600) < 1e-7)
27         if len(indices) != 1:
28             raise RuntimeError('profile time missing from saved states')
29         state = z['V'][indices[0]]
30         ax.plot(z['r'] * 100, state[n:], lw=1.2, label=fr'$t={hour:g}\,\mathrm{{{h}}}$')
31     ax.set_xlabel('\u534a\u5f84 $r$ (cm)')
32     ax.set_ylabel('\u5e72\u57fa\u542b\u6c34\u7387 $C$ (kg/kg)')
33     ax.set_xlim(0, 2)
34     ax.legend(frameon=False, ncol=3, loc='lower center', bbox_to_anchor=(.5, 1.01))
35     fig.tight_layout()
36     fig.savefig(FIGURES_DIR / 'q3_C_profiles.pdf')
37     plt.close(fig)
38
39     critical, report = float(z['t_star']), float(z['t_rep'])
40     mask = z['times'] >= critical - 90
41     times = np.r_[z['M_hist'][mask], critical, report]
42     maxima = np.r_[z['M_hist'][mask], np.max(z['y_star'][n:]), np.max(z['y_end'][n:])]
43     fig, ax = plt.subplots(figsize=(6.4, 3.8))
44     ax.axhline(0, color='.4', lw=1, ls='--', label='$C=0.15$')
45     ax.plot(times - critical, (maxima - .15) * 1e6, marker='o', ms=3, lw=1.2,
46             label=r'$\max_i C_i-0.15$')
47     ax.axvline(0, color='.2', lw=.9, ls=':', label=fr'$t_*={critical/3600:.6f}\,\mathrm{{{h}}}$')
48     ax.axvline(report - critical, color='#c44242', lw=.9, ls='-.',
49             label=fr'$t_{\{\mathrm{{rep}}\}}={report/3600:.6f}\,\mathrm{{{h}}}$')
50     ax.set_xlabel('\u76f8\u5bf9\u4e34\u754c\u65f6\u523b $t-t_*$ (s)')
51     ax.set_ylabel(r'$\{\max_i C_i-0.15\}\, /\, 10^{-6}$ (kg/kg)')
52     ax.set_xlim(-90, 7)
53     ax.legend(frameon=False)
54     fig.tight_layout()
55     fig.savefig(FIGURES_DIR / 'q3_event_zoom.pdf')
56     plt.close(fig)
57     artifacts = ['code/plot_q3_results.py', 'results/q3_solution.npz',
58                 'results/q3_validation.json', 'figures/q3_C_profiles.pdf', 'figures/q3_event_zoom.pdf']
59     provenance = {'result_version': validation['result_version'],
60                  'event_times_s': times.tolist(), 'event_maxima': maxima.tolist(),
61                  'event_lines': 'linear guides connecting saved states; no new PDE solve',
62                  'sha256': {p: hashlib.sha256((ROOT / p).read_bytes()).hexdigest() for p in artifacts}}
63     (out / 'figure_provenance.json').write_text(json.dumps(provenance, indent=2), encoding='utf-8')
64     print('Q3 profiles and event window redrawn from saved states')
65
66
67 if __name__ == '__main__':
68     main()

```

B.15 code/verify_closeout.py

```

1  """Audit the Q1/Q2 closeout against the preserved pre-closeout deliverables.
2
3  Run with numerical Python for --gate/--modal; bundled Python for --files/--render.
4  """
5  from pathlib import Path
6  import ast
7  import hashlib
8  import json
9  import sys
10 import tempfile
11 import numpy as np
12
13 ROOT = Path(__file__).resolve().parents[1]
14 OUT = ROOT / 'results/verification/closeout'
15 ARCHIVE = ROOT / 'results/archive/pre-closeout-20260911'
16
17
18 def dump(name, report):

```

```

19     OUT.mkdir(parents=True, exist_ok=True)
20     (OUT/name).write_text(json.dumps(report, ensure_ascii=False, indent=2), encoding='utf-8')
21     print(json.dumps(report, ensure_ascii=True, indent=2), flush=True)
22
23
24 def gate_test():
25     from delivery import make_gate, require_delivery
26     results = {}
27     with tempfile.TemporaryDirectory(dir=ROOT/'tmp') as d:
28         folder = Path(d)
29         for q in (1,2):
30             v = json.loads((ROOT/f'results/q{q}_validation.json').read_text(encoding='utf-8'))
31             baseline = v['delivery_gate']['checks']
32             assert baseline and all(baseline.values())
33             for key in baseline:
34                 checks = dict(baseline, **{key: False})
35                 sentinel = folder/'result.xlsx'
36                 sentinel.write_bytes(b'exiting approved data')
37                 try:
38                     require_delivery({'delivery_gate': make_gate(checks)}, folder/'diagnostic.json')
39                     sentinel.write_bytes(b'invalid new result')
40                 except RuntimeError:
41                     pass
42                 assert sentinel.read_bytes() == b'exiting approved data'
43                 assert key in json.loads((folder/'diagnostic.json').read_text(encoding='utf-8'))['delivery_
↳ _gate']['failed_checks']
44                 require_delivery({'delivery_gate': make_gate(baseline)}, folder/'diagnostic.json')
45                 # Ensure the production main itself puts the guard ahead of all writers.
46                 tree = ast.parse((ROOT/f'code/problem{q}.py').read_text(encoding='utf-8'))
47                 main = next(n for n in tree.body if isinstance(n, ast.FunctionDef) and n.name == 'main')
48                 calls = [n for n in ast.walk(main) if isinstance(n, ast.Call)]
49                 guard = next(n.lineno for n in calls if isinstance(n.func, ast.Name) and
↳ n.func.id == 'require_delivery')
50                 writers = [n.lineno for n in calls if (isinstance(n.func, ast.Name) and n.func.id in
↳ ('write_result_workbook', 'write_paper_csv', 'plot_figures')) or
↳ (isinstance(n.func, ast.Attribute) and n.func.attr in ('savez', 'write_text'))]
51                 assert writers and min(writers) > guard
52                 results[f'q{q}'] = {'failure_cases_blocked': len(baseline), 'valid_case_passed': True,
↳ 'all_production_writers_after_guard': True}
53     dump('gate_test.json', results)
54
55
56 def files():
57     from openpyxl import load_workbook
58     report = {}
59     for q, end, table_t in
↳ [(1,1800,np.array([100,300,600,900,1200,1500,1800])),(2,10800,np.arange(1800,10801,1800))]:
60         z = np.load(ROOT/f'results/q{q}_solution.npz')
61         old = np.load(ARCHIVE/f'results/q{q}_solution.npz')
62         validation = json.loads((ROOT/f'results/q{q}_validation.json').read_text(encoding='utf-8'))
63         r = {'version': validation['result_version'], 'delivery_passed':
↳ validation['delivery_gate']['meets_delivery_gate'], 'fields':{}}
64         assert r['delivery_passed']
65         assert np.array_equal(z['times'], np.arange(1,end+1))
66         assert np.all(np.diff(z['r'])>0)
67         assert np.allclose(z['output_radai_m'], np.arange(21)*.001,rtol=0,atol=1e-15)
68         idx = np.array([np.argmax(abs(z['r']-x)) for x in z['output_radai_m']])
69         assert np.max(abs(z['r'][idx]-z['output_radai_m'])) < 1e-14
70         wb = load_workbook(ROOT/f'results/result{q}.xlsx', read_only=True, data_only=False)
71         before = load_workbook(ARCHIVE/f'results/result{q}.xlsx', read_only=True, data_only=False)
72         assert wb.sheetnames == before.sheetnames == ['温度', '水分浓度']
73         for key, name, kind in [('T', '温度', 'temperature'), ('C', '水分浓度', 'moisture')]:
74             ws = wb[name]
75             assert (ws.max_row, ws.max_column) == (end+1, 22)
76             same = True

```

```

77     rows = ws.iter_rows()
78     header=next(rows)
79     assert np.allclose([c.value for c in header[1:]], np.arange(21)*.1,atol=1e-14,rtol=0)
80     values=[]
81     numeric=formatted=0
82     for i,row in enumerate(rows,1):
83         assert row[0].value==i
84         values.append([c.value for c in row[1:]])
85         numeric+=sum(c.data_type=='n' for c in row[1:])
86         formatted+=sum(c.number_format=='0.0000' for c in row[1:])
87     a=np.array(values)
88     assert np.isfinite(a).all()
89     assert numeric==formatted==end*21
90     assert np.max(abs(a-z[key+'_out']))<1e-12
91     for row1,row0 in zip(ws.iter_rows(),before[name].iter_rows(),strict=True):
92         if any((a.value,a.number_format,a.data_type)!=
93             (b.value,b.number_format,b.data_type) for a,b
94             ↪ in zip(row1,row0,strict=True)):
95             same=False
96         diff=float(np.max(abs(z[key+'_out']-old[key+'_out'])))
97         assert diff<=2e-5
98         assert np.array_equal(np.round(z[key+'_out'][table_t-1,:5],4),
99             ↪ np.round(old[key+'_out'][table_
100             ↪ t-1,:5],4))
101         csv=np.loadtxt(ROOT/f'results/q{q}_table_{kind}.csv',delimiter=',',skiprows=1)
102         assert np.max(abs(csv[:,1:]-z[key+'_out'][table_t-1,:5]))<=5.1e-9 # CSV intentionally stores
103             ↪ 8 decimals.
104         cp=z[key+'_checkpoints']
105         assert cp.shape==(len(z['checkpoint_times']),len(z['r']))
106         assert np.all(cp[0]==(28 if key=='T' else 2.55))
107         assert np.allclose(cp[1:,idx],z[key+'_out'][z['checkpoint_times'][1:].astype(int)-1],rtol=0,at
108             ↪ ol=1e-12)
109         if key+'_full' in z.files:
110             assert np.max(abs(z[key+'_full'][:,idx]-z[key+'_out']))<1e-12
111         r['fields'][key]={ 'output_cells':int(a.size), 'max_change_from_previous':diff, 'all_workbook_val
112             ↪ ues_types_formats_unchanged':same, 'paper_four_decimals_unchanged':True, 'excel_npz_csv_chec
113             ↪ kpoints_consistent':True}
114     wb.close();before.close()
115     r['source_sha256']={str(p.relative_to(ROOT)):hashlib.sha256(p.read_bytes()).hexdigest() for p in
116         ↪ [ROOT/f'code/problem{q}.py',ROOT/f'results/q{q}_solution.npz',ROOT/f'results/q{q}_validation.j
117         ↪ son',ROOT/f'results/result{q}.xlsx']}
118     report[f'q{q}']=r
119     dump('files.json',report)
120
121
122 def modal():
123     from scipy.special import hyp1f1
124     from q1_analytic import BesselDuhamel
125     report={}
126     for q in (1,2):
127         p=__import__(f'problem{q}')
128         _,env,_=getattr(p,f'load_q{q}_inputs')()
129         D=float(p.moisture_diffusivity(p.C0) if q==1 else p.moisture_diffusivity_q2(p.C0,p.T0))
130         values={}
131         coeff=env._pchip.c[:,0]
132         for terms in (120 if q==1 else 80,320,640,1280):
133             ref=BesselDuhamel(p.HM*p.R/D,D,p.R,terms)
134             moments=[hyp1f1(1,k+2,-ref.beta)/(k+1) for k in range(3)]
135             I=(env(0)-p.C0)*np.exp(-ref.beta)+coeff[2]*moments[0]+2*coeff[1]*moments[1]+3*coeff[0]*moments
136             ↪ [2]
137             exact=env(1)-I@ref.radial_modes(p.OUTPUT_RADII_M)
138             values[terms]=exact
139         ode=BesselDuhamel(p.HM*p.R/D,D,p.R,640).evaluate_many(p.OUTPUT_RADII_M,np.array([1.]),env,p.C0)[0]
140         diff=float(np.max(abs(ode-values[640])))
141         assert diff<1e-8
142         assert np.max(abs(values[640]-values[1280]))<1e-8

```

```

132     report[f'q{q}']={f'surface_C_at_1s':{str(k):float(v[-1]) for k,v in
    ↪ values.items()}, '640_vs_1280_max':float(np.max(abs(values[640]-values[1280]))), 'ode_vs_exact_c'
    ↪ onvolution_max':diff, 'passed':True}
133 dump('modal.json',report)
134
135
136 def render():
137     import pypdfium2 as pdfium
138     from PIL import Image, ImageDraw
139     folder=OUT/'previews';folder.mkdir(parents=True,exist_ok=True)
140     records=[]
141     paths=sorted((ROOT/'figures').glob('q[12]*.pdf'))
142     if (ROOT/'paper/q12_closeout.pdf').exists(): paths.append(ROOT/'paper/q12_closeout.pdf')
143     for path in paths:
144         doc=pdfium.PdfDocument(str(path))
145         for i in range(len(doc)):
146             page=doc[i];bm=page.render(scale=1.5)
147             dest=folder/f'{path.stem}_{i+1}.png'
148             im=bm.to_pil().convert('RGB');im.save(dest)
149             records.append((dest,im.copy()))
150             bm.close();page.close()
151         doc.close()
152     for offset in range(0,len(records),6):
153         contact=Image.new('RGB',(1500,1800),'white');draw=ImageDraw.Draw(contact)
154         for j,(path,im) in enumerate(records[offset:offset+6]):
155             im.thumbnail((740,560));x=j%2*750;y=j//2*600
156             contact.paste(im,(x,y+25));draw.text((x+5,y+5),path.stem,fill='black')
157         contact.save(folder/f'contact_{offset//6+1}.png')
158     dump('renders.json',{'pages':[str(p.relative_to(ROOT)) for p,_ in records]})
159
160
161 if __name__=='__main__':
162     if '--gate' in sys.argv: gate_test()
163     elif '--modal' in sys.argv: modal()
164     elif '--render' in sys.argv: render()
165     else: files()

```

B.16 code/verify_q2_results.py

```

1  """Q2 audit. Original results and production solver are never overwritten."""
2  from pathlib import Path
3  import hashlib
4  import json
5  import re
6  import sys
7  import time
8  import numpy as np
9
10 ROOT = Path(__file__).resolve().parents[1]
11 OUT = ROOT / 'results/verification/q2'
12 OUT.mkdir(parents=True, exist_ok=True)
13
14 def dump(name, data):
15     (OUT/name).write_text(json.dumps(data, ensure_ascii=False, indent=2), encoding='utf-8')
16     print(name, json.dumps(data, ensure_ascii=True), flush=True)
17
18 def files():
19     from openpyxl import load_workbook
20     import pypdfium2 as pdfium
21     from PIL import Image, ImageDraw
22     paths = [ROOT/'data/A.pdf', ROOT/'data/附件/附件 1.xlsx', ROOT/'data/附件/附件 3/result2.xlsx']
23     paths += list((ROOT/'results').glob('q2*')) + [ROOT/'results/result2.xlsx']
24     paths += list((ROOT/'figures').glob('q2*.pdf'))
25     paths += [ROOT/'code'/n for n in
    ↪ ['problem2.py', 'radial_coupled.py', 'appendix3.py', 'utils.py', 'radial_fvm.py']]

```

```

26 hashes = {str(p.relative_to(ROOT)):hashlib.sha256(p.read_bytes()).hexdigest() for p in paths}
27 z = np.load(ROOT/'results/q2_solution.npz')
28 v = json.loads((ROOT/'results/q2_validation.json').read_text(encoding='utf-8'))
29 report = dict(source_sha256=hashes, npz_keys=z.files, workbook={}, tables={}, figures=[])
30 report['axes'] = dict(times_ok=bool(np.array_equal(z['times'],np.arange(1,10801))),
31                       grid_increasing=bool(np.all(np.diff(z['r'])>0)), n_nodes=len(z['r']))
32 template = load_workbook(ROOT/'data/附件/附件 3/result2.xlsx',read_only=True)
33 report['template'] = {s.title:dict(dimensions=[s.max_row,s.max_column], first_rows=list(s.values)[:4])
34 ↪ for s in template}
35 template.close()
36 wb = load_workbook(ROOT/'results/result2.xlsx',read_only=True,data_only=False)
37 report['sheet_order_ok'] = wb.sheetnames == [' 温度', ' 水分浓度']
38 src = (ROOT/'reports/RESULTS_REPORT.md').read_text(encoding='utf-8').split('## 问题二结果')[1]
39 lines = [x for x in src.splitlines() if re.match(r'^\s*[\d\.\d\s\w\.,\s\<\/p>
40 ws=wb[sheet]; it=ws.iter_rows(); header=next(it)
41 values=[]; times=[]; numeric=formatted=0
42 for row in it:
43     times.append(row[0].value)
44     values.append([c.value for c in row[1:]])
45     numeric += sum(c.data_type=='n' for c in row[1:])
46     formatted += sum(c.number_format=='0.0000' for c in row[1:])
47 a=np.array(values,dtype=float); expected=z[key+'_out']
48 report['workbook'][key] = dict(dimensions=[ws.max_row,ws.max_column],
49                                times_ok=bool(np.array_equal(times,z['times'])),
50                                radii_ok=bool(np.allclose([c.value for c in header[1:]],np.arange(21)*.1,rtol=0,atol=1e-14)),
51                                all_finite=bool(np.isfinite(a).all()), numeric_cells=numeric, formatted_cells=formatted,
52                                max_difference_vs_npz=float(np.max(abs(a-expected))), minimum=float(a.min()),
53                                ↪ maximum=float(a.max()))
54 expected=expected[np.arange(1800,10801,1800)-1,:5]
55 csv=np.loadtxt(ROOT/'results/q2_table_{filename}.csv',delimiter=',',skiprows=1)
56 printed=np.array([[float(s.strip()) for s in line.split('|')[2:-1]] for line in lines[6*k:6*k+6]])
57 j=np.array([v['paper_table_'+key][f'h{h:g}_r{r:g}'] for h in np.arange(.5,3.1,.5) for r in
58 ↪ np.arange(5)*.5]).reshape(6,5)
59 report['tables'][key]=dict(csv_max_difference=float(abs(csv[:,1:]-expected).max()),
60                            csv_hours_ok=bool(np.array_equal(csv[:,0],np.arange(.5,3.1,.5))),
61                            report_four_decimal_matches=bool(np.array_equal(printed,np.round(expected,4))),
62                            json_max_difference=float(abs(j-expected).max()))
63 wb.close()
64 previews=[]
65 for p in sorted((ROOT/'figures').glob('q2*.pdf')):
66     doc=pdfium.PdfDocument(str(p)); texts=[]
67     for i in range(len(doc)):
68         pg=doc[i]; tp=pg.get_textpage(); texts.append(tp.get_text_range()); tp.close()
69         bm=pg.render(scale=2); im=bm.to_pil().convert('RGB'); im.save(OUT/'f'{p.stem}_{i+1}.png')
70         previews.append((p.stem,im.copy())); bm.close(); pg.close()
71     report['figures'].append(dict(name=p.name,pages=len(doc),text='\n'.join(texts)))
72     doc.close()
73 contact=Image.new('RGB',(1600,1600),'white'); draw=ImageDraw.Draw(contact)
74 for i,(name,im) in enumerate(previews):
75     im.thumbnail((790,480)); x=i%2*800; y=i//2*530
76     contact.paste(im,(x,y+25));draw.text((x+10,y+5),name,fill='black')
77 contact.save(OUT/'figure_contact.png')
78 doc=pdfium.PdfDocument(str(ROOT/'data/A.pdf'))
79 for i in [1,3]:
80     pg=doc[i]; bm=pg.render(scale=1.5); bm.to_pil().save(OUT/'f'problem_page_{i+1}.png');
81     ↪ bm.close();pg.close()
82 doc.close()
83 report['source_files_unchanged']=all(hashlib.sha256((ROOT/p).read_bytes()).hexdigest()==h for p,h in
84 ↪ hashes.items())
85 dump('file_audit.json',report)
86
87 def numerical():
88     import problem2 as p
89     start=time.perf_counter()

```

```

86 z=np.load(ROOT/'results/q2_solution.npz')
87 Ta,Ca,knots=p.load_q2_inputs()
88 report={'scope':'G4 reproduction, G4 time refinement, G8 space refinement, all 10800 seconds',
89 ↪ 'runs':{}}
90 for tag,level,rtol,at,step in
91 ↪ [ ('G4_reproduction',4,1e-9,1e-8,1.), ('G4_time',4,2e-10,2e-9,.5), ('G8_space',8,1e-9,1e-8,1.) ]:
92     print('Starting',tag,flush=True)
93     f=p.CoupledRadialFVM(p.graded_radial_nodes(level)); n=f.n_nodes
94     y0=f.pack(np.full(n,p.T0),np.full(n,p.C0))
95     ts,Y=p.integrate_coupled(f,y0,p.T_END,Ta,Ca,rtol,at,step)
96     T,C=p.extract_output(f,Y[1:])
97     result={key:**p.compare_on_output(z[key+'_out'],a,ts[1:]),**p.four_decimal_stats(z[key+'_out'],a,
98 ↪ ts[1:])} for key,a in [ ('T',T), ('C',C) ]
99     if tag=='G4_reproduction':
100         result['conservation_C']=p.conservation_moisture(f,ts,Y,Ca)
101         result['conservation_T']=p.conservation_heat(f,ts,Y,Ta)
102         result['full_grid_bounds']={ 'T_min':float(Y[:,n].min()), 'T_max':float(Y[:,n].max()), 'C_min'
103 ↪ :float(Y[:,n].min()), 'C_max':float(Y[:,n].max()) }
104         cp=np.array([0,1,60,100,1800,3600,5400,7200,9000,10800])
105         np.savez_compressed(OUT/'G4_checkpoints.npz',times=ts[cp],r=f.r,Y=Y[cp])
106     if tag=='G8_space':
107         changed=[]
108         for key,a in [ ('T',T), ('C',C) ]:
109             for ti in np.arange(1800,10801,1800)-1:
110                 for ri in range(0,21,5):
111                     b=float(z[key+'_out'][ti,ri]); q=float(a[ti,ri])
112                     if round(b,4)!=round(q,4):
113                         ↪ changed.append(dict(field=key,t_s=int(ti+1),r_cm=ri*.1,G4=b,G8=q))
114         result['paper_changed_details']=changed
115         np.savez_compressed(OUT/f'{tag}_outputs.npz',times=ts[1:],T_out=T,C_out=C)
116         report['runs'][tag]=result
117         dump('numerical_audit.json',report)
118         del Y,T,C,f
119     report['manufactured_solution']=p.run_mms(4)
120     # A short frozen-D check separates modal truncation from early FVM error.
121     f=p.CoupledRadialFVM(p.graded_radial_nodes(4)); n=f.n_nodes
122     D0=float(p.moisture_diffusivity_q2(p.C0,p.T0))
123     ts,Y=p.integrate_coupled(f,f.pack(np.full(n,p.T0),np.full(n,p.C0)),100.,Ta,Ca,2e-10,2e-9,.1,D_const=D0)
124     _,Cf=p.extract_output(f,Y[1:]); refs={}
125     for terms in [80,160,320]:
126         ref=p.BesselDuhamel(p.HM*p.R/D0,D0,p.R,n_terms=terms).evaluate_many(p.OUTPUT_RADII_M,ts[1:],Ca,p.C
127 ↪ 0)
128         refs[terms]=ref
129         report.setdefault('frozen_D_early',{str(terms)}[dict(max_abs=float(abs(Cf-ref).max()),t1_surfac
130 ↪ e_numeric=float(Cf[0,-1]),t1_surface_analytic=float(ref[0,-1]))
131     report['frozen_D_early']['160_vs_320_max_abs']=float(abs(refs[160]-refs[320]).max())
132     report['elapsed_s']=time.perf_counter()-start
133     dump('numerical_audit.json',report)
134
135 def modal():
136     # Closed integration of the first PCHIP derivative polynomial at t=1 s.
137     # This avoids the time integrator used by the production modal reference.
138     import problem2 as p
139     from scipy.special import hyp1f1
140     _,env,_=p.load_q2_inputs()
141     coeff=env._pchip.c[:,0]
142     D0=float(p.moisture_diffusivity_q2(p.C0,p.T0))
143     values={}
144     for n in [80,160,320,640]:
145         ref=p.BesselDuhamel(p.HM*p.R/D0,D0,p.R,n_terms=n)
146         beta=ref.beta
147         moments=[hyp1f1(1,k+2,-beta)/(k+1) for k in range(3)]
148         modes=(float(env(0))-p.C0)*np.exp(-beta)+coeff[2]*moments[0]+2*coeff[1]*moments[1]+3*coeff[0]*mome
149 ↪ nts[2]

```

```

142     values[str(n)]=float(env(1)-modes@ref.radial_modes(np.array([p.R]))[:,0])
143     dump('modal_truncation_audit.json',dict(t_s=1,surface_C=values,
144         difference_320_vs_640=abs(values['320']-values['640']),method='exact first-interval PCHIP
        ↪ polynomial convolution, hypergeometric moments'))
145
146 if __name__=='__main__':
147     if '--numerical' in sys.argv: numerical()
148     elif '--modal' in sys.argv: modal()
149     else: files()

```

B.17 code/verify_q3_closeout.py

```

1  """Regression and exhaustive delivery checks for Q3 closeout."""
2  from __future__ import annotations
3  import copy
4  import hashlib
5  import json
6  import tempfile
7  from pathlib import Path
8  from unittest.mock import patch
9  import numpy as np
10 from openpyxl import load_workbook
11 import problem3
12 import q3_closeout
13 from utils import ROOT,RESULTS_DIR
14 from q3_balance import combine
15
16
17 def main():
18     v=json.loads((RESULTS_DIR/'q3_validation.json').read_text(encoding='utf-8'))
19     assert v['result_version']=='q3-closeout-v1'
20     assert v['delivery_gate']['meets_delivery_gate'] and all(v['delivery_gate']['checks'].values())
21     checked=[]
22     with tempfile.TemporaryDirectory(prefix='q3_gate_',dir=ROOT/'tmp') as td:
23         root=Path(td);out=root/'result3.xlsx';out.write_bytes(b'ORIGINAL')
24         for name in v['delivery_gate']['checks']:
25             bad=copy.deepcopy(v);bad['delivery_gate']['checks'][name]=False
26             # Deliberately leave the top-level flag true to test recomputation.
27             with patch.object(problem3,RESULTS_DIR,root),patch.object(q3_closeout,'export_workbook') as
                ↪ writer:
28                 try:
29                     problem3.write_result3_xlsx(out,np.zeros((1,21)),np.array([60.]),validation=bad)
30                     raise AssertionError(f'failed gate allowed writing: {name}')
31                 except RuntimeError:
32                     pass
33                 assert not writer.called and out.read_bytes()==b'ORIGINAL'
34             checked.append(name)
35             with patch.object(q3_closeout,'DIAG',root/'diagnostics'):
36                 try:
37                     q3_closeout.publish_results(None, None, bad, None)
38                     raise AssertionError('failed gate reached production publisher')
39                 except RuntimeError:
40                     pass
41             missing=copy.deepcopy(v);del missing['delivery_gate']['checks'][name]
42             with patch.object(q3_closeout,'DIAG',root/'diagnostics'):
43                 try:
44                     q3_closeout.publish_results(None, None, missing, None)
45                     raise AssertionError('missing check reached production publisher')
46                 except RuntimeError:
47                     pass
48             empty=copy.deepcopy(v);empty['delivery_gate']['checks']={}
49             with patch.object(problem3,RESULTS_DIR,root),patch.object(q3_closeout,'export_workbook') as
                ↪ writer:
50                 try:
51                     problem3.write_result3_xlsx(out,np.zeros((1,21)),np.array([60.]),validation=empty)

```

```

52         raise AssertionError('empty gate passed')
53     except RuntimeError:
54         pass
55     assert not writer.called
56     with patch.object(problem3, 'RESULTS_DIR', root), patch.object(q3_closeout, 'export_workbook') as
57         ↪ writer:
58         problem3.write_result3_xlsx(out, np.zeros((1, 21)), np.array([60.]), validation=v)
59         assert writer.call_count==1
60     strict=problem3.find_strict_report(lambda t:.15-1e-6*(t-100.2), 100.2, 300., margin=1e-5)
61     assert strict['t_rep_s']==int(strict['t_rep_s'])
62     assert strict['M_hour4']<.15-1e-5
63     assert abs(strict['t_hour4_h']*10000-round(strict['t_hour4_h']*10000))<1e-8
64     for evaluator, hi in [(lambda t:float('nan'), 300.), (lambda t:.15-1e-6*(t-100.2), 100.21)]:
65         try:
66             problem3.find_strict_report(evaluator, 100.2, hi)
67             raise AssertionError('invalid/out-of-range strict report passed')
68         except RuntimeError:
69             pass
70     parts=v['continuous_balances']['parts']
71     combine(parts)
72     for field, delta in [(('start_s', .001), ('M_start', 1e-5), ('E_start', 1.))]:
73         bad=copy.deepcopy(parts); bad[1][field]+=delta
74         try:
75             combine(bad)
76             raise AssertionError('discontinuous balance segments passed')
77         except ValueError:
78             pass
79     # Independent weighted RHS identity checks both signs and the B-prime term.
80     from appendix3 import heat_capacity, heat_capacity_deriv
81     from radial_coupled import CoupledRadialFVM
82     from utils import R, L, H, HM
83     fvm=CoupledRadialFVM(np.linspace(0, R, 41)); n=fvm.n_nodes
84     T=35+8*(fvm.r/R)**2; C=1.2-.5*(fvm.r/R)**2
85     y=np.r_[T, C]; dy=fvm.rhs(0, y, 50., .05); factor=2*np.pi*L
86     storage_rate=factor*np.dot(fvm.W, heat_capacity(C)*dy[:n]+heat_capacity_deriv(C)*(T-28)*dy[n:])
87     correction=factor*np.dot(fvm.W, heat_capacity_deriv(C)*(T-28)*dy[n:])
88     def energy(z):
89         return factor*np.dot(fvm.W, heat_capacity(z[n:])*dy[n:-28])
90     finite_difference=(energy(y+1e-3*dy)-energy(y-1e-3*dy))/(2e-3)
91     assert np.isclose(finite_difference, storage_rate, rtol=1e-7)
92     assert np.isclose(storage_rate-correction, factor*R*H*(50-T[-1]), rtol=1e-12)
93     assert np.isclose(fvm.W@dy[n:], -R*HM*(C[-1]-.05), rtol=1e-12)
94     source=np.load(RESULTS_DIR/'q3_solution.npz')
95     wb=load_workbook(RESULTS_DIR/'result3.xlsx', read_only=True, data_only=True)
96     assert wb.sheetnames==[' 水分浓度']
97     rows=list(wb.active.iter_rows())
98     assert len(rows)==len(source['excel_times'])+1
99     assert all(len(row)==22 for row in rows)
100    assert np.allclose([c.value for c in rows[0][1:]], np.arange(21)*.1, atol=1e-14, rtol=0)
101    arr=np.array([[c.value for c in row] for row in rows[1:]], dtype=float)
102    delta=float(np.max(abs(arr[:, 1:]-source['C_out'])))
103    assert delta<1e-14
104    assert np.allclose(arr[:, 0], source['excel_times'], atol=1e-8, rtol=0)
105    assert all(c.number_format=='0.0000' for row in rows[1:] for c in row[1:])
106    wb.close()
107    assert np.array_equal(source['excel_times'][:-1], np.arange(60., source['t_rep'], 60.))
108    assert float(source['t_rep'])==v['t_rep_s']
109    n=int(source['n_nodes'])
110    assert source['Y'].shape==(len(source['times']), 2*n)
111    assert np.max(source['y_end'][n:]+v['moisture_margin'])<.15
112    indices=np.array([np.argmin(abs(source['r']-r)) for r in problem3.OUTPUT_RADII_M])
113    times, iy, ix=np.intersect1d(source['times'], source['excel_times'], return_indices=True)
114    assert len(times)==len(source['excel_times'])-1
115    assert np.array_equal(source['Y'][iy, n:][:, indices], source['C_out'][ix])
116    assert np.array_equal(source['Y'][iy, n:][:, indices], source['T_out'][ix])

```

```

116 assert np.array_equal(source['y_end'][n:][indices],source['C_out'][-1])
117 assert np.array_equal(source['Y'][0,:n],np.full(n,28.))
118 assert np.array_equal(source['Y'][0,n:],np.full(n,2.55))
119 assert abs(source['table_hours'][-1]-v['t_rep_h'])<1e-12
120 table=np.loadtxt(RESULTS_DIR/'q3_table_moisture.csv',delimiter=',',skiprows=1)
121 assert np.max(abs(table[:,1:]-source['C_table']))<6e-11
122 assert np.max(abs(table[:,0]-source['table_hours']))<6e-9
123 arc=RESULTS_DIR/'archive/q3-pre-closeout-20260911'
124 manifest=json.loads((arc/'manifest.json').read_text(encoding='utf-8-sig'))
125 archive_checks={p['path']:hashlib.sha256((arc/p['path']).read_bytes()).hexdigest().lower()==p['sha256']
↳ ].lower() for p in manifest}
126 assert all(archive_checks.values()),archive_checks
127 old=np.load(arc/'results/q3_solution.npz')
128 # Compare common regular 6-hour rows; the two report times differ deliberately.
129 table_changed=int(np.count_nonzero(np.round(old['C_table'][:-1],4)!=np.round(source['C_table'][:-1],4))
↳ )
130 matched,io,inew=np.intersect1d(old['excel_times'],source['excel_times'],return_indices=True)
131 common_delta=float(np.max(abs(old['C_out'][io]-source['C_out'][inew])))
132 protected=json.loads((arc/'protected.json').read_text(encoding='utf-8-sig'))
133 protected_checks={p['path']:hashlib.sha256((ROOT/p['path']).read_bytes()).hexdigest().lower()==p['sha2
↳ 56'].lower() for p in protected}
134 assert all(protected_checks.values()),protected_checks
135 source_checks={p:hashlib.sha256((ROOT/p).read_bytes()).hexdigest()==sha for p,sha in
↳ v['source_sha256'].items()}
136 assert all(source_checks.values()),source_checks
137 js=json.loads((q3_closeout.DIAG/'js_gate_test.json').read_text(encoding='utf-8'))
138 assert js['pass'] and set(js['required_checks'])==q3_closeout.REQUIRED_CHECKS
139 baseline={}
140 for q in (1,2,3,4):
141     data=json.loads((RESULTS_DIR/f'q{q}_validation.json').read_text(encoding='utf-8'))
142     gate=data.get('delivery_gate',{})
143     baseline[f'q{q}']=bool(gate.get('meets_delivery_gate',data.get('pass',False)))
144 assert all(baseline.values()),baseline
145 report_rows=[]
146 for line in (ROOT/'reports/Q3_VERIFY_REPORT.md').read_text(encoding='utf-8').splitlines():
147     cells=[c.strip() for c in line.strip('|').split('|')]
148     if not line.startswith('|') or len(cells)!=6:
149         continue
150     try:
151         report_rows.append([float(c) for c in cells])
152     except ValueError:
153         continue
154 report_table=np.asarray(report_rows)
155 assert report_table.shape==(len(source['table_hours']),6)
156 assert np.allclose(report_table[:,0],source['table_hours'],rtol=0,atol=1e-10)
157 assert np.array_equal(report_table[:,1:],np.round(source['C_table'],4))
158 provenance=json.loads((RESULTS_DIR/'diagnostics/q3_delivery/figure_provenance.json').read_text(encodin
↳ g='utf-8'))
159 assert all(hashlib.sha256((ROOT/p).read_bytes()).hexdigest()==sha for p,sha in
↳ provenance['sha256'].items())
160 visual=json.loads((RESULTS_DIR/'diagnostics/q3_delivery/visual_review.json').read_text(encoding='utf-8')
↳ )
161 assert visual['pass']
162 figure_hashes={str(p.relative_to(ROOT)):hashlib.sha256(p.read_bytes()).hexdigest()
↳ for p in sorted((ROOT/'figures').glob('q3_*.pdf'))}
163 assert len(figure_hashes)==5
164 prior=json.loads((RESULTS_DIR/'verification/closeout/files.json').read_text(encoding='utf-8'))
165 prior_hashes={path:hashlib.sha256((ROOT/path).read_bytes()).hexdigest()==sha
↳ for q in ('q1','q2') for path,sha in prior[q]['source_sha256'].items()}
166 assert all(prior_hashes.values()),prior_hashes
167 audit={'pass':True,'gate_failures_blocked':checked,'missing_required_checks_blocked':checked,
170 'publisher_failure_blocked':True,'js_gate_test':js,'balance_continuity_regression':True,
171 'weighted_rhs_balance_identity':True,'source_hashes_match':source_checks,

```

```

172         'internal_fields_and_initial_state_match':True,'empty_gate_blocked':True,'valid_gate_reaches_wr|
↪ iter':True,
173         'strict_report_rounding_regression':True,'workbook_rows_including_header':len(rows),'columns':2|
↪ 2,
174         'export_max_abs_difference':delta,'regular_table_four_decimal_changes_from_old':table_changed,
175         'common_excel_max_C_change_from_old':common_delta,'old_critical_h':float(old['t_star'])/3600,
176         'new_critical_h':v['t_star_h'],'old_report_h':float(old['t_rep'])/3600,'new_report_h':v['t_rep_|
↪ h'],
177         'protected_files_unchanged':protected_checks,'archive_hashes_match':archive_checks,
178         'q12_prior_evidence_hashes_match':prior_hashes,'four_question_numerical_gates':baseline,
179         'report_table_matches_source':True,'figure_provenance_matches':True,
180         'figure_sha256':figure_hashes,'visual_review':visual,
181         'result3_sha256':hashlib.sha256((RESULTS_DIR/'result3.xlsx').read_bytes()).hexdigest()}
182     (RESULTS_DIR/'q3_closeout_delivery_validation.json').write_text(json.dumps(audit,ensure_ascii=False,in|
↪ dent=2),encoding='utf-8')
183     print(json.dumps(audit,ensure_ascii=False),flush=True)
184
185
186 if __name__=='__main__':main()

```

B.18 code/verify_q4_delivery.py

```

1  """Read-only exhaustive audit of the exported Q4 workbook."""
2  import hashlib
3  import json
4  from pathlib import Path
5  import numpy as np
6  from openpyxl import load_workbook
7  from problem4 import OUT
8  from utils import RESULTS_DIR
9
10 payload=json.loads((OUT/'excel_payload.json').read_text(encoding='utf-8'))
11 path=RESULTS_DIR/'result4.xlsx'
12 wb=load_workbook(path,read_only=True,data_only=True)
13 assert wb.sheetnames==['Sheet1'],wb.sheetnames
14 ws=wb.active
15 rows=list(ws.iter_rows())
16 assert len(rows)==len(payload['rows'])+1
17 actual_columns=max(map(len,rows))
18 assert actual_columns==len(payload['headers'])
19 assert all(len(row)==actual_columns for row in rows)
20 assert [c.value for c in rows[0]]==payload['headers']
21 max_error=0.;blanks=0;numeric=0
22 for row,expected in zip(rows[1:],payload['rows']):
23     for j,(cell,value) in enumerate(zip(row,expected)):
24         if value is None:
25             assert cell.value is None,(cell.coordinate,cell.value)
26             blanks+=1
27         else:
28             assert isinstance(cell.value,(int,float)),(cell.coordinate,cell.value)
29             max_error=max(max_error,abs(cell.value-value))
30             numeric+=1
31             if j>0:
32                 assert cell.number_format=='0.0000',(cell.coordinate,cell.number_format)
33 assert max_error<1e-10,max_error
34 wb.close()
35 source=np.load(RESULTS_DIR/'q4_solution.npz')
36 times=source['times_s'][1:]
37 assert np.array_equal(times[:-1],np.arange(60.,times[-1],60.))
38 assert source['C_physical'].shape[0]==len(times)+1
39 n=len(source['xi'])
40 imax=np.argmax(source['Y'][:,n:],axis=1)
41 center_gap=float(np.max(np.max(source['Y'][:,n:],axis=1)-source['Y'][:,n]))
42 assert center_gap<1e-10,center_gap

```

```

43 audit={'pass':True,'sheet':'Sheet1','rows_including_header':len(rows),'columns':actual_columns,
44       'numeric_cells':numeric,'domain_blank_cells':blanks,'max_export_abs_error':max_error,
45       'initial_state_only_in_source':True,'final_time_s':float(times[-1]),
46       'max_position_is_center_all_saved_times':bool(np.all(imax==0)),
47       'max_center_gap_all_saved_times':center_gap,
48       'event_max_position_xi':float(source['xi'][np.argmax(source['critical_state'])[n:]]),
49       'report_max_position_xi':float(source['xi'][np.argmax(source['report_state'])[n:]]),
50       'sha256':hashlib.sha256(path.read_bytes()).hexdigest()}
51 (RESULTS_DIR/'q4_delivery_validation.json').write_text(json.dumps(audit,ensure_ascii=False,indent=2),encoding='utf-8')
52 print(json.dumps(audit,ensure_ascii=False))

```

B.19 tmp/q3_export/build.mjs

```

1 import fs from 'node:fs/promises';
2 import path from 'node:path';
3 import {fileURLToPath} from 'node:url';
4 import {FileBlob,SpreadsheetFile} from '@oai/artifact-tool';
5 import {requireQ3Delivery} from './gate.mjs';
6 const here=path.dirname(fileURLToPath(import.meta.url));
7 const root=path.resolve(here,'../..');
8 const original=path.join(root,'results/result3.xlsx');
9 const wb=await SpreadsheetFile.importXlsx(await FileBlob.load(original));
10 if(process.argv.includes('--preview-only')){
11   console.log((await
12     ↪ wb.inspect({kind:'workbook',sheet:'table',maxChars:1200,tableMaxRows:3,tableMaxCols:4})).ndjson);
13   const view=await wb.render({sheetName:' 水分浓度',range:'A1:H10',scale:1.3,format:'png'});
14   await fs.writeFile(path.join(here,'before.png'),new Uint8Array(await view.arrayBuffer()));
15 } else {
16   const v=JSON.parse(await
17     ↪ fs.readFile(path.join(root,'results/diagnostics/q3_closeout/pre_export.json'),'utf8'));
18   requireQ3Delivery(v);
19   const data=JSON.parse(await
20     ↪ fs.readFile(path.join(root,'results/diagnostics/q3_closeout/excel_payload.json'),'utf8'));
21   const sheet=wb.worksheets.getItem(data.sheet);
22   const matrix=[data.headers,...data.rows],nr=matrix.length,nc=matrix[0].length;
23   sheet.getUsedRange().clear({applyTo:'contents'});
24   sheet.getRangeByIndexes(0,0,nr,nc).values=matrix;
25   sheet.getRangeByIndexes(1,1,nr-1,nc-1).setNumberFormat('0.0000');
26   sheet.getRangeByIndexes(1,0,nr-1,1).setNumberFormat('0');
27   sheet.getRangeByIndexes(nr-1,0,1,1).setNumberFormat('0.00');
28   sheet.getRangeByIndexes(0,0,nr,1).format.columnWidth=30;
29   sheet.getRangeByIndexes(0,1,nr,nc-1).format.columnWidth=11;
30   wb.calculate();
31   console.log((await wb.inspect({kind:'table',range:'水分浓度!A1:F4',include:'values,formulas',tableMaxRow:
32     ↪ s:4,tableMaxCols:6,maxChars:1200})).ndjson);
33   console.log((await wb.inspect({kind:'match',searchTerm:'#REF!|#DIV/0!|#VALUE!|#NAME\\?|#NUM!',options:{u:
34     ↪ seRegex:true,maxResults:10},maxChars:1000})).ndjson);
35   const preview=await wb.render({sheetName:data.sheet,sheet,range:'A1:H10',scale:1.3,format:'png'});
36   await fs.writeFile(path.join(here,'after.png'),new Uint8Array(await preview.arrayBuffer()));
37   const tail=await wb.render({sheetName:data.sheet,range:`A${nr-6}:V${nr}`,scale:1,format:'png'});
38   await fs.writeFile(path.join(here,'tail.png'),new Uint8Array(await tail.arrayBuffer()));
39   const output=await SpreadsheetFile.exportXlsx(wb);await output.save(data.path);
40   console.log(JSON.stringify({path:data.path,rows:nr,cols:nc}));
41 }

```

B.20 tmp/q3_export/gate.mjs

```

1 export const requiredChecks = [
2   'event_found', 'finite', 'strict_below_all_verified_grids',
3   'strict_below_with_empirical_margin', 'moisture_bounds', 'temperature_bounds',
4   'moisture_balance_1e-6', 'heat_balance_1e-6', 'quadrature_convergence_1e-7',
5   'space_event_60s', 'time_event_60s', 'space_full_fields_2e-5',

```

```

6   'time_full_fields_2e-5', 'table_four_decimals', 'q2_overlap',
7 ];
8
9 export function requireQ3Delivery(v) {
10  const gate = v?.delivery_gate;
11  if (v?.result_version !== 'q3-closeout-v1' || gate?.meets_delivery_gate !== true ||
12      !requiredChecks.every(name => gate?.checks?.[name] === true) ||
13      !Object.values(gate.checks).every(value => value === true)) {
14    throw new Error('Numerical validation failed. Q3 export blocked.');
```

B.21 tmp/q3_export/test_gate.mjs

```

1  import assert from 'node:assert/strict';
2  import fs from 'node:fs/promises';
3  import {requireQ3Delivery, requiredChecks} from './gate.mjs';
4  const valid = {result_version:'q3-closeout-v1', delivery_gate:{
5    checks:Object.fromEntries(requiredChecks.map(name=>[name,true])), meets_delivery_gate:true,
6  }};
7  requireQ3Delivery(valid);
8  for (const name of requiredChecks) {
9    const bad = structuredClone(valid);
10   bad.delivery_gate.checks[name] = false;
11   assert.throws(()=>requireQ3Delivery(bad));
12   delete bad.delivery_gate.checks[name];
13   assert.throws(()=>requireQ3Delivery(bad));
14 }
15 for (const bad of [{}, {...valid,delivery_gate:{checks:{},meets_delivery_gate:true}},
16   {...valid,delivery_gate:{...valid.delivery_gate,meets_delivery_gate:false}}]) {
17   assert.throws(()=>requireQ3Delivery(bad));
18 }
19 const result = {pass:true, required_checks:requiredChecks, failed_and_missing_checks_blocked:30,
20   malformed_gates_blocked:3, valid_gate_passes:true};
21 await fs.writeFile(new URL('../results/diagnostics/q3_closeout/js_gate_test.json',import.meta.url),
22   ↪ JSON.stringify(result,null,2));
23 console.log(JSON.stringify(result));
```

B.22 tmp/q4_export/build.mjs

```

1  import fs from 'node:fs/promises';
2  import path from 'node:path';
3  import {fileURLToPath} from 'node:url';
4  import {Workbook,SpreadsheetFile} from '@oai/artifact-tool';
5
6  const here=path.dirname(fileURLToPath(import.meta.url));
7  const root=path.resolve(here,'../..');
8  const audit=JSON.parse(await fs.readFile(path.join(root,'results/q4_validation.json'),'utf8'));
9  if(!audit.pass) throw new Error('Q4 numerical validation failed; export stopped.');
```

```

24 sheet.getRangeByIndexes(nr-1,0,1,1).setNumberFormat('0.00');
25 const header=sheet.getRangeByIndexes(0,0,1,nc);
26 header.format.fill='#E7EDF4';
27 header.format.font={name:'Microsoft YaHei',size:10,bold:true};
28 header.format.rowHeight=26;
29 header.format.horizontalAlignment='center';
30 sheet.getRangeByIndexes(1,0,nr-1,nc).format.horizontalAlignment='right';
31 sheet.freezePanes.freezeRows(1);
32 sheet.freezePanes.freezeColumns(1);
33 sheet.showGridLines=false;
34 wb.recalculate();
35 console.log((await wb.inspect({kind:'table',range:'Sheet1!A1:F5',include:'values,formulas',tableMaxRows:5,
    ↳ tableMaxCols:6,maxChars:1800})).ndjson);
36 console.log((await wb.inspect({kind:'match',searchTerm:'#REF!|#DIV/0!|#VALUE!|#NAME\\?|#NUM!',options:{use
    ↳ Regex:true,maxResults:10},maxChars:1000})).ndjson);
37 const preview=await wb.render({sheetName:'Sheet1',range:'A1:H12',scale:1.4,format:'png'});
38 await fs.writeFile(path.join(here,'preview.png'),new Uint8Array(await preview.arrayBuffer()));
39 const tailPreview=await wb.render({sheetName:'Sheet1',range:`A${nr-7}:W${nr}`,scale:1,format:'png'});
40 await fs.writeFile(path.join(here,'preview_tail.png'),new Uint8Array(await tailPreview.arrayBuffer()));
41 const xlsx=await SpreadsheetFile.exportXlsx(wb);
42 await xlsx.save(path.join(root,'results/result4.xlsx'));
43 console.log(JSON.stringify({output:'results/result4.xlsx',rows:nr,columns:nc,report_h:payload.report_h}));

```

补充对照程序与复现

附录 4 固定半径对照的完整程序如下。复算依次运行：

```

python code/q4_fixed_comparison.py --level 8
python code/q4_fixed_comparison.py --level 16
python code/q4_fixed_comparison.py --level 8 --tight
python code/q4_fixed_comparison.py --summarize
python code/plot_q4_fixed_comparison.py

```

汇总需先按前述步骤生成第四问原收缩结果。若仅复核图形，可直接使用支撑材料所附的对照 JSON 与两组中心曲线 CSV；较大的固定组内部数组可重算。

```

1 """Appendix-4 fixed-radius control. Existing production results are read-only.
2
3 Run --level 8, --level 16, --level 8 --tight, then --summarize.
4 The fixed control requires no extrapolation of the measured shrinking radius.
5 """
6 from pathlib import Path
7 import argparse
8 import csv
9 import hashlib
10 import json
11 import math
12 import time
13 import numpy as np
14 from numpy.polynomial.legendre import leggauss
15 from scipy.integrate import solve_ivp
16 from problem4 import Inputs, ShrinkingFVM, properties, SWITCH, THRESHOLD
17 from radial_fvm import graded_radial_nodes, harmonic_mean
18 from utils import RESULTS_DIR, R, T0, C0, H, HM
19
20 OUT = RESULTS_DIR / 'q4_fixed_comparison'
21
22
23 def dump(path, obj):
24     path.write_text(json.dumps(obj, ensure_ascii=False, indent=2), encoding='utf-8')

```

```

25
26
27 def run(level, tight):
28     OUT.mkdir(parents=True, exist_ok=True)
29     inputs = Inputs()
30     fvm = ShrinkingFVM(graded_radial_nodes(level)/R)
31     n = fvm.n
32     label = f'G{level}' + ('_tight' if tight else '')
33     rtol = 1e-9 if tight else 1e-8
34     atol = np.r_[np.full(n, 1e-9 if tight else 1e-8), np.full(n, 1e-11 if tight else 1e-10)]
35     y = np.r_[np.full(n, T0), np.full(n, C0)]
36     times, states = [np.array([0.])], [y[None, :]]
37     event = lambda t, z: float(z[n:].max())-THRESHOLD
38     event.direction, event.terminal = -1., False
39     breaks = np.unique(np.r_[0., SWITCH, np.arange(21600., 864000.+1., 21600.)])
40     gx, gw = leggauss(4)
41     loss, mass_error, heat_error = 0., 0., 0.
42     bounds = [T0, T0, C0, C0]
43     segments = []
44     critical = None
45     started = time.perf_counter()
46     for a, b in zip(breaks[:-1], breaks[1:]):
47         late = a >= SWITCH
48         def rhs(t, z):
49             ta, ca = inputs.environment(t, late=late)
50             return fvm.rhs(z, R, ta, ca)
51         step = (5. if tight else 10.) if not late else (30. if tight else 60.)
52         sol = solve_ivp(rhs, (a, b), y, method='BDF', rtol=rtol, atol=atol,
53             max_step=step, jac_sparsity=fvm.sparsity(), dense_output=True, events=event)
54         if not sol.success:
55             raise RuntimeError(sol.message)
56         ts = np.arange(a+60., b+1, 60.)
57         yy = sol.sol(ts).T
58         times.append(ts); states.append(yy)
59         for start in range(0, len(sol.t)-1, 128):
60             aa, bb = sol.t[start:-1][:128], sol.t[start+1:][:128]
61             qq = ((aa+bb)[:,:None]/2 + (bb-aa)[:,:None]/2*gx).ravel()
62             surf = sol.sol(qq)[-1]
63             ca = np.full_like(qq, .05) if late else inputs.Ca(qq)
64             flux = 2*HM/R*(surf-ca)
65             loss += float(np.sum(flux.reshape(-1,4)*gw*(bb-aa)[:,:None]/2))
66             y = sol.y[:,-1]
67             mass_error = max(mass_error, abs(float(2*np.dot(fvm.w,y[n:])-C0+loss)))
68             bounds = [min(bounds[0],float(sol.y[n:].min())), max(bounds[1],float(sol.y[n:].max())),
69                 min(bounds[2],float(sol.y[n:].min())), max(bounds[3],float(sol.y[n:].max()))]
70         for tq in (a,(a+b)/2,b):
71             z = sol.sol(tq); dz = rhs(tq,z); B,_,_ = properties(z[n:],z[n:])
72             ta,_ = inputs.environment(tq,late=late)
73             heat_error = max(heat_error,abs(float(np.dot(fvm.w*B,dz[:n])+H/R*(z[n-1]-ta))))
74             segments.append({'start_s':float(a),'end_s':float(b),'steps':len(sol.t)-1,'mass_abs':mass_error})
75             print(f'{label}: {b/3600:.0f} h, maxC={y[n:].max():.8f}, wall={time.perf_counter()-started:.1f}s',
76                 ↵ flush=True)
77             if len(sol.t_events[0]):
78                 critical = float(sol.t_events[0][0])
79                 event_state = sol.y_events[0][0]
80                 break
81         if critical is None:
82             raise RuntimeError('Fixed control has not crossed threshold by the 240 h numerical cap')
83     tt, YY = np.concatenate(times), np.vstack(states)
84     if not np.all(np.isfinite(YY)):
85         raise RuntimeError('nonfinite state')
86     # Independent physical-ring assembly verifies fixed-radius transformation.
87     rp = R*fvm.xi; rf = (rp[:-1]+rp[1:])/2
88     vol = np.diff(np.r_[0.,rf,R]**2)/2
89     z = np.r_[35.+10*fvm.xi**2, 1.7-1.2*fvm.xi**2]

```

```

89     B,k,D = properties(z[n:],z[:n])
90     ft = np.r_[0.,-rf*harmonic_mean(k[:-1],k[1:])*np.diff(z[:n])/np.diff(rp),R*H*(z[n-1]-50.)]
91     fc = np.r_[0.,-rf*harmonic_mean(D[:-1],D[1:])*np.diff(z[:n])/np.diff(rp),R*HM*(z[-1]-.05)]
92     ref = np.r_[-np.diff(ft)/(vol*B),-np.diff(fc)/vol]
93     operator_error = float(np.max(abs(ref-fvm.rhs(z,R,50.,.05))))
94     meta = {'label':label,'nodes':n,'rtol':rtol,'atol_T':float(atol[0]),'atol_C':float(atol[n]),
95            'critical_s':critical,'critical_h':critical/3600,'radius_m':R,'properties':'Appendix 4',
96            'input_audit':inputs.audit,'mass_relative_error':mass_error/C0,'heat_equation_residual':heat_e
↪         rror,
97            'physical_operator_error':operator_error,'bounds':bounds,'segments':segments,
98            'max_center_gap':float(np.max(YY[:,n:].max(axis=1)-YY[:,n])),
99            'wall_s':time.perf_counter()-started,
100            'source_sha256':{p.name:hashlib.sha256(p.read_bytes()).hexdigest() for p in
101                            [Path(__file__),Path(__file__).with_name('problem4.py'),Path(__file__).with_n
↪                             ame('radial_fvm.py')]]}
102     np.savez_compressed(OUT/f'{label}.npz',times_s=tt,Y=YY,xi=fvm.xi,event_state=event_state)
103     dump(OUT/f'{label}.json',meta)
104     print(json.dumps(meta,ensure_ascii=False),flush=True)
105
106
107 def state_at(data, t):
108     times = data['times_s']; i = int(np.searchsorted(times,t,side='right')-1)
109     if i < 0 or t > times[-1]:
110         raise ValueError('state query outside saved trajectory')
111     if abs(times[i]-t)<1e-8:
112         return data['Y'][i].copy()
113     assert times[i]>=SWITCH
114     fvm = ShrinkingFVM(data['xi']); n=fvm.n
115     sol = solve_ivp(lambda tt,y:fvm.rhs(y,R,50.,.05),(times[i],t),data['Y'][i],
116                    method='BDF',rtol=1e-10,atol=np.r_[np.full(n,1e-10),np.full(n,1e-12)],
117                    max_step=1.,jac_sparsity=fvm.sparsity())
118     if not sol.success: raise RuntimeError(sol.message)
119     return sol.y[:,-1]
120
121
122 def summarize():
123     labels = ['G8','G16','G8_tight']
124     ds = {k:np.load(OUT/f'{k}.npz') for k in labels}
125     ms = {k:json.loads((OUT/f'{k}.json').read_text(encoding='utf-8')) for k in labels}
126     a,b,c = (ds[k] for k in labels); n,m = len(a['xi']),len(b['xi'])
127     assert np.allclose(a['xi'],b['xi'][:2],rtol=0,atol=1e-14)
128     count = min(len(d['times_s']) for d in ds.values())
129     assert all(np.array_equal(a['times_s'][:count],d['times_s'][:count]) for d in ds.values())
130     space = [float(np.max(abs(a['Y'][:count,n]-b['Y'][:count,m:2]))),
131             float(np.max(abs(a['Y'][:count,n]-b['Y'][:count,m:2])))]
132     temporal = [float(np.max(abs(a['Y'][:count,n]-c['Y'][:count,n]))),
133               float(np.max(abs(a['Y'][:count,n]-c['Y'][:count,n])))]
134     critical = ms['G8']['critical_s']
135     crit_max = {k:float(state_at(d,critical)[len(d['xi']):].max()) for k,d in ds.items()}
136     margin = 2*max(abs(crit_max['G8']-crit_max[k]) for k in labels[1:])+1e-8
137     report = math.ceil((max(v['critical_s'] for v in ms.values())+2)/.36)*.36
138     for _ in range(120):
139         report_states = {k:state_at(d,report) for k,d in ds.items()}
140         maxima = {k:float(report_states[k][len(ds[k]['xi']):].max()) for k in labels}
141         if max(maxima.values())<THRESHOLD and maxima['G8']+margin<THRESHOLD: break
142         report += .36
143     spatial_event = abs(ms['G8']['critical_s']-ms['G16']['critical_s'])
144     temporal_event = abs(ms['G8']['critical_s']-ms['G8_tight']['critical_s'])
145     shrink = json.loads((RESULTS_DIR/'q4_validation.json').read_text(encoding='utf-8'))
146     checks = {'spatial_T_2e-5':space[0]<2e-5,'spatial_C_2e-5':space[1]<2e-5,
147             'temporal_T_2e-5':temporal[0]<2e-5,'temporal_C_2e-5':temporal[1]<2e-5,
148             'spatial_event_1s':spatial_event<1,'temporal_event_1s':temporal_event<1.,
149             'mass_1e-6':all(v['mass_relative_error']<1e-6 for v in ms.values()),
150             'heat_equation_1e-6':all(v['heat_equation_residual']<1e-6 for v in ms.values()),
151             'fixed_operator_1e-8':all(v['physical_operator_error']<1e-8 for v in ms.values()),

```

```

152         'physical_C_bounds':all(v['bounds'][2]>=0 and v['bounds'][3]<=C0+1e-8 for v in ms.values()),
153         'center_is_maximum':all(v['max_center_gap']<1e-10 for v in ms.values()),
154         'strict_report':max(maxima.values())<THRESHOLD and maxima['G8']+margin<THRESHOLD,
155         'shrink_validated':shrink['pass'],
156         'same_inputs':all(v['input_audit']['source_sha256']==shrink['runs']['G8']['input_audit']['source_sha256'] for v in ms.values())}
157     result = {'version':'q4-fixed-comparison-v1','fixed_critical_h':critical/3600,
158             'fixed_report_h':report/3600,'fixed_report_s':report,'fixed_report_maxima':maxima,
159             'fixed_empirical_margin':margin,'fixed_spatial_T_C':space,'fixed_temporal_T_C':temporal,
160             'fixed_event_space_s':spatial_event,'fixed_event_time_s':temporal_event,
161             'shrink_critical_h':shrink['critical_h'],'shrink_report_h':shrink['report_h'],
162             'saving_critical_h':(critical-shrink['critical_s'])/3600,
163             'saving_report_h':report/3600-shrink['report_h'],
164             'saving_percent':100*(report-shrink['report_s'])/report,
165             'checks':checks,'pass':all(checks.values()),'runs':ms,
166             'scope':'Controlled model comparison under the same Appendix-4 properties and environment;
167             ↳ not experimental causal evidence.'}
168     dump(OUT/'comparison.json',result)
169     if not result['pass']: raise RuntimeError(checks)
170     mask=a['times_s']<report
171     tt=np.r_[a['times_s'][mask],report]
172     yy=np.vstack([a['Y'][mask],report_states['G8']])
173     np.savez_compressed(OUT/'fixed_solution.npz',times_s=tt,xi=a['xi'],Y=yy)
174     sd=np.load(RESULTS_DIR/'q4_solution.npz'); sn=len(sd['xi'])
175     for name, ts, cc in [(('fixed',tt,yy[:,n]),('shrink',sd['times_s'],sd['Y'][:,sn]))]:
176         with (OUT/f'{name}_center.csv').open('w',newline='',encoding='utf-8-sig') as f:
177             w=csv.writer(f);w.writerow(['time_s','center_C_kg_per_kg']);w.writerows(zip(ts,cc))
178     print(json.dumps({k:v for k,v in result.items() if k!='runs'},ensure_ascii=False,indent=2))
179
180 if __name__=='__main__':
181     p=argparse.ArgumentParser();p.add_argument('--level',type=int,default=8)
182     p.add_argument('--tight',action='store_true');p.add_argument('--summarize',action='store_true')
183     args=p.parse_args()
184     summarize() if args.summarize else run(args.level,args.tight)

1 """Plot verified Appendix-4 control trajectories, without rerunning the PDE."""
2 import json
3 import numpy as np
4 from q4_fixed_comparison import OUT
5 from problem2 import setup_mpl
6 from utils import FIGURES_DIR
7
8
9 def main():
10     result=json.loads((OUT/'comparison.json').read_text(encoding='utf-8'))
11     if not result['pass']: raise RuntimeError('Unverified comparison')
12     data={k:np.loadtxt(OUT/f'{k}_center.csv',delimiter=',',skiprows=1,encoding='utf-8-sig') for k in
13     ↳ ['fixed','shrink']}
14     plt=setup_mpl()
15     plt.rcParams.update({'font.size':9,'axes.labelsize':9,'legend.fontsize':8,'pdf.fonttype':42})
16     fig,(ax,zoom)=plt.subplots(1,2,figsize=(7.2,3.15),gridspec_kw={'width_ratios':[1.05,1]})
17     styles={'fixed':('#B65B3B','--',' 固定半径 $R=R_0$'), 'shrink':('#247B87','-', ' 实测收缩 $R(t)$')}
18     for key,a in data.items():
19         color,ls,label=styles[key]
20         for axis in (ax,zoom):
21             axis.plot(a[:,0]/3600,a[:,1],color=color,ls=ls,lw=1.5,label=label)
22             zoom.plot(a[-1,0]/3600,a[-1,1],color=color,ms=4)
23     for axis in (ax,zoom):
24         axis.axhline(.15,color='.4',ls=':',lw=1,label=' 达标阈值 0.15')
25         axis.set_xlabel(' 时间 / h')
26         axis.spines[['top','right']].set_visible(False)
27     ax.set_ylabel=' 中心干基含水率 / (kg/kg)',xlim=(0,138),ylim=(0,2.65))
28     ax.legend(frameon=False,loc='upper right')
29     zoom.set(xlim=(24,138),ylim=(.135,.43),ylabel=' 中心干基含水率 / (kg/kg)')

```

```

29     zoom.annotate(f"{result['shrink_report_h']:.4f} h", (result['shrink_report_h'], .15), xytext=(40, .22),
30                   arrowprops={'arrowstyle':'-', 'color':styles['shrink'][0]}, color=styles['shrink'][0], font_
    ↪ size=8)
31     zoom.annotate(f"{result['fixed_report_h']:.4f} h", (result['fixed_report_h'], .15), xytext=(97, .25),
32                   arrowprops={'arrowstyle':'-', 'color':styles['fixed'][0]}, color=styles['fixed'][0], fontsi_
    ↪ ze=8)
33     fig.tight_layout(w_pad=1.8)
34     FIGURES_DIR.mkdir(exist_ok=True)
35     fig.savefig(FIGURES_DIR/'q4_fixed_vs_shrink.pdf', bbox_inches='tight')
36     fig.savefig(FIGURES_DIR/'q4_fixed_vs_shrink.png', dpi=180, bbox_inches='tight')
37     plt.close(fig)
38
39
40 if __name__=='__main__': main()

```

第三问敏感性程序

敏感性程序从原初值独立计算各情景，参数口径和结果见随附报告。新增后期水分量情景可单独复算：

python code/q3_sensitivity.py --only Ca_minus10pct Ca_plus10pct

```

1  """Bounded Q3 sensitivity analysis for late environment and hm."""
2  from __future__ import annotations
3
4  import argparse
5  import json
6  from pathlib import Path
7  import numpy as np
8
9  import problem3
10 import radial_coupled
11 from utils import H, HM
12
13 OUT = Path(problem3.RESULTS_DIR) / "diagnostics/q3_sensitivity"
14
15 # hm_scale is applied for the whole trajectory; Ta/Ca use the 4 h switch.
16 CASES = [
17     ("baseline", 50.0, 0.05, 1.0),
18     ("Ta_minus2", 48.0, 0.05, 1.0),
19     ("Ta_plus2", 52.0, 0.05, 1.0),
20     ("hm_minus20pct", 50.0, 0.05, 0.8),
21     ("hm_plus20pct", 50.0, 0.05, 1.2),
22     ("Ca_minus10pct", 50.0, 0.045, 1.0),
23     ("Ca_plus10pct", 50.0, 0.055, 1.0),
24 ]
25
26
27 def make_case_functions(Ta0, Ca0, hm_scale):
28     Ta_base, Ca_base, _ = problem3.load_q3_inputs()
29
30     class Late:
31         def __init__(self, base, after):
32             self.base, self.after = base, float(after)
33
34         def __call__(self, t):
35             if np.ndim(t) == 0:
36                 return float(self.base(t)) if float(t) <= problem3.T_SWITCH else self.after
37             t = np.asarray(t, dtype=float)
38             return np.where(t <= problem3.T_SWITCH, self.base(t), self.after)
39
40     return Late(Ta_base, Ta0), Late(Ca_base, Ca0), hm_scale

```

```

41
42
43 def run_case_sensitivity(level, Ta_fun, Ca_fun, hm_scale, tag):
44     """Use the production integrator with a scaled surface mass-transfer coefficient."""
45     original = problem3.CoupledRadialFVM.rhs
46
47     def rhs_scaled(self, t, y, Ta, Ca, **kwargs):
48         # run_case supplies boundary functions; scale only the prescribed hm.
49         n = self.n_nodes
50         T, C = self.unpack(y)
51         kn = np.asarray(radial_coupled.conductivity(C), dtype=float)
52         Bn = np.asarray(radial_coupled.heat_capacity(C), dtype=float)
53         Dn = np.asarray(radial_coupled.moisture_diffusivity_q2(C, T, protect=True), dtype=float)
54         kf = radial_coupled.harmonic_mean(kn[:-1], kn[1:])
55         Df = radial_coupled.harmonic_mean(Dn[:-1], Dn[1:])
56         FT = -self.r_half*kf*np.diff(T)/self.dr_face
57         FC = -self.r_half*Df*np.diff(C)/self.dr_face
58         td = np.empty_like(T); cd = np.empty_like(C)
59         td[0] = -FT[0]/(Bn[0]*self.W[0]); cd[0] = -FC[0]/self.W[0]
60         td[1:-1] = (FT[:-1]-FT[1:])/((Bn[1:-1]*self.W[1:-1]))
61         cd[1:-1] = (FC[:-1]-FC[1:])/self.W[1:-1]
62         radius = float(self.r[-1])
63         fts = radius*H*(T[-1]-Ta)
64         fcs = radius*(HM*hm_scale)*(C[-1]-Ca)
65         td[-1] = (FT[-1]-fts)/(Bn[-1]*self.W[-1]); cd[-1] = (FC[-1]-fcs)/self.W[-1]
66         return self.pack(td, cd)
67
68     problem3.CoupledRadialFVM.rhs = rhs_scaled
69     try:
70         return problem3.run_case(level, Ta_fun, Ca_fun, 72*3600., problem3.RTOL,
71                                 problem3.ATOL_T, problem3.ATOL_C, problem3.MAX_STEP_EARLY,
72                                 problem3.MAX_STEP_LATE, tag=tag, audit_balances=False)
73     finally:
74         problem3.CoupledRadialFVM.rhs = original
75
76
77 def run_one(tag, ta, ca, scale):
78     taf, caf, hs = make_case_functions(ta, ca, scale)
79     result = run_case_sensitivity(8, taf, caf, hs, tag)
80     record = {
81         "case": tag, "Ta_after_C": ta, "Ca_after": ca, "hm_scale": scale,
82         "hm_window": "full_horizon", "Ca_window": "after_4h", "Ta_window": "after_4h",
83         "t_star_s": float(result["t_star"]), "t_star_h": float(result["t_star"])/3600,
84         "M_star": float(result["event"]["M_star"]), "rmax_star_cm": float(result["event"]["rmax_star_cm"]),
85         "nodes": int(result["n_nodes"]), "rtol": problem3.RTOL,
86         "atol_T": problem3.ATOL_T, "atol_C": problem3.ATOL_C,
87     }
88     (OUT / f"{tag}.json").write_text(json.dumps(record, ensure_ascii=False, indent=2), encoding="utf-8")
89     np.savez_compressed(OUT / f"{tag}.npz", times=result["times"], Y=result["Y"], r=result["fvm"].r,
90                       t_star=result["t_star"], y_star=result["y_star"])
91     return record
92
93
94 def write_summary(rows):
95     base = next(r["t_star_h"] for r in rows if r["case"] == "baseline")
96     for row in rows:
97         row["relative_change_pct"] = 100.0 * (row["t_star_h"] - base) / base
98     (OUT / "summary.json").write_text(json.dumps(rows, ensure_ascii=False, indent=2), encoding="utf-8")
99     lines = ["case,Ta_after_C,Ca_after,hm_scale,t_star_h,relative_change_pct"]
100     lines += [f'{{r["case"]}},{{r["Ta_after_C"]}},{{r["Ca_after"]}},{{r["hm_scale"]}},{{r["t_star_h"]:.6f}},{{r["relative_change_pct"]:.3f}}' for r in rows]
101     (OUT / "summary.csv").write_text("\n".join(lines) + "\n", encoding="utf-8")
102     return rows
103
104

```

```

105 def main():
106     parser = argparse.ArgumentParser()
107     parser.add_argument("--only", nargs="*", default=None)
108     parser.add_argument("--rerun", action="store_true")
109     args = parser.parse_args()
110     OUT.mkdir(parents=True, exist_ok=True)
111     wanted = CASES if args.only is None else [c for c in CASES if c[0] in args.only]
112     rows = []
113     for tag, ta, ca, scale in wanted:
114         existing = OUT / f"{tag}.json"
115         if existing.exists() and not args.rerun:
116             rows.append(json.loads(existing.read_text(encoding="utf-8")))
117             continue
118         rows.append(run_one(tag, ta, ca, scale))
119     if args.only is not None:
120         by_tag = {r["case"]: r for r in rows}
121         ordered = []
122         for tag, ta, ca, scale in CASES:
123             path = OUT / f"{tag}.json"
124             if tag in by_tag:
125                 ordered.append(by_tag[tag])
126             elif path.exists():
127                 ordered.append(json.loads(path.read_text(encoding="utf-8")))
128         rows = ordered
129     write_summary(rows)
130     print(json.dumps(rows, ensure_ascii=False, indent=2))
131
132
133 if __name__ == "__main__":
134     main()

```

第三问二维端面校核程序与复现

本项采用独立二维求解器，保留第三问原初值、物性与环境。支撑材料附全部校核网格的 NPZ、配对曲线及验证 JSON；可直接运行下列最后两项复核和重绘，重算全部对照则依次执行完整命令。运行时需提供原附件 1。

```

python code/q3_axisymmetric.py --checks
python code/q3_axisymmetric.py --radial --nr 80
python code/q3_axisymmetric.py --radial --nr 160
python code/q3_axisymmetric.py --radial --nr 160 --tight
python code/q3_axisymmetric.py --nr 80 --nz 40 --end-scale 0
python code/q3_axisymmetric.py --nr 80 --nz 40
python code/q3_axisymmetric.py --nr 160 --nz 40
python code/q3_axisymmetric.py --nr 80 --nz 80
python code/q3_axisymmetric.py --nr 160 --nz 80
python code/q3_axisymmetric.py --nr 160 --nz 80 --tight
python code/verify_q3_axisymmetric.py
python code/plot_q3_axisymmetric.py

```

```

1 """Bounded Q3 end-face audit; node-centered annular FVM and streaming BDF."""
2 from __future__ import annotations
3
4 import argparse
5 import hashlib
6 import json

```

```

7 import platform
8 import time
9 from pathlib import Path
10
11 import numpy as np
12 import scipy
13 from numpy.polynomial.legendre import leggauss
14 from scipy.integrate import BDF, solve_ivp
15 from scipy.optimize import brentq
16 from scipy.sparse import coo_matrix
17 from threadpoolctl import threadpool_limits
18
19 from appendix3 import conductivity, heat_capacity, moisture_diffusivity_q2
20 from problem3 import load_q3_inputs, T_SWITCH, TA_AFTER, CA_AFTER
21 from radial_coupled import CoupledRadialFVM
22 from radial_fvm import harmonic_mean
23 from utils import ATTACH_DIR, C0, T0, R, L, H, HM, RESULTS_DIR
24
25 OUT = RESULTS_DIR / "q3_axisymmetric"
26 THRESHOLD = 0.15
27 CAP = 72.0 * 3600.0
28 Z = L / 2.0
29
30
31 def nodes(intervals: int, extent: float, power: float) -> np.ndarray:
32     if intervals < 4:
33         raise ValueError("At least four intervals are required")
34     u = np.linspace(0.0, 1.0, intervals + 1)
35     x = extent * (1.0 - (1.0 - u) ** power)
36     x[0], x[-1] = 0.0, extent
37     return x
38
39
40 class AxisymmetricFVM:
41     def __init__(self, r, z, end_scale=1.0):
42         self.r, self.z = np.asarray(r, float), np.asarray(z, float)
43         if (len(self.r) < 3 or len(self.z) < 3 or self.r[0] != 0 or self.z[0] != 0
44             or self.r[-1] != R or self.z[-1] != Z
45             or np.any(np.diff(self.r) <= 0) or np.any(np.diff(self.z) <= 0)
46             or end_scale < 0):
47             raise ValueError("Invalid half-cylinder grid or end coefficient")
48         self.nr, self.nz = len(self.r), len(self.z)
49         self.n = self.nr * self.nz
50         self.rf = (self.r[:-1] + self.r[1:]) / 2.0
51         self.zf = (self.z[:-1] + self.z[1:]) / 2.0
52         self.wr = np.diff(np.r_[0., self.rf, R] ** 2) / 2.0
53         self.wz = np.diff(np.r_[0., self.zf, Z])
54         self.volume = self.wr[:, None] * self.wz[None, :]
55         self.normalizer = R * R * Z / 2.0
56         self.end_scale = float(end_scale)
57         self.dr, self.dz = np.diff(self.r), np.diff(self.z)
58
59     def pack(self, T, C):
60         return np.stack((T, C), axis=-1).ravel()
61
62     def unpack(self, y):
63         a = np.asarray(y).reshape(self.nr, self.nz, 2)
64         return a[:, :, 0], a[:, :, 1]
65
66     def divergence(self, state, coefficient, ambient, exchange):
67         fr = np.empty((self.nr + 1, self.nz))
68         fz = np.empty((self.nr, self.nz + 1))
69         fr[0] = 0.0
70         fr[1:-1] = (-self.rf[:, None]
71                     * harmonic_mean(coefficient[:-1], coefficient[1:]))

```

```

72         * np.diff(state, axis=0) / self.dr[:, None])
73     fr[-1] = R * exchange * (state[-1] - ambient)
74     fz[:, 0] = 0.0
75     fz[:, 1:-1] = (-harmonic_mean(coefficient[:, :-1], coefficient[:, 1:]))
76         * np.diff(state, axis=1) / self.dz[None, :])
77     fz[:, -1] = self.end_scale * exchange * (state[:, -1] - ambient)
78     return -np.diff(fr, axis=0) / self.wr[:, None] - np.diff(fz, axis=1) / self.wz[None, :]
79
80     def rhs(self, t, y, Ta, Ca):
81         T, C = self.unpack(y)
82         B = heat_capacity(C)
83         k = conductivity(C)
84         D = moisture_diffusivity_q2(C, T, protect=True)
85         return self.pack(self.divergence(T, k, Ta, H) / B,
86             self.divergence(C, D, Ca, HM))
87
88     def loss_rates(self, y, Ca):
89         # Rates normalized by dry mass; the unknown uniform dry density cancels.
90         C = self.unpack(y)[1]
91         side = R * HM * np.dot(self.wz, C[-1] - Ca) / self.normalizer
92         end = self.end_scale * HM * np.dot(self.wr, C[:, -1] - Ca) / self.normalizer
93         return np.array([side, end])
94
95     def mean_C(self, y):
96         return float(np.sum(self.volume * self.unpack(y)[1]) / self.normalizer)
97
98     def sparsity(self):
99         ids = np.arange(self.n).reshape(self.nr, self.nz)
100         pairs = [(ids.ravel(), ids.ravel())]
101         for a, b in ((ids[:-1].ravel(), ids[1:].ravel()),
102             (ids[:, :-1].ravel(), ids[:, 1:].ravel())):
103             pairs.extend((a, b), (b, a)))
104         rows, cols = [], []
105         for a, b in pairs:
106             for i in (0, 1):
107                 for j in (0, 1):
108                     rows.append(2 * a + i)
109                     cols.append(2 * b + j)
110         row, col = np.concatenate(rows), np.concatenate(cols)
111         return coo_matrix((np.ones(len(row)), (row, col)), shape=(2*self.n, 2*self.n)).tocsc()
112
113
114     def structural_checks():
115         f = AxisymmetricFVM(nodes(12, R, 2), nodes(10, Z, 2.5))
116         rr, zz = f.r[:, None] / R, f.z[None, :] / Z
117         T = 32 + 8*rr**2 + 3*zz**2
118         C = 1.8 - .4*rr**2 - .3*zz**2
119         y = f.pack(T, C)
120         dy = f.rhs(0, y, 50., .05)
121         td, cd = f.unpack(dy)
122         mass = abs(np.sum(f.volume*cd)/f.normalizer + f.loss_rates(y,.05).sum())
123         heat_in = (R*H*np.dot(f.wz, 50-T[-1])
124             + H*np.dot(f.wr, 50-T[:, -1]))
125         heat = abs(np.sum(f.volume*heat_capacity(C)*td) - heat_in)
126
127         f0 = AxisymmetricFVM(f.r, f.z, 0.0)
128         radial = CoupledRadialFVM(f.r)
129         t1, c1 = 30 + 7*(f.r/R)**2, 2 - .5*(f.r/R)**2
130         uniform_z = f0.pack(np.broadcast_to(t1[:, None], (f.nr,f.nz)),
131             np.broadcast_to(c1[:, None], (f.nr,f.nz)))
132         two = f0.unpack(f0.rhs(0,uniform_z,50.,.05))
133         one = radial.unpack(radial.rhs(0,radial.pack(t1,c1),50.,.05))
134         reduction = max(float(np.max(abs(a-b[:,None])))) for a,b in zip(two,one))
135
136     # Constant-coefficient polynomial has a known cylindrical Laplacian (4+2).

```

```

137     q = rr**2 + zz**2
138     lap = f.divergence(q, np.ones_like(q), 0., 0.)
139     exact = 4/R**2 + 2/Z**2
140     # On a uniform grid the interior polynomial is exact; graded node averages are approximate.
141     fu = AxisymmetricFVM(np.linspace(0,R,13), np.linspace(0,Z,11))
142     q = (fu.r[:,None]/R)**2 + (fu.z[None,:]/Z)**2
143     lap = fu.divergence(q, np.ones_like(q),0.,0.)
144     polynomial = float(np.max(abs(lap[:-1,:-1]-exact))/exact)
145
146     # Check all structurally nonzero derivative locations against a numerical Jacobian.
147     pat = f.sparsity().toarray() != 0
148     missing = 0.0
149     eps = 1e-6
150     for j in range(len(y)):
151         shifted = y.copy()
152         shifted[j] += eps
153         diff = (f.rhs(0,shifted,50.,.05)-dy)/eps
154         missing = max(missing, float(np.max(abs(diff[~pat[:,j]]), initial=0.0)))
155     out = {"mass_rate_abs":float(mass), "effective_heat_rate_abs":float(heat),
156           "sealed_rhs_vs_independent_radial_max_abs":float(reduction),
157           "uniform_polynomial_relative_error":polynomial,
158           "jacobian_missing_entry_max_abs":missing,
159           "volume_relative_error":float(abs(f.volume.sum())/f.normalizer-1))}
160     out["pass"] = bool(mass < 1e-12 and heat < 1e-10 and reduction < 1e-10
161                       and polynomial < 1e-10 and missing < 1e-10)
162     return out
163
164
165 def provenance():
166     files = [Path(__file__), Path(__file__).with_name("appendix3.py"),
167              Path(__file__).with_name("problem3.py"), Path(__file__).with_name("utils.py"),
168              ATTACH_DIR / " 附件 1.xlsx", RESULTS_DIR / "q3_validation.json"]
169     return {str(p):hashlib.sha256(p.read_bytes()).hexdigest() for p in files}
170
171
172 def run(nr, nz, end_scale=1.0, tight=False, tag=None, cap=CAP):
173     OUT.mkdir(parents=True, exist_ok=True)
174     tag = tag or f"r{nr}_z{nz}_end{end_scale:g}" + ("_tight" if tight else "")
175     f = AxisymmetricFVM(nodes(nr,R,2.0), nodes(nz,Z,2.5), end_scale)
176     Ta_fun, Ca_fun, _ = load_q3_inputs()
177     rtol = 1e-8 if tight else 1e-7
178     atol = np.tile([1e-8 if tight else 1e-7, 1e-10 if tight else 1e-9],f.n)
179     y = f.pack(np.full((f.nr,f.nz),T0), np.full((f.nr,f.nz),C0))
180     sparsity = f.sparsity()
181     history = []
182     snapshots = {}
183     snap_times = [12*3600., 36*3600., 48*3600.]
184     loss4, loss8 = np.zeros(2), np.zeros(2)
185     quad4, weight4 = leggauss(4)
186     quad8, weight8 = leggauss(8)
187     max_mass_res = 0.
188     bounds = [T0,T0,C0,C0]
189     max_radial_increase = max_axial_increase = max_center_gap = 0.
190     sealed_axial_spread = 0.
191     event_t, event_y = None, None
192     accepted, nfev, njev, nlu = 0,0,0,0
193     wall = time.perf_counter()
194     print_clock = wall
195     next_sample = 0.
196
197     def record(t, state):
198         nonlocal max_radial_increase, max_axial_increase, max_center_gap, sealed_axial_spread
199         T,C = f.unpack(state)
200         loc = np.unravel_index(np.argmax(C),C.shape)
201         max_radial_increase = max(max_radial_increase,float(np.max(np.diff(C,axis=0))))

```

```

202     max_axial_increase = max(max_axial_increase, float(np.max(np.diff(C, axis=1))))
203     max_center_gap = max(max_center_gap, float(C.max() - C[0, 0]))
204     if end_scale == 0:
205         sealed_axial_spread = max(sealed_axial_spread, float(np.ptp(C, axis=1).max()),
206                                   float(np.ptp(T, axis=1).max()))
207     history.append([t, C.max(), C[0, 0], C[-1, 0], C[0, -1], T[0, 0],
208                   f.mean_C(state), f.r[loc[0]], f.z[loc[1]]])
209
210 for start, end in ((0., min(T_SWITCH, cap)), (T_SWITCH, cap)):
211     if start >= end or event_t is not None:
212         continue
213     late = start >= T_SWITCH
214     def environment(t):
215         return (TA_AFTER, CA_AFTER) if late else (float(Ta_fun(t)), float(Ca_fun(t)))
216     def rhs(t, state):
217         return f.rhs(t, state, *environment(t))
218     dt = (5. if tight else 10.) if not late else (30. if tight else 60.)
219     solver = BDF(rhs, start, y, end, rtol=rtol, atol=atol, max_step=dt, jac_sparsity=sparsity)
220     previous_M = float(f.unpack(y)[1].max())
221     while solver.status == "running":
222         a = solver.t
223         message = solver.step()
224         if solver.status == "failed":
225             raise RuntimeError(message)
226         accepted += 1
227         b = solver.t
228         dense = solver.dense_output()
229         T, C = f.unpack(solver.y)
230         if not np.all(np.isfinite(solver.y)) or C.min() <= 0:
231             raise RuntimeError("Nonfinite or nonpositive accepted state")
232         bounds = [min(bounds[0], float(T.min())), max(bounds[1], float(T.max())),
233                  min(bounds[2], float(C.min())), max(bounds[3], float(C.max()))]
234         current_M = float(C.max())
235         if previous_M > THRESHOLD and current_M <= THRESHOLD:
236             event_t = brentq(lambda t: float(f.unpack(dense(t))[1].max() - THRESHOLD), a, b,
237                             xtol=1e-6, rtol=1e-13)
238             event_y = dense(event_t)
239             b = event_t
240         for gx, gw, total in ((quad4, weight4, loss4), (quad8, weight8, loss8)):
241             q = (a+b)/2 + (b-a)/2 * gx
242             vals = dense(q)
243             rates = np.array([f.loss_rates(vals[:, j], environment(t)[1]) for j, t in enumerate(q)])
244             total += (b-a)/2 * np.sum(gw[:, None] * rates, axis=0)
245         state_b = event_y if event_t is not None else solver.y
246         residual = abs(f.mean_C(state_b) - C0 + loss8.sum())
247         max_mass_res = max(max_mass_res, residual)
248         while next_sample <= b + 1e-8:
249             if next_sample >= a - 1e-8:
250                 record(next_sample, dense(next_sample))
251             next_sample += 600.
252         for target in snap_times:
253             if a < target <= b:
254                 snapshots[f"t{int(target)}"] = dense(target).reshape(f.nr, f.nz, 2)
255         if time.perf_counter() - print_clock > 25:
256             print(f"{tag}: t={b/3600:.3f} h maxC={f.unpack(state_b)[1].max():.8f} "
257                   f"mass={residual:.2e} steps={accepted} "
258                   f"↪ wall={time.perf_counter() - wall:.1f}s", flush=True)
259             print_clock = time.perf_counter()
260         if event_t is not None:
261             record(event_t, event_y)
262             snapshots["event"] = event_y.reshape(f.nr, f.nz, 2)
263             break
264         previous_M = current_M
265     nfev += solver.nfev
266     njev += solver.njev

```

```

266     nlu += solver.nlu
267     y = solver.y.copy()
268     if event_t is None:
269         raise RuntimeError(f"Threshold not reached before {cap/3600:g} hours")
270     T,C = f.unpack(event_y)
271     loc = np.unravel_index(np.argmax(C),C.shape)
272     tolerance = 1e-7
273     wet = np.argwhere(C.max()-C <= tolerance)
274     meta = {"version":"q3-axisymmetric-v1","tag":tag,"nr":f.nr,"nz":f.nz,
275           "unknowns":2*f.n,"end_scale":end_scale,"tight":tight,
276           "r_mesh_power":2.0,"z_mesh_power":2.5,"rtol":rtol,
277           "atol_T":float(atol[0]),"atol_C":float(atol[1]),
278           "early_max_step_s":5. if tight else 10.,"late_max_step_s":30. if tight else 60.,
279           "t_star_s":float(event_t),"t_star_h":float(event_t/3600),
280           "event_max_C":float(C.max()),"event_center_C":float(C[0,0]),
281           "event_argmax_rz_m":[float(f.r[loc[0]]),float(f.z[loc[1]])],
282           "wet_region_tolerance_C":tolerance,
283           "event_wet_region_max_rz_m":[float(f.r[wet[:,0]].max()),float(f.z[wet[:,1]].max())],
284           "mean_C_event":f.mean_C(event_y),"side_loss_per_dry_mass":float(loss8[0]),
285           "ends_loss_per_dry_mass":float(loss8[1]),
286           "mass_balance_max_abs":max_mass_res,"mass_balance_relative_initial":max_mass_res/C0,
287           "mass_balance_relative_loss":max_mass_res/(C0-f.mean_C(event_y)),
288           "quadrature_4_vs_8_abs":float(np.max(abs(loss4-loss8))),
289           "bounds_Tmin_Tmax_Cmin_Cmax":bounds,
290           "sampled_max_radial_C_increase":max_radial_increase,
291           "sampled_max_axial_C_increase":max_axial_increase,
292           "sampled_max_C_minus_center":max_center_gap,
293           "sealed_sampled_axial_spread":sealed_axial_spread,
294           "accepted_steps":accepted,"nfev":nfev,"njev":njev,"nlu":nlu,
295           "wall_s":time.perf_counter()-wall,"python":platform.python_version(),
296           "numpy":np.__version__,"scipy":scipy.__version__,"source_sha256":provenance(),
297           "history_columns":["time_s","max_C","center_C","side_mid_C","end_axis_C",
298                           "center_T","mean_C","argmax_r_m","argmax_z_m"],
299           "limitations":["effective no-latent baseline","equal end/side coefficients in open case",
300                        "isotropic coefficients","identical end environments","Q3 only"]}
301     meta["numerical_checks_pass"] = bool(max_mass_res/C0 < 1e-6 and abs(C.max()-15) < 1e-9
302                                     and bounds[2] > 0 and bounds[3] < C0+1e-7)
303     np.savez_compressed(OUT/f"{tag}.npz",r=f.r,z=f.z,wr=f.wr,wz=f.wz,
304                       history=np.asarray(history),**snapshots)
305     (OUT/f"{tag}.json").write_text(json.dumps(meta,ensure_ascii=False,indent=2),encoding="utf-8")
306     print(json.dumps({k:meta[k] for k in ("tag","t_star_h","mass_balance_relative_loss",
307                                         "event_argmax_rz_m","wall_s","numerical_checks_pass")}),flush=True)
308     ↪ e)
309
310
311 def paired_radial(nr=80,tight=False):
312     f = CoupledRadialFVM(nodes(nr,R,2.0))
313     Ta,Ca,_ = load_q3_inputs()
314     y = f.pack(np.full(f.n_nodes,T0),np.full(f.n_nodes,C0))
315     histories = []
316     event_t = None
317     start_wall = time.perf_counter()
318     for a,b in ((0.,T_SWITCH),(T_SWITCH,CAP)):
319         late = a >= T_SWITCH
320         def rhs(t,state):
321             return f.rhs(t,state,TA_AFTER if late else float(Ta(t)),CA_AFTER if late else float(Ca(t)))
322         def event(t,state):
323             return float(f.unpack(state)[1].max()-15)
324         event.terminal,event.direction = True,-1
325         sol = solve_ivp(rhs,(a,b),y,method="BDF",rtol=1e-8 if tight else 1e-7,
326                       atol=np.r_[np.full(f.n_nodes,1e-8 if tight else 1e-7),
327                                np.full(f.n_nodes,1e-10 if tight else 1e-9)],
328                       max_step=(5. if tight else 10.) if not late else (30. if tight else 60.),
329                       jac_sparsity=f.jac_sparsity(),dense_output=True,events=event)

```

```

330         if not sol.success:
331             raise RuntimeError(sol.message)
332         ts = np.arange(a, sol.t[-1]+1e-8, 600.)
333         vals = sol.sol(ts)
334         histories.extend(np.column_stack([ts, vals[f.n_nodes:].max(axis=0), vals[0]]).tolist())
335         y = sol.y[:, -1]
336         if len(sol.t_events[0]):
337             event_t = float(sol.t_events[0][0])
338             histories.append([event_t, float(y[f.n_nodes:].max()), float(y[0])])
339         break
340     if event_t is None:
341         raise RuntimeError("Radial reference did not reach the threshold")
342     label = f"radial_r{nr}"+("_tight" if tight else "")
343     meta = {"tag":label, "nr":f.n_nodes, "tight":tight, "t_star_s":event_t, "t_star_h":event_t/3600,
344            "wall_s":time.perf_counter()-start_wall, "source_sha256":provenance()}
345     OUT.mkdir(parents=True, exist_ok=True)
346     np.savez_compressed(OUT/f"{label}.npz", r=f.r, event=y, history=np.asarray(histories))
347     (OUT/f"{label}.json").write_text(json.dumps(meta, ensure_ascii=False, indent=2), encoding="utf-8")
348     print(json.dumps(meta), flush=True)
349     return meta
350
351
352 if __name__ == "__main__":
353     parser = argparse.ArgumentParser()
354     parser.add_argument("--nr", type=int, default=80)
355     parser.add_argument("--nz", type=int, default=40)
356     parser.add_argument("--end-scale", type=float, default=1.)
357     parser.add_argument("--tight", action="store_true")
358     parser.add_argument("--checks", action="store_true")
359     parser.add_argument("--radial", action="store_true")
360     args = parser.parse_args()
361     with threadpool_limits(limits=1):
362         if args.checks:
363             checks = structural_checks()
364             OUT.mkdir(parents=True, exist_ok=True)
365             (OUT/"structural_checks.json").write_text(json.dumps(checks, indent=2), encoding="utf-8")
366             print(json.dumps(checks, indent=2))
367             if not checks["pass"]:
368                 raise SystemExit(1)
369         elif args.radial:
370             paired_radial(args.nr, args.tight)
371         else:
372             run(args.nr, args.nz, args.end_scale, args.tight)

1 """Audit Q3 dimensional comparisons without updating the four official solutions."""
2 from __future__ import annotations
3
4 import argparse
5 import csv
6 import hashlib
7 import json
8 from pathlib import Path
9
10 import numpy as np
11
12 from q3_axisymmetric import OUT
13 from utils import RESULTS_DIR
14
15
16 def read_case(label):
17     return json.loads((OUT/f"{label}.json").read_text(encoding="utf-8"))
18
19
20 def summarize(production="r160_z80_end1", tight="r160_z80_end1_tight"):
21     original = json.loads((RESULTS_DIR/"q3_validation.json").read_text(encoding="utf-8"))

```

```

22 structural = json.loads((OUT/"structural_checks.json").read_text(encoding="utf-8"))
23 labels = ["r80_z40_end1", "r160_z40_end1", "r80_z80_end1", "production", "tight"]
24 labels = list(dict.fromkeys(labels))
25 cases = {label: read_case(label) for label in labels}
26 sealed = read_case("r80_z40_end0")
27 radial80 = read_case("radial_r80")
28 rows = []
29 for label, d in cases.items():
30     radial = read_case(f"radial_r{d['nr']-1}" + ("tight" if d["tight"] else ""))
31     delta = d["t_star_s"] - radial["t_star_s"]
32     rows.append({"label": label, "nr": d["nr"], "nz": d["nz"], "t_star_h": d["t_star_h"],
33                 "paired_radial_h": radial["t_star_h"], "end_effect_s": delta,
34                 "end_effect_pct": 100*delta/radial["t_star_s"],
35                 "difference_from_official_s": d["t_star_s"] - original["t_star_s"],
36                 "mass_relative_loss": d["mass_balance_relative_loss"], "wall_s": d["wall_s"]})
37 base = cases[production]
38 numerical = {
39     "radial_refinement_at_z40_s": cases["r160_z40_end1"]["t_star_s"] - cases["r80_z40_end1"]["t_star_s"],
40     "radial_refinement_at_z80_s": cases["r160_z80_end1"]["t_star_s"] - cases["r80_z80_end1"]["t_star_s"],
41     "axial_refinement_at_r80_s": cases["r80_z80_end1"]["t_star_s"] - cases["r80_z40_end1"]["t_star_s"],
42     "axial_refinement_at_r160_s": cases["r160_z80_end1"]["t_star_s"] - cases["r160_z40_end1"]["t_star_s"],
43     "time_tightening_s": cases[tight]["t_star_s"] - base["t_star_s"],
44     "sealed_2d_vs_paired_1d_s": sealed["t_star_s"] - radial80["t_star_s"],
45 }
46 production_radial = read_case(f"radial_r{base['nr']-1}")
47 relative_to_original = 100*(base["t_star_s"] - original["t_star_s"])/original["t_star_s"]
48 source_checks = {}
49 for label, d in (**cases, "sealed": sealed, "radial80": radial80).items():
50     source_checks[label] = all(Path(p).is_file() and
51                               ↪ hashlib.sha256(Path(p).read_bytes()).hexdigest() == sha
52                               for p, sha in d["source_sha256"].items())
53 full_fields = {}
54 fine = np.load(OUT/"r160_z80_end1.npz")
55 for label, axis in (("r80_z80_end1", 0), ("r160_z40_end1", 1), (tight, None)):
56     other = np.load(OUT/f"{label}.npz")
57     errors = np.zeros(2)
58     for key in ("t43200", "t129600", "t172800"):
59         selected = fine[key][:, 2] if axis == 0 else fine[key][:, 2] if axis == 1 else fine[key]
60         errors = np.maximum(errors, np.max(abs(selected - other[key]), axis=(0, 1)))
61     full_fields[label] = {"max_T_difference_C": float(errors[0]), "max_C_difference": float(errors[1]),
62                          "times_h": [12, 36, 48], "comparison": "shared nodes at identical physical times"}
63 snapshots = np.load(OUT/f"{production}.npz")
64 radial_arrays = np.load(OUT/f"radial_r{base['nr']-1}.npz")
65 h2, h1 = snapshots["history"], radial_arrays["history"]
66 times, i2, i1 = np.intersect1d(h2[:, 0], h1[:, 0], return_indices=True)
67 difference = h2[i2, 1] - h1[i1, 1]
68 snapshot_stats = {}
69 for key in ("t43200", "t129600", "t172800", "event"):
70     C = snapshots[key][:, :, 1]
71     snapshot_stats[key] = {"center_C": float(C[0, 0]), "side_mid_C": float(C[-1, 0]),
72                           "end_axis_C": float(C[0, -1]), "corner_C": float(C[-1, -1]),
73                           "min_C": float(C.min()), "max_C": float(C.max())}
74 gates = {
75     "structural_checks": structural["pass"],
76     "source_hashes_match": all(source_checks.values()),
77     "sealed_trajectory_reduction": abs(numerical["sealed_2d_vs_paired_1d_s"]) < .1
78                                     and sealed["sealed_sampled_axial_spread"] < 1e-6,
79     "all_cases_mass_and_physical_checks": all(d["numerical_checks_pass"] for d in cases.values()),
80     "quadrature_check": all(d["quadrature_4_vs_8_abs"] < 1e-6 for d in cases.values()),
81     "radial_event_change_below_0_1pct": abs(numerical["radial_refinement_at_z80_s"])/base["t_star_s"] < .
82     ↪ 001,
83     "axial_event_change_below_0_1pct": abs(numerical["axial_refinement_at_r160_s"])/base["t_star_s"] < .0
84     ↪ 01,
85     "time_event_change_below_0_01pct": abs(numerical["time_tightening_s"])/base["t_star_s"] < .0001,

```

```

83     "event_at_center_with_tolerance":all(d["event_max_C"]-d["event_center_C"]<1e-7 for d in
    ↪ cases.values()),
84     "sampled_center_controls":all(d["sampled_max_C_minus_center"]<1e-7 for d in cases.values()),
85 }
86 summary = {"version":"q3-axisymmetric-v1","production":production,"time_check":tight,
87            "official_1d_t_star_h":original["t_star_h"],"official_1d_t_rep_h":original["t_rep_h"],
88            "production_2d_t_star_h":base["t_star_h"],
89            "production_paired_1d_t_star_h":production_radial["t_star_h"],
90            "production_vs_official_pct":relative_to_original,
91            "paired_end_effect_s":base["t_star_s"]-production_radial["t_star_s"],
92            "paired_end_effect_pct":100*(base["t_star_s"]-production_radial["t_star_s"])/production_radial["t_star_s"],
93            "paired_common_600s_max_C_difference":float(abs(difference).max()),
94            "common_samples":len(times),"snapshot_stats":snapshot_stats,"convergence":numerical,
95            "shared_node_field_checks":full_fields,
96            "sealed_axial_spread":sealed["sealed_sampled_axial_spread"],
97            "max_mass_relative_loss":max(d["mass_balance_relative_loss"] for d in cases.values()),
98            "production_mean_C_event":base["mean_C_event"],
99            "production_end_loss_fraction":base["ends_loss_per_dry_mass"]/(base["ends_loss_per_dry_mass"]
    ↪ +base["side_loss_per_dry_mass"]),
100           "wet_region_tolerance_C":base["wet_region_tolerance_C"],
101           "production_wet_region_max_rz_m":base["event_wet_region_max_rz_m"],
102           "rows":rows,"gates":gates,"pass":all(gates.values()),
103           "interpretation":"Dimensional sensitivity of the stipulated effective Q3 model; not
    ↪ experimental validation or a rigorous PDE error bound.",
104           "q4_scope":"Not extended; Q3 evidence does not validate the Q4 time-saving percentage.")
105 deltas = {row["label"]:row["end_effect_s"] for row in rows}
106 summary["paired_effect_convergence_s"] = {
107     "radial_at_z40":deltas["r160_z40_end1"]-deltas["r80_z40_end1"],
108     "radial_at_z80":deltas["r160_z80_end1"]-deltas["r80_z80_end1"],
109     "axial_at_r80":deltas["r80_z80_end1"]-deltas["r80_z40_end1"],
110     "axial_at_r160":deltas["r160_z80_end1"]-deltas["r160_z40_end1"],
111     "time":deltas[tight]-deltas[production],
112 }
113 (OUT/"comparison.json").write_text(json.dumps(summary,ensure_ascii=False,indent=2),encoding="utf-8")
114 with (OUT/"comparison.csv").open("w",newline="",encoding="utf-8-sig") as fh:
115     writer=csv.DictWriter(fh,fieldnames=list(rows[0]))
116     writer.writeheader()
117     writer.writerows(rows)
118 np.savetxt(OUT/"paired_history.csv",np.column_stack([times,h1[i1,1],h2[i2,1],difference]),delimiter=",",
    ↪ ",
119           header="time_s,max_C_1d,max_C_2d,difference_2d_minus_1d",comments="")
120 print(json.dumps(summary,ensure_ascii=False,indent=2))
121 if not summary["pass"]:
122     raise RuntimeError("Dimensional comparison verification failed")
123 return summary
124
125
126 if __name__=="__main__":
127     parser=argparse.ArgumentParser()
128     parser.add_argument("--production",default="r160_z80_end1")
129     parser.add_argument("--tight",default="r160_z80_end1_tight")
130     args=parser.parse_args()
131     summarize(args.production,args.tight)

1 """Plot verified Q3 end-face sensitivity using saved arrays only."""
2 import hashlib
3 import json
4
5 import numpy as np
6
7 from problem2 import setup_mpl
8 from q3_axisymmetric import OUT
9 from utils import FIGURES_DIR

```

```

10
11
12 def main():
13     summary=json.loads((OUT/"comparison.json").read_text(encoding="utf-8"))
14     if not summary["pass"]:
15         raise RuntimeError("Comparison must pass before plotting")
16     path=OUT/f"{summary['production']}.npz"
17     data=np.load(path)
18     history=np.loadtxt(OUT/"paired_history.csv",delimiter=",",skiprows=1)
19     C=data["t129600"][:, :, 1]
20     z=np.r_-data["z"][:0:-1],data["z"]]*100
21     r=np.r_-data["r"][:0:-1],data["r"]]*100
22     full=np.concatenate([C[: , :0:-1],C],axis=1)
23     full=np.concatenate([full[:0:-1],full],axis=0)
24     plt=setup_mpl()
25     fig=plt.figure(figsize=(7.2,3.55),layout="constrained")
26     gs=fig.add_gridspec(2,2,height_ratios=[.62,1.18],hspace=.06,wspace=.16)
27     field=fig.add_subplot(gs[0,:])
28     levels=np.linspace(np.floor(full.min()*100)/100,np.ceil(full.max()*100)/100,29)
29     im=field.contourf(z,r,full,levels=levels,cmap="viridis")
30     field.contour(z,r,full,levels=[.08,.12,.16,.18],colors="white",linewidths=.45,alpha=.65)
31     field.set_aspect("equal")
32     field.set(xlabel=" 轴向距离 $z$ / cm",ylabel=" 横向位置 / cm",xlim=(-12.5,12.5),ylim=(-2,2))
33     field.text(.01,1.04,"(a) 36 h 含水率分布",transform=field.transAxes,fontsize=9)
34     cb=fig.colorbar(im,ax=field,orientation="horizontal",fraction=.08,pad=.10,aspect=48)
35     cb.set_label(" 干基含水率 / (kg/kg)",fontsize=8)
36     cb.set_ticks([.05, .08, .11, .14, .17])
37     cb.ax.xaxis.set_major_formatter("{x:.2f}")
38     cb.ax.tick_params(labelsize=8)
39     ax=fig.add_subplot(gs[1,0])
40     ax.plot(history[:,0]/3600,history[:,1],color="#B65B3B",lw=1.8,label=" 配对一维")
41     ax.plot(history[:,0]/3600,history[:,2],color="#247B87",lw=1.3,ls="--",label=" 二维开放端面")
42     ax.axhline(.15,color=".45",lw=.8,ls=":",label=" 阈值 0.15")
43     ax.set(xlabel=" 时间 / h",ylabel=" 全域最大含水率 / (kg/kg)",xlim=(0,60),ylim=(0,2.65))
44     ax.text(.02,1.03,"(b) 全域最大含水率",transform=ax.transAxes,fontsize=9)
45     ax.legend(frameon=False,fontsize=8)
46     diff=fig.add_subplot(gs[1,1])
47     diff.plot(history[:,0]/3600,history[:,3]*1e5,color="#247B87",lw=1.3)
48     diff.axhline(0,color=".5",ls=":",lw=.8)
49     diff.set(xlabel=" 时间 / h",ylabel=" 二维减一维 / ($10^{-5}$ kg/kg)",xlim=(0,60))
50     diff.text(.02,1.03,"(c) 同时刻、同径向网格的差异",transform=diff.transAxes,fontsize=9)
51     for axis in (ax,diff):
52         axis.spines[["top","right"]].set_visible(False)
53     FIGURES_DIR.mkdir(exist_ok=True)
54     for ext in ("pdf","png"):
55         fig.savefig(FIGURES_DIR/f"q3_axisymmetric_check.{ext}",dpi=200,bbox_inches="tight")
56     plt.close(fig)
57     evidence={"figure":"figures/q3_axisymmetric_check.pdf","field_time_s":129600,
58             "sources":{str(p):hashlib.sha256(p.read_bytes()).hexdigest()
59                       for p in (path,OUT/"paired_history.csv",OUT/"comparison.json")},
60             "field_reconstruction":"Mirror the symmetric half-cylinder across z=0 and transverse axis.",
61             "curve_comparison":"Matched radial mesh; common exact 600 s sample times only."}
62     (OUT/"figure_provenance.json").write_text(json.dumps(evidence,ensure_ascii=False,indent=2),encoding="u
↪ tf-8")
63
64
65 if __name__=="__main__":
66     main()

```

附录 C 问题一温度参考解的推导

本附录补充第 5.5 节温度参考解的构造过程。先求恒定环境下的阶跃响应，再对环境温度的连续变化进行叠加。

当烘房温度恒为 T_f 时，令 $u(r, t) = T(r, t) - T_f$ ，取分离变量形式 $u = X(r)G(t)$ 。将其代入常热物性导热方程，得到

$$\frac{G'}{\alpha G} = \frac{X'' + X'/r}{X} = -\frac{\lambda^2}{R^2}.$$

时间部分为 $G(t) = \exp(-\alpha\lambda^2 t/R^2)$ ，径向部分满足

$$r^2 X'' + rX' + \lambda^2 \frac{r^2}{R^2} X = 0.$$

该方程为零阶 Bessel 方程。中心有界性排除第二类 Bessel 函数，取 $X(r) = J_0(\lambda r/R)$ 。由 $J'_0 = -J_1$ ，将其代入表面条件 $-ku_r(R, t) = hu(R, t)$ ，即得正文式(12)。

以 r 为权展开均匀初始温差 $28 - T_f$ 。令 $x = r/R$ ，利用径向模态的正交性，得到

$$A_n = \frac{\int_0^1 x J_0(\lambda_n x) dx}{\int_0^1 x J_0^2(\lambda_n x) dx} = \frac{2J_1(\lambda_n)}{\lambda_n [J_0^2(\lambda_n) + J_1^2(\lambda_n)]}.$$

因此，恒定环境下的解为

$$T(r, t) = T_f + (28 - T_f) \sum_{n=1}^{\infty} A_n J_0(\lambda_n r/R) e^{-\beta_n t}, \quad \beta_n = \alpha\lambda_n^2/R^2.$$

对随时间变化的 $T_a(t)$ ，利用热方程的线性性，将环境变化分解为一系列微小阶跃。单位环境阶跃的温度响应为

$$H(r, t) = 1 - \sum_{n=1}^{\infty} A_n J_0(\lambda_n r/R) e^{-\beta_n t}.$$

由 $T_a(0) = 28$ 及 Duhamel 叠加，有

$$T_{\text{ref}}(r, t) = 28 + \int_0^t H(r, t - \tau) T'_a(\tau) d\tau.$$

代入 H 并利用 $\int_0^t T'_a(\tau) d\tau = T_a(t) - 28$ ，即可得到正文式(13)。

计算中对 PCHIP 插值函数求导。各阶卷积积分记为

$$I_n(t) = \int_0^t e^{-\beta_n(t-\tau)} T'_a(\tau) d\tau,$$

对时间求导可将其改写为相互独立的模态常微分方程

$$\dot{I}_n = -\beta_n I_n + T'_a(t), \quad I_n(0) = 0.$$

由此得到独立于径向有限体积离散的温度参考计算，并通过增加模态数检查级数截断的影响。